

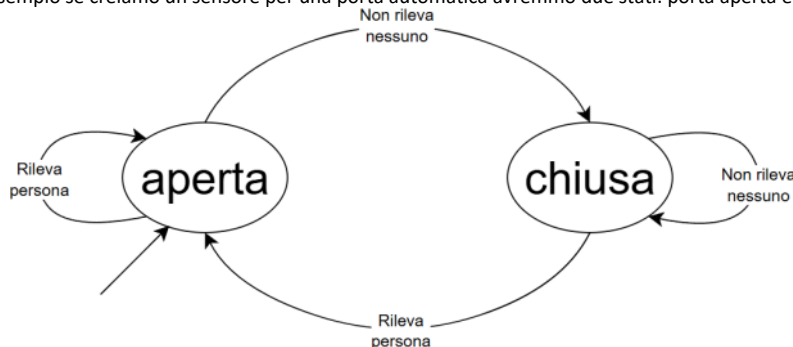
Automa a Stati Finiti (DFA)

mercoledì 24 settembre 2025 15:43

Un **automa a stati finiti (DFA)** è un modello computazionale semplice che prende un input finito e lo "**scorre**" **sequenzialmente** carattere per carattere.

Un automa è un insieme di **stati e transizioni** che collegano i vari stati.

Ad esempio se creiamo un sensore per una porta automatica avremmo due stati: porta aperta e porta chiusa



- Dallo stato "**aperta**" (che è anche quello iniziale):
 - Se **rileva una persona**, allora deve aprire la porta, ma essendo già aperta rimane sullo stesso stato (**loop**)
 - Se invece **non rileva nessuno**, chiude la porta, passando allo stato "**chiusa**"
- Dallo stato "**chiusa**":
 - Se **rileva una persona**, allora apre la porta, passando allo stato "**aperta**"
 - Se **non rileva nessuno**, rimane chiusa, quindi rimane allo stesso stato (**loop**)

Definizione di DFA (Deterministic Finite Automa)

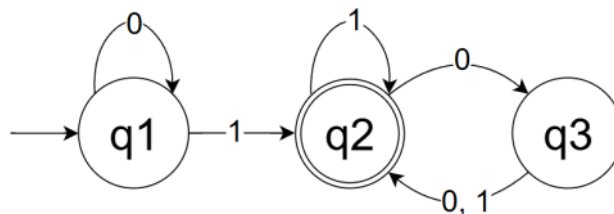
Una **DFA** è una tupla $(Q, \Sigma, \delta, q_0, F)$:

- Q è l'insieme finito degli stati
- Σ è l'insieme finito dei caratteri dell'input
- δ è una **funzione di transizione** (più semplicemente le frecce dell'automa): $Q \times \Sigma \rightarrow Q$.
Se abbiamo $\delta(q_x, a) = q_y$, indica che partiamo dallo stato q_x e ci spostiamo sulla freccia "a" fino allo stato q_y
- q_0 è lo stato iniziale di partenza
- F è l'insieme degli stati di accettazione dell'automa

Esempio più astratto di una DFA è:

- $Q = \{q_1, q_2, q_3\}$
- $\Sigma = \{0, 1\}$
- $q_0 = q_1$
- $F = \{q_2\}$

$\delta(Q, \Sigma)$	0	1
q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_2	q_2



Qui abbiamo che q_1, q_2 e q_3 sono i 3 stati dell'automa dove q_1 è lo stato di partenza e q_2 , segnato col doppio cerchio (o con delle sbarre nel cerchio semplice), è lo **stato di accettazione** e indica lo stato in cui deve **terminare l'esecuzione** dell'automa per poter definire **valido l'input**.
Se la computazione finisce al di fuori dello stato di accettazione allora l'input non è valido.

Es con stringa **w = 11101**

$q_1 \rightarrow q_2 \rightarrow q_2 \rightarrow q_2 \rightarrow q_3 \rightarrow q_2$

Quindi finiamo in q_2 che è lo stato di accettazione quindi la stringa è valida

Es con stringa **w = 111010**

$q_1 \rightarrow q_2 \rightarrow q_2 \rightarrow q_2 \rightarrow q_3 \rightarrow q_2 \rightarrow q_3$

In questo caso finiamo in q_3 e poiché non è stato di accettazione la stringa non è valida

(Attenzione: i calcoli sequenziali dei caratteri vanno visti guardando le frecce e non gli stati. Ad esempio con $w = 11101$ abbiamo 5 caratteri ma abbiamo visitato 6 stati poiché il primo è di partenza)

Il **linguaggio dell'automa** è un insieme di stringhe finite o infinite. Il linguaggio senza stringhe è il **linguaggio vuoto** $\emptyset$

Sia M una DFA, chiamiamo $L(M)$ l'insieme delle stringhe che M accetta, cioè che fanno terminare M su uno stato di accettazione.

Usando l'esempio precedente avremmo:

$L(M) = \{w : w \text{ contiene almeno un "1" e un num. pari di "0" dopo l'ultimo "1"}\}$

usando questa formula per filtrare i nostri input avremmo sempre stringhe accettate da M poiché finiranno tutte in q_2

Funzione di Transizione Estesa

Mentre la funzione di transizione δ normale legge un carattere solo, la **funzione di transizione estesa** δ^* può leggere **più caratteri insieme**

$\delta^*: Q * \Sigma^* \rightarrow Q$

con $\Sigma = \{0, 1\}$, $\Sigma^* = \{\epsilon, 0, 1, 00, 01, 10, 11, 000, \dots, 101001, \dots\}$

$$\begin{cases} \delta^*(q, \epsilon) = \delta(q, \epsilon) \\ \delta^*(q, ax) = \delta^*(\delta(q, a), x) \end{cases}$$

con $a \in \Sigma$ (singolo car.), $x \in \Sigma^*$ (insieme successivo di car.), ϵ = **stringa vuota**

cioè:

- Il δ^* della stringa vuota è il δ normale della stringa vuota
- Il δ^* di una stringa è il δ^* del δ con il primo carattere della stringa e il resto della stringa (cioè si **calcola** in modo ricorsivo **un carattere alla volta**)

La **configurazione** di una DFA è una "**fotografia**" dell'automa ad un certo punto della sua esecuzione, ed è la coppia:

$Q * \Sigma^*$

che rappresenta uno stato e la stringa di input che rimane da leggere

Il **passo di computazione** è una **transizione** da una **configurazione ad un'altra** rispettando δ (le transizioni degli stati) ed è una **relazione binaria** $\vdash_M: Q * \Sigma^* \rightarrow Q * \Sigma^*$ t.c.:

$(p, ax) \vdash_M (q, x) \Leftrightarrow \delta(p, a) = q$ dove $p, q \in Q, a \in \Sigma, x \in \Sigma^*$

E possiamo estendere $\vdash_M$ considerando la sua **chiusura riflessiva e transitiva** $\vdash_M^*$:

1) $(q, x) \vdash_M^* (q, x)$

$$2) (q, aby) \vdash_M^* (p, by) \wedge (p, by) \vdash_M^* (r, y) \Rightarrow (q, aby) \vdash_M^* (r, y)$$

dove $q, p, r \in Q, a, b \in \Sigma, y \in \Sigma^*$

Definizione di Linguaggio Accettato

Diciamo che $x \in \Sigma^*$ è **accettato** dalla DFA M se:

$$\delta^*(q_0, x) \in F$$

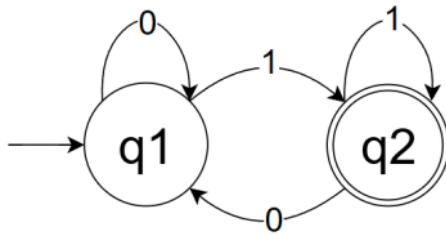
oppure

$$(q_0, x) \vdash_M^* (q, \epsilon)$$

In altre parole:

$$L(M) = \{x \in \Sigma^* : \delta^*(q_0, x) \in F\}$$

Cioè che la stringa x di partenza porta ad uno stato di accettazione



$$(q_1, 011) \vdash_M (q_1, 11)$$

$$(q_1, 11) \vdash_M (q_2, 1)$$

$$(q_2, 1) \vdash_M (q_2, \epsilon)$$

$$\Rightarrow (q_1, 011) \vdash_M^* (q_2, \epsilon) = \delta^*(q_1, 011) = q_2$$

e siccome $q_2 \in F, w = 011 \in L(M)$

Linguaggi Regolari

sabato 27 settembre 2025 19:52

Un linguaggio è detto **regolare (REG)** se esiste una DFA che lo riconosce

$$REG = \{L \subseteq \Sigma^* : \exists \text{ DFA } M \text{ t.c. } L(M) = L\}$$

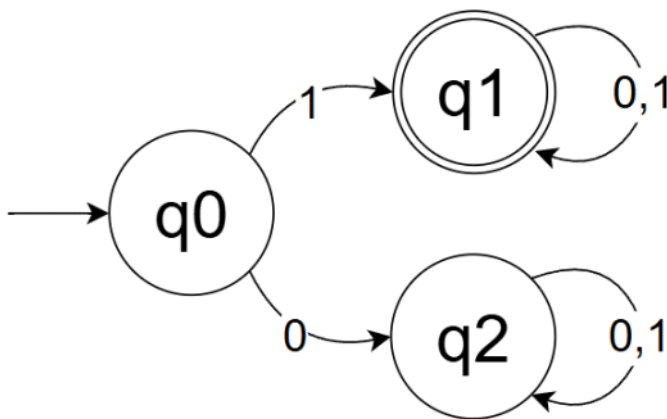
Cioè REG è un linguaggio L qualsiasi su Σ^* t.c esiste una DFA che riconosce L.

Vogliamo capire come progettare una DFA per un dato linguaggio.

Es:

$$L = \{x \in \{0, 1\}^* : x = 1y, \quad y \in \{0, 1\}^*\}$$

cioè x = tutte le stringhe che **iniziano con 1** (y sono i caratteri successivi)



Da q_0 :

- Se leggo **1 (mi serve)**, vado nello stato **accettante** q_1 e rimango la poiche è una **stringa valida**
- Se leggo **0 (non mi serve)** quindi vado in q_2 (**stato pozzo**, poiche vado la senza poter uscire) e rimango la

Invece togliendo q_2 e mettendo una transizione loop da q_0 a q_0 se abbiamo 0, non va bene perche se ci capita "01" passiamo per $q_0 \rightarrow q_0 \rightarrow q_1$ quindi alla fine viene accettato pero non è una stringa valida siccome inizia con "0" e non con "1"

Se **non è definita una transizione** da uno stato ad un altro allora significa che la stringa finisce in quello stato e non è valida, siccome non può passare ad uno stato accettante

Lo stato pozzo, può essere anche rimosso se togliamo anche la transizione $\delta(q_0, q_2)$ (quindi rimane solo la transizione $\delta(q_0, q_1)$)

Prova di correttezza

DFA accetta $x \Leftrightarrow x \in L$

Osserviamo:

$$\delta^*(q_1, u) = q_1 \quad \forall u \in \{0, 1\}^* \text{ (per ogni transizione da } q_1 \text{ torniamo in } q_1)$$

$$\delta^*(q_2, u) = q_2 \quad \forall u \in \{0, 1\}^* \text{ (stessa cosa per } q_2)$$

Per induzione dimostriamo $x \in L \Leftrightarrow \text{DFA accetta } x$

- Base:** $|x| = 0$, se $x = \epsilon$, $\delta^*(q_0, \epsilon) = \delta(q_0, \epsilon) = q_0 \notin F$ (ricordiamo ϵ = stringa vuota)
- Induttivo:** Sia $n \geq 0$, supponiamo che $|w| \leq n$

$$\delta^*(q_0, w) = \begin{cases} q_0 & \text{se } w = \epsilon \\ q_1 & \text{se } w \text{ inizia con } 1 \\ q_2 & \text{se } w \text{ inizia con } 0 \end{cases}$$

Prendo x t.c. $|x| = n+1$ e lo penso $x = au$ con $a \in \{0, 1\}$ e $u \in \{0, 1\}^n$ (cioè a è il primo carattere e u i successivi)

$$\delta^*(q_0, x) = \delta^*(q_0, au) = \delta^*(\delta(q_0, a), u)$$

Ora abbiamo due casi:

- $\delta(q_0, a) = q_2$ se $a = 0$
- $\delta(q_0, a) = q_1$ se $a = 1$

e per osservazione iniziale (cioè che rimani in q_1 se vai in q_1 , stessa cosa per q_2)

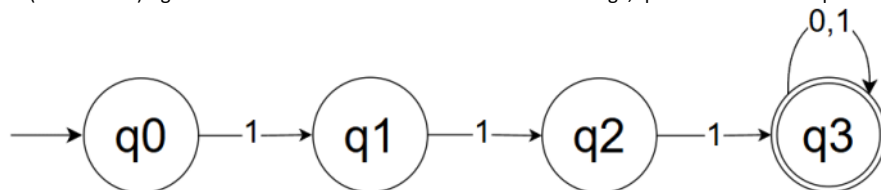
$$\Rightarrow \delta^*(q_0, x) = q_1 \Leftrightarrow a = 1$$

Esercizi

- $L = \{x \in \{0, 1\}^* : w_h(x) \geq 3\}$ dove $w_h(x)$ = num. di "1" in $x \in \{0, 1\}^*$

(Riconoscere tutte le stringhe che hanno hamming weight (num. di caratteri uguali a "1") almeno 3, cioè ci sono almeno 3 "1")

Usiamo 4 stati a catena dove il primo è quello di partenza e l'ultimo è quello accettante. Ci spostiamo da uno stato al successivo solo se incontriamo un "1" e quando arriviamo all'ultimo (l'accettante) significa che abbiamo trovato almeno 3 "1" nella stringa, quindi rimaniamo in questo stato finché non finisce la stringa

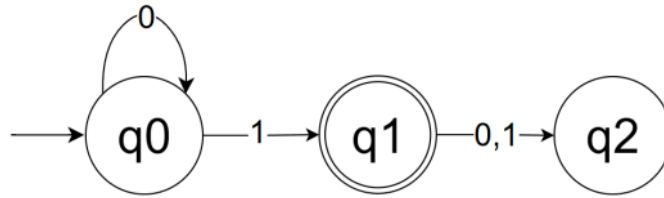


- $L = \{x \in \{0, 1\}^* : x = 0^n 1 \text{ con } n \in \mathbb{N}\}$

(Stringa di n "0" e l'ultimo carattere è "1")

Il num. di "0" può essere da 0 a ∞ , quindi dallo stato di partenza rimaniamo in quello stato se incontriamo "0" e appena incontriamo il primo "1" andiamo subito allo stato accettante e abbiamo due casi:

- Se è l'ultimo carattere, allora la stringa è **valida** poiche contiene n "0" e l'"1" finale
- Se incontriamo altri caratteri dopo quell'"1" allora andiamo in uno stato pozzo, poiche in questo caso la stringa non è valida



Operazioni sui linguaggi

Fissiamo $\Sigma = \{0, 1\}$. Per $n \in \mathbb{N}$, $[n]$ (cioè definiamo $n = \{1, 2, \dots, n\}$)

Siccome i linguaggi sono insiemi di stringhe posso considerare su di essi delle **operazioni** (come con gli insiemi):

- **Unione:** $L_1 \cup L_2 = \{x \in \Sigma^* : x \in L_1 \text{ oppure } x \in L_2\}$ (la stringa x appartiene a L_1 oppure a L_2)
- **Intersezione:** $L_1 \cap L_2 = \{x \in \Sigma^* : x \in L_1 \text{ e } x \in L_2\}$ (la stringa x appartiene a L_1 e anche a L_2)
- **Complemento:** $L^c = \{x \in \Sigma^* : x \notin L\}$ (la stringa x non appartiene a L)

Concatenazione di stringhe: Se abbiamo

$$x = a_1, \dots, a_n; y = b_1, \dots, b_n \text{ dove } n, m > 0$$

allora

$$xy = a_1, \dots, a_n, b_1, \dots, b_m \in \Sigma^*$$

$$\epsilon x = x \epsilon = x$$

$$\Rightarrow \begin{cases} x\epsilon = x \\ x(ya) = (xy)a \end{cases}$$

$$\text{dove } x, y \in \Sigma^*, a \in \Sigma$$

si concatena xy con a (ultimo carattere)

Concatenazione di linguaggi

$$L_1 \circ L_2 = \{xy : x \in L_1 \text{ e } y \in L_2\}$$

(simbolo $\circ$ = concatenazione)

Esempio:

$$\Sigma = \{a, b\}$$

$$L_1 = \{a, ab, ba\}$$

$$L_2 = \{ab, b\}$$

$$L_1 \circ L_2 = \{aab, ab, abab, abb, baab, bab\}$$

(concatenare ogni elemento di L_1 per ogni elemento di L_2) (prodotto cartesiano)

La **potenza** è un caso speciale. Per le stringhe $x^n = x, \dots, x$ n volte. $x \in \Sigma^*$

$$\begin{cases} x^0 = \epsilon \\ x^{n+1} = x^n x \end{cases}$$

Lo stesso per i linguaggi

$$\begin{cases} L^0 = \{\epsilon\} \\ L^{n+1} = L^n \circ L \end{cases}$$

Es:

$$L = \{a, ab, ba\}$$

$$L^2 = \{aa, aab, aba, abab, abba, baa, baab, baba\}$$

L^* sarebbe:

$$L^* = \bigcup_{n \geq 0} L^n = \{\epsilon\} \cup L^1 \cup L^2 \cup \dots$$

Es: $L = \{a, b\}$

$$L^* = \{\epsilon, a, b, aa, ab, ba, bb, aaa, aab, aba, abb, baa, \dots\}$$

Vogliamo studiare le proprietà di **chiusura** dei linguaggi regolari

Ovvero: se $L_1, L_2 \in \text{REG}$, posso dire che $L_1 \cup L_2 \in \text{REG}$? o $L_1 \cap L_2 \in \text{REG}$? o $L_1^c \in \text{REG}$? o $L^* \in \text{REG}$?

Teorema: REG è chiusa per unione

- Intuizione: $L_1, L_2 \in \text{REG} \Rightarrow \exists M_1, M_2 \text{ DFA t.c. } L(M_1) = L_1, L(M_2) = L_2$

Devo definire M t.c. $L(M) = L_1 \cup L_2$ (cioè che x sta in L_1 oppure in L_2)

Problema: Dato x candidato non posso prima provare a vedere se $M_1(x)$ accetta (perché dopo che esegue l'automa M_1 poi finisce la stringa e non ha più nulla per M_2)

Quindi bisogna provarlo in **parallelo**, quindi M deve **eseguire M_1 e M_2 in parallelo** e **accettare $\Leftrightarrow$ uno dei due accetta**

Così ci sarà uno stato per ogni coppia di stati M_1 e M_2

Dimostrazione:

$$M_1 = (Q_1, \Sigma, \delta_1, q_0^1, F_1)$$

$$M_2 = (Q_2, \Sigma, \delta_2, q_0^2, F_2)$$

t.c. $L(M_1) = L_1$ e $L(M_2) = L_2$

Costruiamo una $M = (Q, \Sigma, \delta, q_0, F)$ che simula contemporaneamente l'esecuzione di D_1 e D_2 , in cui gli stati possibili saranno le possibili combinazioni di coppie di stati.

$$\bullet Q = \{(r_1, r_2) : r_1 \in Q_1, r_2 \in Q_2\} = Q_1 \times Q_2$$

$$\bullet \delta = Q \times \Sigma \rightarrow Q$$

$$\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a)) \quad \text{dove } a \in \Sigma, (r_1, r_2) \in Q \quad (\text{transizione estesa di uno stato coppia, si calcola in parallelo})$$

$$\bullet F = \{(r_1, r_2) : r_1 \in F_1 \vee r_2 \in F_2\} = (F_1 \times Q_2) \cup (Q_1 \times F_2)$$

(Se dovessimo dimostrare $L = L_1 \cap L_2$ dovremmo usare $F = F_1 \times F_2$)

Correttezza tramite induzione per dimostrare la seguente proprietà:

$$\delta^*(q_0, x) = \delta^*((q_0^1, q_0^2), x) = (\delta_1^*(q_0^1, x), \delta_2^*(q_0^2, x)) \quad \forall x \in \Sigma^*$$

Dobbiamo dimostrare che per ogni $x \in \Sigma^*$, vale che la transizione estesa dello stato coppia di partenza è uguale alla coppia degli stati raggiunti separatamente dagli automi M_1 e M_2

- **Caso Base:** $|x| = 0$, se $x = \epsilon$ allora $\delta^*(q, \epsilon) = q$, quindi:

- $\delta^*((q_0^1, q_0^2), \epsilon) = (q_0^1, q_0^2)$
- $(\delta_1^*(q_0^1, \epsilon), \delta_2^*(q_0^2, \epsilon)) = (q_0^1, q_0^2)$

Quindi la proprietà è vera per ϵ

- **Passo induttivo:** assumiamo che la proprietà sia vera per una stringa arbitraria di lunghezza n , prendo x t.c. $|x| = n+1$ e lo penso $x = au$ con $a \in \{0, 1\}$ e $u \in \{0, 1\}^n$
 - Per definizione della funzione estesa:
 $\delta^*((q_0^1, q_0^2), ua) = \delta(\delta^*((q_0^1, q_0^2), u), a)$
 - Applicando la proprietà da dimostrare abbiamo:
 $\delta(\delta^*((q_0^1, q_0^2), u), a) = \delta(\delta_1^*(q_0^1, u), \delta_2^*(q_0^2, u), a)$
 - Usando la definizione di δ di M , cioè $\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$:
 $\delta(\delta_1^*(q_0^1, u), \delta_2^*(q_0^2, u), a) = (\delta_1(\delta_1^*(q_0^1, u), a), \delta_2(\delta_2^*(q_0^2, u), a))$
 - Riapplicando la funzione estesa al contrario abbiamo $\delta_1(\delta_1^*(q_0^1, u), a) = \delta_1^*(q_0^1, ua)$ (stessa cosa per δ_2^*):
 $(\delta_1(\delta_1^*(q_0^1, u), a), \delta_2(\delta_2^*(q_0^2, u), a)) = (\delta_1^*(q_0^1, ua), \delta_2^*(q_0^2, ua))$
 - Quindi abbiamo che:
 $\delta^*((q_0^1, q_0^2), ua) = (\delta_1^*(q_0^1, ua), \delta_2^*(q_0^2, ua))$

Quindi la proprietà è vera per il passo induttivo

$\forall x \in \Sigma^*$ abbiamo

$$x \in L(M) \Leftrightarrow x \in L_1 \cup L_2$$

$\Rightarrow$ Se $x \in L(M)$, allora

$$\delta^*(q_0, x) = (p, q) \in F = (F_1 \times Q_2) \cup (Q_1 \times F_2)$$

dove

$$p = \delta_1^*(q_0^1, x) \text{ e } q = \delta_2^*(q_0^2, x)$$

$\Rightarrow p \in F_1$ oppure $q \in F_2$

$\Rightarrow x \in L_1$ oppure $x \in L_2$

$\Leftarrow$ se $x \in L_1$, $\delta^*(q_0^1, x) \in F_1$ poichè:

$$\delta^*(q_0, x) = (\delta_1^*(q_0^1, x), \delta_2^*(q_0^2, x)) \in F_1 \times Q_2 \subseteq F \Rightarrow M \text{ accetta } x$$

Stessa cosa se $x \in L_2$

Quindi $x \in L(M) \Leftrightarrow x \in L_1 \cup L_2$

Teorema: REG è chiusa per complemento

Sia L un REG e M un automa che accetta $L(M) = L$. Vogliamo dimostrare che esiste un automa che accetti $L^c = \{x \in \Sigma^* \mid x \notin L\}$

Considero $M' = (Q, \Sigma, \delta, q_0, F^c)$ dove $F^c = Q/F$ (cioè gli stati di Q senza gli stati di accettazione di F) quindi gli stati di accettazione sono tutti quelli di Q diversi da F , poichè se usassimo gli stati di F possiamo solo accettare L e non L^c .

Se $x \in L^c$, se dato x in input a M' allora finiremo in uno stato di F^c poichè se finisce in F allora sarebbe accettato da M ma L^c contiene solo stringhe che non sono accettate da M , quindi:

- $x \in L^c$
- La computazione termina in uno stato di F^c
- M' accetta x

Quindi L^c è un **linguaggio regolare**

Non Determinismo

venerdì 3 ottobre 2025 19:01

Per dimostrare la **concatenazione** $L_1 \circ L_2 \in \text{REG}$, dato x devo "capire" come **spezzare** x in $x_1 \circ x_2 = x_1 x_2$ t.c. $x_1 \in L_1, x_2 \in L_2$.

Per risolvere questo problema introduciamo:

Non Determinismo

Nella computazione **deterministica** abbiamo $D \in \text{DFA}$ e D legge $a \in \Sigma$ nello stato $q \in Q$, allora D può andare in un **unico altro stato** $p \in Q$.

Nel caso **non deterministico**:

- Quando l'automa è in q e legge a può andare in **diversi stati** $p_1, p_2, \dots$
- Sono ammessi " **ϵ -archi**" che consentono di **aprire rami** di computazioni **parallele senza leggere la stringa**, quindi anche **prima o dopo** aver letto un simbolo, l'automa può spostarsi in altri stati
- **Accettazione**: accetto solo se esiste **almeno un ramo che accetta**

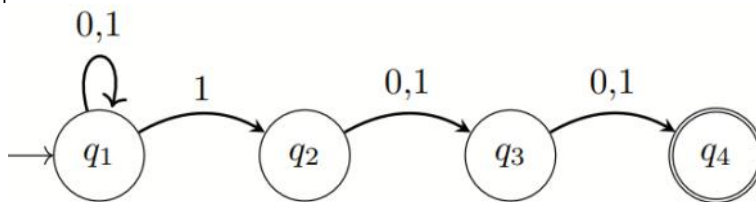
Definizione di NFA (Non-Deterministic Finite Automa)

Un **NFA** è una tupla $(Q, \Sigma, \delta, q_0, F)$ dove:

- Q, Σ, q_0, F sono come nel DFA
- $\delta: Q \times \Sigma \rightarrow \mathcal{P}(Q)$
 - $\Sigma = \Sigma \cup \{\epsilon\}$
 - $\mathcal{P}(Q)$: insieme di **tutti i possibili sottoinsiemi** di Q

(es, se $Q = \{q_0, q_1, q_2\}$ allora $\mathcal{P}(Q) = \{\emptyset, \{q_0\}, \{q_1\}, \{q_2\}, \{q_0, q_1\}, \{q_0, q_2\}, \{q_1, q_2\}, \{q_0, q_1, q_2\}\}$)

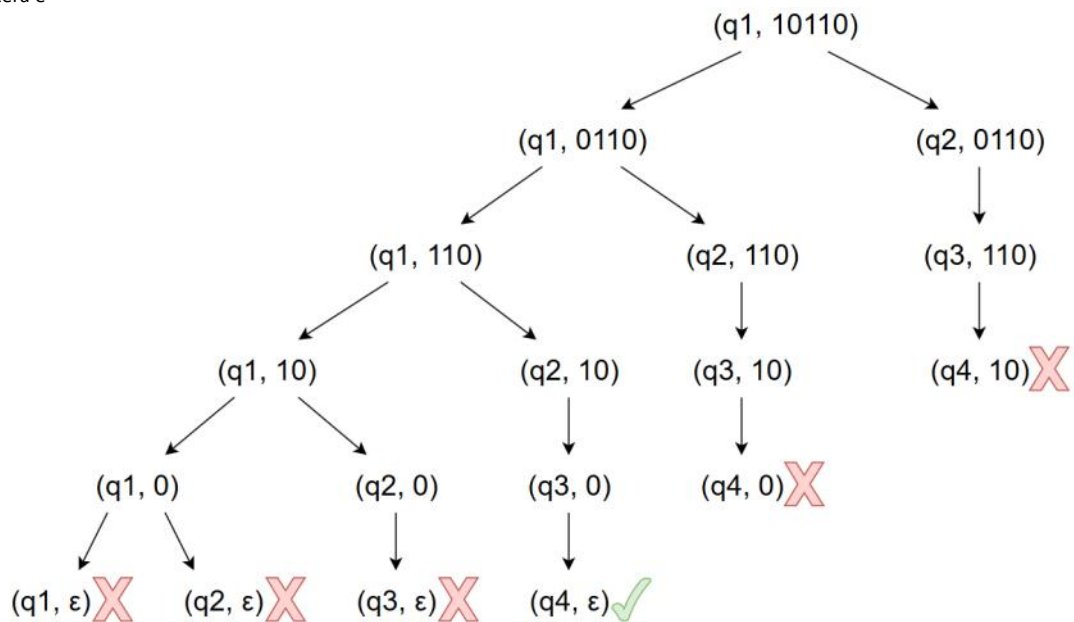
Esempio:



$L = \{x \in \{0, 1\}^* : x \text{ ha "1" in terzultima posizione}\}$

$x = 10110 \in L$

La computazione intera è

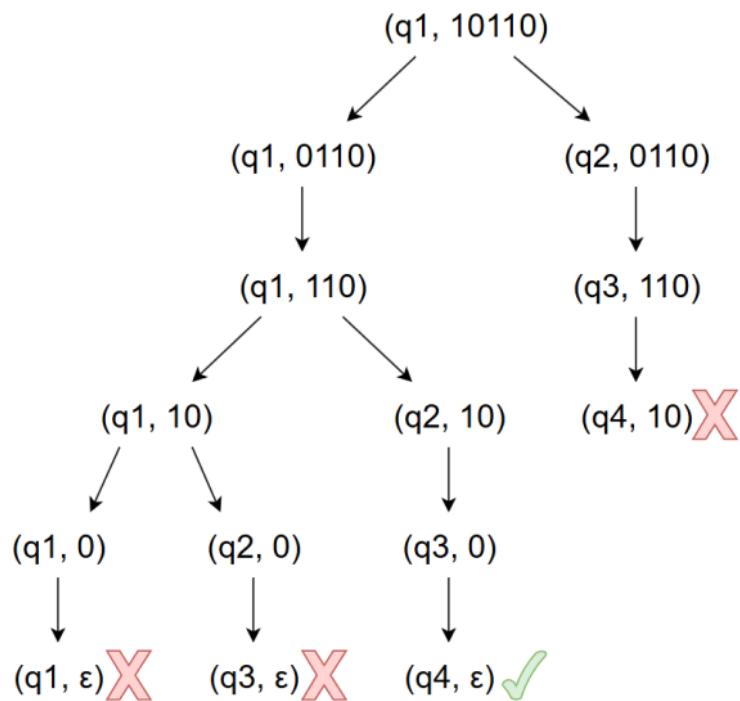


Ma si possono **ridurre** il numero di rami da disegnare togliendo i **rami che non porteranno mai ad accettazione**

Ad esempio in questo caso siccome dobbiamo controllare se "1" è terzultimo allora per l'automa ogni "1" potrebbe essere il terzultimo (e quindi dobbiamo controllare ogni ramo) mentre è inutile dividere i rami se stiamo controllando il carattere "0".

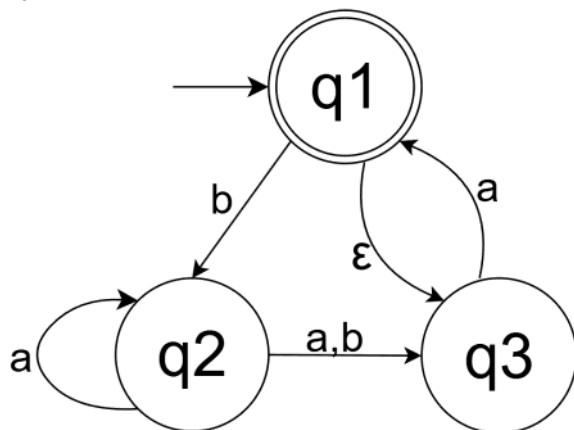
Infatti vediamo che solo "1" provoca **non determinismo** in q_1 , in quanto fa un loop e si connette a q_2 , mentre con "0" ritorna a q_1 e basta. Per cui la computazione non si ramifica mai da q_1 quando leggiamo 0, **a prescindere dalla posizione del carattere che stiamo leggendo**

Ad esempio quando siamo in $(q_1, 0110)$ invece di dividere in $(q_1, 110)$ e $(q_2, 110)$



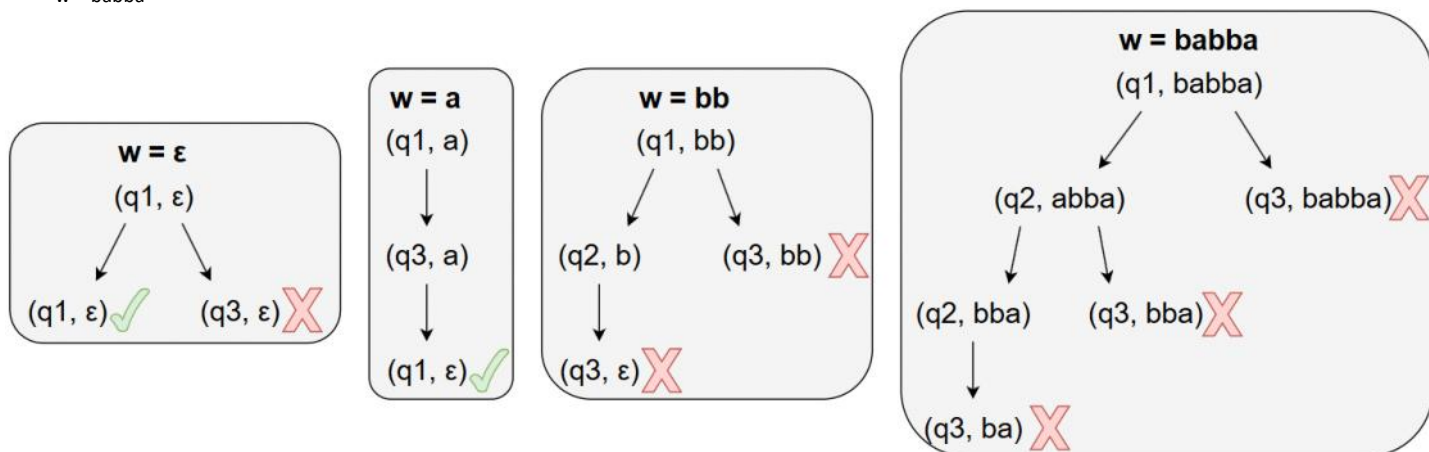
Siccome abbiamo un ramo in cui si accetta x, allora la NFA accetta x

Esercizio:



Controllare computazione per

- $w = \epsilon$
- $w = a$
- $w = bb$
- $w = babba$



Estensione di Configurazione

Possiamo estendere il concetto di **configurazione**:

per un NFA N , sarà una coppia

$$(q, x) \in Q \times \Sigma \epsilon^*$$

e avremmo:

$$(p, ax) \vdash_N (q, x) \Leftrightarrow q \in \delta(p, a)$$

con

$$p, q \in Q, a \in \Sigma, x \in \Sigma \epsilon^*, \delta(p, a) \in \mathcal{P}(Q)$$

se w è una stringa $\in \Sigma^*$, abbiamo che:
 $w \in L(N) \Leftrightarrow \exists q \in F \mid (q_0, w) \vdash_N^* (q, \varepsilon)$

dove $\vdash_N^*$ è la **relazione estesa**

cioè N accetta w valida se e solo se esiste uno stato di accettazione q t.c. dallo stato di partenza con w possiamo arrivare allo stato q finendo la stringa

Insieme Linguaggi Accettati da DFA

Denotiamo $\mathcal{L}(\text{DFA})$ l'insieme dei linguaggi accettati da un qualsiasi DFA, che per definizione è REG, e denotiamo allo stesso modo $\mathcal{L}(\text{NFA})$, cioè

$$\mathcal{L}(\text{NFA}) = \{L : \exists \text{ NFA } N \text{ t.c. } L(N) = L\}$$

Teorema: $\text{REG} = \mathcal{L}(\text{NFA})$

L'insieme dei linguaggi accettati da una qualsiasi DFA sono gli stessi dell'insieme dei linguaggi accettati da una qualsiasi NFA, quindi

$$\mathcal{L}(\text{DFA}) = \mathcal{L}(\text{NFA}) = \text{REG}$$

Dimostrazione:

devo dimostrare $\mathcal{L}(\text{DFA}) \subseteq \mathcal{L}(\text{NFA})$ e $\mathcal{L}(\text{NFA}) \subseteq \mathcal{L}(\text{DFA})$

Caso $\mathcal{L}(\text{DFA}) \subseteq \mathcal{L}(\text{NFA})$ (ogni linguaggio accettato da un DFA è accettato anche da un NFA):

Se $L \in \mathcal{L}(\text{DFA})$ allora $\exists \text{ DFA } D$ t.c. $L(D) = L$, e siccome un DFA è un NFA non-deterministico (l'NFA è più generale di un DFA) quindi considero D come un NFA N ($N = D$) allora anche da NFA accetta L ($L(N) = L$)

Caso $\mathcal{L}(\text{NFA}) \subseteq \mathcal{L}(\text{DFA})$ (ogni linguaggio accettato da un NFA è accettato anche da un DFA):

Vogliamo "simulare" un NFA tramite un DFA che rappresenti ogni possibile stato di computazione

Se $L \in \mathcal{L}(\text{NFA})$ allora $\exists \text{ NFA } N = (Q_N, \Sigma, \delta_N, q_0^N, F_N)$ t.c. $L(N) = L$

Devo costruire un DFA $D = (Q_D, \Sigma, \delta_D, q_0^D, F_D)$ t.c. $L(D) = L$

- **Caso semplice: no ε -archi**, avremmo:

- $Q_D = \mathcal{P}(Q_N)$, ovvero gli stati del DFA corrispondono agli elementi di tutti i sottoinsiemi di Q dell'NFA
- $q_0^D = \{q_0^N\}$, ovvero lo stato iniziale del DFA è l'insieme degli stati iniziali dell'NFA
- $F_D = \{R \in Q_D : R \cap F_N \neq \emptyset\}$, ovvero D accetta tutti gli insiemi in cui compare almeno un elemento accettato da N (R contiene uno stato finale di Q_N)
- Sia $R \in Q_D$, $a \in \Sigma$, si definisce:

$$\delta_D(R, a) = \bigcup_{r \in R} \delta_N(r, a)$$

($= \{q \in Q_N : q \in \delta_N(r, a) \text{ per qualche } r \in R\}$), ovvero il nuovo stato del DFA è l'unione di tutti gli stati raggiungibili da ciascun r in R , poiché in un NFA si può passare contemporaneamente a più stati, quindi il DFA tiene traccia di tutti gli stati che NFA può raggiungere leggendo a

Di fatto D sta **emulando deterministicamente** tutti i rami di computazione **non-deterministici** di N

- **Caso generale:** ci sono ε -archi e dobbiamo tener traccia degli stati che possiamo raggiungere tramite ε -archi

Sia $R \in Q_D$, si definisce la funzione estesa E :

$E(R) = \{q \in Q_N : q \text{ può essere raggiunto da un qualsiasi stato } r \in R \text{ attraverso } \geq 0 \text{ } \varepsilon\text{-archi}\}$, ovvero $E(R)$ è l'insieme di tutti gli stati raggiungibili da R senza leggere nessun simbolo

(Es: NFA con $\delta(q_0, \varepsilon) = q_1$ e $\delta(q_1, \varepsilon) = q_2$, allora $E(\{q_0\}) = \{q_0, q_1, q_2\}$, $E(\{q_1\}) = \{q_1, q_2\}$ e $E(\{q_2\}) = \{q_2\}$)

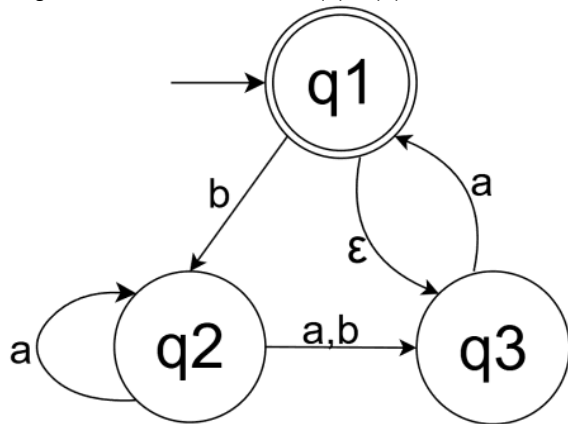
Quindi la definizione di DFA cambia con:

- $q_0^D = E(\{q_0^N\})$
- $\delta_D(R, a) = \bigcup_{r \in R} E(\delta_N(r, a))$

Quindi dagli stati che possiamo raggiungere leggendo "a" aggiungiamo anche gli stati che possiamo raggiungere senza leggere nulla (quelli raggiunti da ε -archi), e poi uniamo tutti gli insiemi trovati

Esempio:

Dato il seguente NFA, definire DFA D t.c. $L(D) = L(N)$

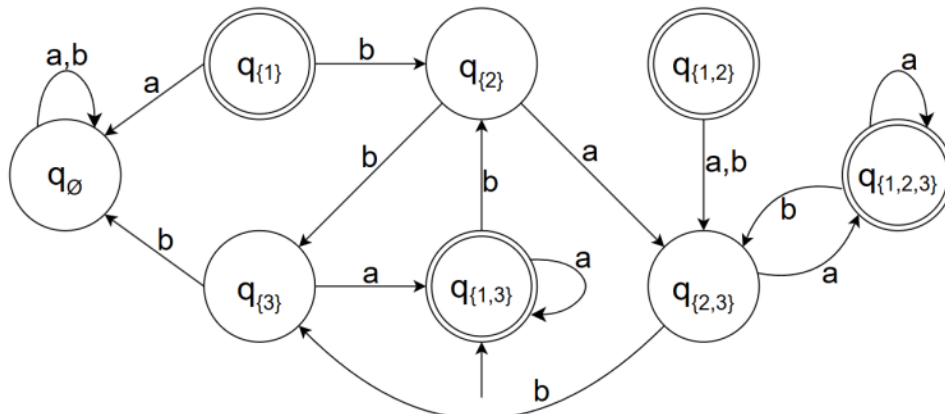


Prima definiamo gli stati:

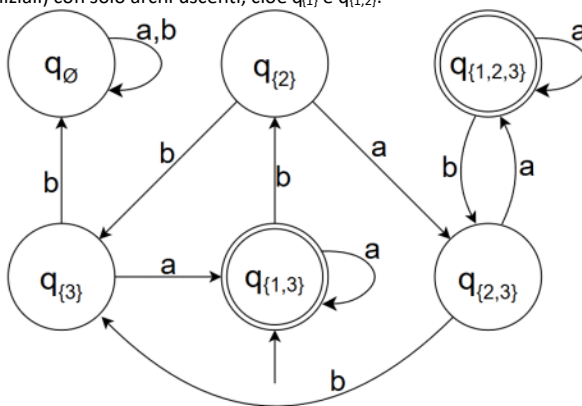
- $Q_D = \{q_\emptyset, q_{\{1\}}, q_{\{2\}}, q_{\{3\}}, q_{\{1,2\}}, q_{\{1,3\}}, q_{\{2,3\}}, q_{\{1,2,3\}}\}$
- $q_0^D = E(\{q_1\}) = q_{\{1,3\}}$
- $F_D = \{q_{\{1\}}, q_{\{1,2\}}, q_{\{1,3\}}, q_{\{1,2,3\}}\}$
- Definiamo δ_D :
 - Da $q_\emptyset$, avremmo:
 - $\delta_D(q_\emptyset, a) = q_\emptyset$ (poiché come stato non esiste quindi può raggiungere solo se stesso)
 - $\delta_D(q_\emptyset, b) = q_\emptyset$
 - Da $q_{\{1\}}$, avremmo:
 - $\delta_D(q_{\{1\}}, a) = q_\emptyset$
 - $\delta_D(q_{\{1\}}, b) = q_{\{2\}}$
 - Da $q_{\{2\}}$, avremmo:
 - $\delta_D(q_{\{2\}}, a) = q_{\{2,3\}}$ (perché da q_2 ci sono due archi "a", uno che porta a q_2 stesso e l'altro che porta a q_3)
 - $\delta_D(q_{\{2\}}, b) = q_{\{3\}}$
 - Da $q_{\{3\}}$, avremmo:

- $\delta_D(q_{\{3\}}, a) = q_{\{1,3\}}$ (perche da q_1 posso raggiungere anche q_3 con l'arco " ϵ ")
- $\delta_D(q_{\{3\}}, b) = q_{\emptyset}$
- Da $q_{\{1,2\}}$, avremmo:
 - $\delta_D(q_{\{1,2\}}, a) = E(\delta_N(q_{\{1\}}, a)) \cup E(\delta_N(q_{\{2\}}, a)) = E(q_{\emptyset}) \cup E(q_{\{2,3\}}) = q_{\emptyset} \cup q_{\{2,3\}} = q_{\{2,3\}}$
 - $\delta_D(q_{\{1,2\}}, b) = E(\delta_N(q_{\{1\}}, b)) \cup E(\delta_N(q_{\{2\}}, b)) = E(q_{\{3\}}) \cup E(q_{\{3\}}) = q_{\{3\}} \cup q_{\{3\}} = q_{\{3\}}$
- ecc

Quindi avremmo:



Dopo aver semplificato togliendo gli stati (non iniziali) con solo archi uscenti, cioè $q_{\{1\}}$ e $q_{\{1,2\}}$:



Teorema di Chiusura per REG

(immagini degli schemi presi dagli appunti di CasuFrost)

A questo punto usiamo gli NFA per completare il **teorema di chiusura** per REG

Unione degli NFA

Consideriamo l'**unione di due NFA**, prendiamo due NFA:

$$N_1 = (Q_1, \Sigma, \delta_1, q_0^1, F_1)$$

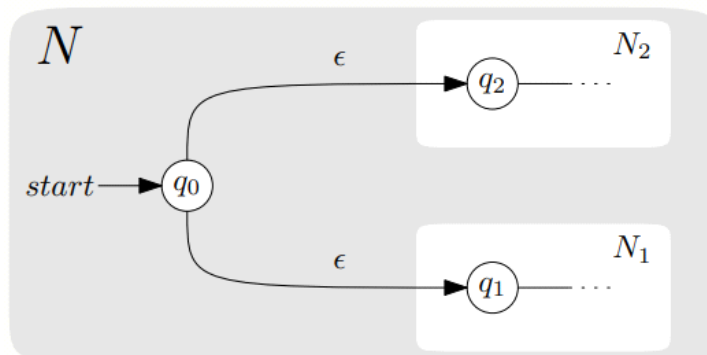
$$N_2 = (Q_2, \Sigma, \delta_2, q_0^2, F_2)$$

Definisco una nuova NFA $N = (Q, \Sigma, \delta, q_0, F)$ t.c.:

- $Q = Q_1 \cup Q_2 \cup \{q_0\}$
- $F = F_1 \cup F_2$
- $\forall q \in Q, a \in \Sigma:$

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & \text{se } q \in Q_1 \\ \delta_2(q, a) & \text{se } q \in Q_2 \\ \{q_0^1, q_0^2\} & \text{se } q = q_0 \wedge a = \epsilon \\ \emptyset & \text{se } q = q_0 \wedge a \neq \epsilon \end{cases}$$

Quindi abbiamo $L(N) = L(N_1) \cup L(N_2)$



Teorema: REG è chiusa rispetto a "o"

Dimostrazione: Dati L_1 e L_2 , esistono due NFA N_1 e N_2 t.c. $L(N_1) = L_1 \wedge L(N_2) = L_2$

$$N_1 = (Q_1, \Sigma, \delta_1, q_0^1, F_1)$$

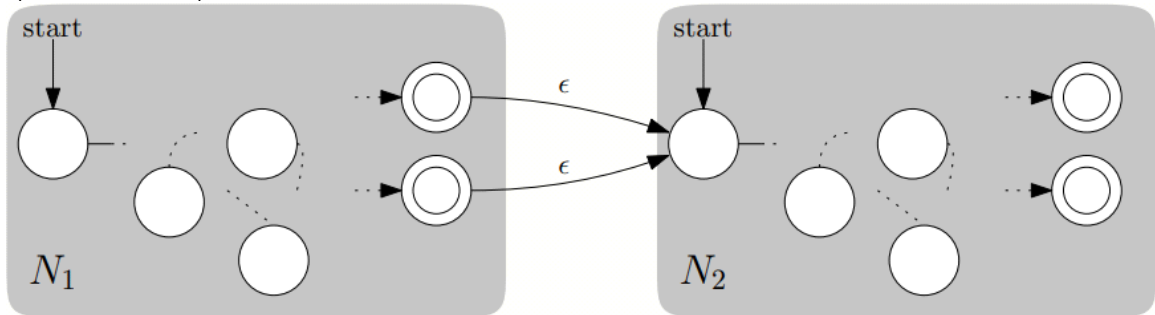
$$N_2 = (Q_2, \Sigma, \delta_2, q_0^2, F_2)$$

Costruisco una NFA $N = (Q, \Sigma, \delta, q_0, F)$ per $L = L_1 \circ L_2$, cioè per concatenare le ramificazioni di N_1 a N_2

- $q_0 = q_0^1$
- $Q = Q_1 \cup Q_2$
- $F = F_2$
- $\forall q \in Q, a \in \Sigma_\epsilon$:

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & \text{se } q \in Q_1, q \notin F_1 \\ \delta_1(q, a) & \text{se } q \in F_1, a \neq \epsilon \\ \delta_1(q, a) \cup \{q_0^2\} & \text{se } q \in F_1, a = \epsilon \\ \delta_2(q, a) & \text{se } q \in Q_2 \end{cases}$$

In questo modo l'automa accetta solo una concatenazione $L_1 \circ L_2$ poiche quando finisce di operare su L_1 e arriva su uno stato di F_1 andrà poi nello stato iniziale di N_2 dove inizierà ad operare su L_2 finendo poi in F_2 , cioè F

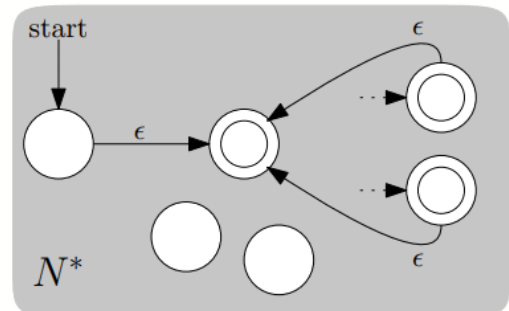
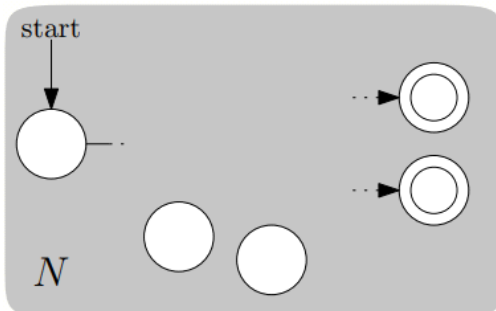


Teorema: REG è chiusa rispetto a "*"

Dimostrazione: dato un NFA $N = (Q, \Sigma_\epsilon, \delta, q_0, F)$ t.c. $L(N) = L$, costruisco un NFA $N^* = (Q', \Sigma_\epsilon, \delta', q_0', F')$ t.c. $L(N^*) = L^*$

- $Q' = Q \cup \{q_0'\}$
- q_0' è un nuovo stato iniziale
- $F' = F \cup \{q_0'\}$
- $\forall q \in Q', a \in \Sigma_\epsilon$:

$$\delta'(q, a) = \begin{cases} \delta(q, a) & \text{se } q \in Q, q \notin F \\ \delta(q, a) & \text{se } q \in F, a \neq \epsilon \\ \delta(q, a) & \text{se } q \in F, a = \epsilon \\ \{q_0'\} & \text{se } q = q_0', a = \epsilon \\ \emptyset & \text{se } q = q_0', a \neq \epsilon \end{cases}$$



Ricordiamo:

$L = \{a, ab, ba\}$

$L^2 = \{aa, aab, aba, abab, abba, baa, baab, baba\}$

...

$$L^* = \bigcup_{n \geq 0} L^n = \{\epsilon\} \cup L^1 \cup L^2 \cup \dots$$

Es: $L = \{a, b\}$

$L^* = \{\epsilon, a, b, aa, ab, ba, bb, aaa, aab, aba, abb, baa, \dots\}$

Espressioni Regolari

martedì 7 ottobre 2025 23:14

Le **espressioni regolari** sono simili alle espressioni algebriche, ma operano sulle stringhe e definiscono linguaggi su un certo **alfabeto**

Es: $(0 \cup 1)^* 0^*$

dove:

- $\{0 \cup 1\} \equiv \{0\} \cup \{1\} \equiv \{0, 1\}$
- $0^* \equiv \{0\}^* (\epsilon, 0, 00, 000, \text{ecc})$

Quindi $(0 \cup 1)^* 0^* \equiv \{0, 1\}^* \circ \{0\}^*$, cioè una stringa che **inizia con 0 o 1 ed è seguita da zero o più 0** (tipo, 0, 0000, 1, 10000, 10, 00, ...)

Definizione di Espressione Regolare

Sia Σ un **alfabeto**, un'espressione regolare su Σ (denotata $re(\Sigma)$) è definita per induzione:

- Caso base:**
 - $r = \emptyset \in re(\Sigma)$
 - $r = \epsilon \in re(\Sigma)$
 - $r = a \in re(\Sigma)$
- Passo induttivo:**
 - $r = R_1 \cup R_2$, dove $R_1, R_2 \in re(\Sigma)$
 - $r = R_1 \circ R_2$, dove $R_1, R_2 \in re(\Sigma)$
 - $r = (R_1)^*$ dove $R_1 \in re(\Sigma)$

Ogni espressione regolare ha associato un **linguaggio** $L(r)$ t.c. $r \in re(\Sigma)$

- Caso base:**
 - $L(r) = \emptyset$, se $r = \emptyset$
 - $L(r) = \epsilon$, se $r = \epsilon$
 - $L(r) = a$, se $r = a$
- Passo induttivo:**
 - $L(r) = L(R_1) \cup L(R_2)$, se $r = R_1 \cup R_2$
 - $L(r) = L(R_1) \circ L(R_2)$, se $r = R_1 \circ R_2$
 - $L(r) = L((R_1)^*)$, se $r = R_1^*$

Esempi: $\Sigma = \{0, 1\}$

- $0^* 10^* = \{w : w \text{ contiene esattamente un "1"}\}$
- $\Sigma^* 1 \Sigma^* = \{w : w \text{ contiene almeno un "1"}\}$
- $\Sigma^* 001 \Sigma^* = \{w : w \text{ contiene "001" come sottostringa}\}$
- $(0 \cup 1000)^* = \{\text{ogni occorrenza di "1" è seguita da "000"}\}$
- $re(\Sigma) = \{\emptyset, \epsilon, 0, 1, 01, 0 \cup 1, 0^*, 1^*, (01)^*, (01 \cup 1)^*, \text{ecc}\}$

Per convenzione:

$$L \circ \emptyset = \emptyset \text{ e } \emptyset^* = \emptyset$$

per qualunque linguaggio L

Teorema

Un linguaggio è **regolare** se e solo se esiste un'espressione regolare che lo descrive:

$$REG \equiv L(re)$$

dove $L(re)$ è l'insieme dei linguaggi accettati da una qualsiasi espressione regolare

Dimostrazione:

Devo dimostrare $L(re) \subseteq REG$ e $REG \subseteq L(re)$

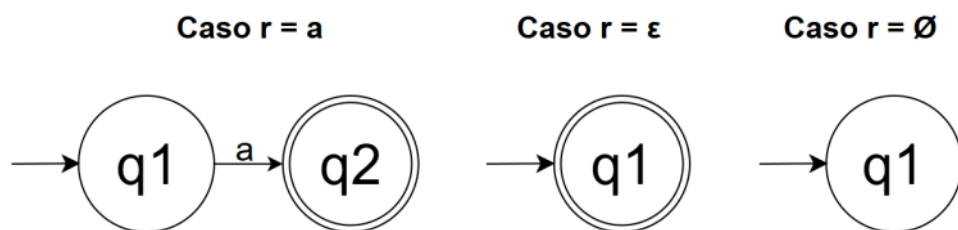
Caso $L(re) \subseteq REG$ (ogni linguaggio che si può descrivere con un'espressione regolare appartiene all'insieme dei linguaggi regolari)

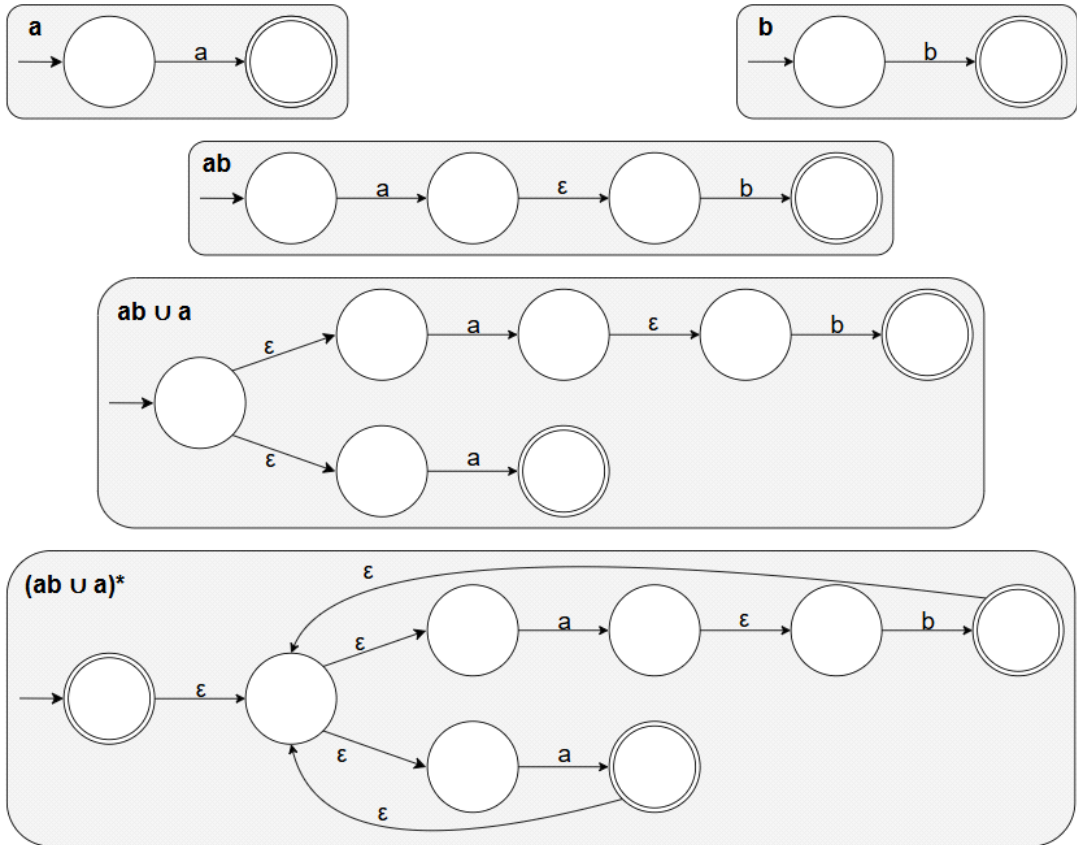
Data l'espressione regolare r , costruisco un NFA/DFA N_r t.c. $L(N_r) = L(r)$, usando la definizione ricorsiva che abbiamo considerato di $L(r)$:

- Caso base:**
 - $r = a, a \in \Sigma$
 $N_r = (\{q_1, q_2\}, \Sigma, \delta, q_1, \{q_2\})$
 $\delta(q_1, a) = q_2$
 $\delta(q, b) = \emptyset$ dove $q \neq q_1, b \neq a$
 - $r = \epsilon$
 $N_r = (\{q_1\}, \Sigma, \delta, q_1, \{q_1\})$
 $\delta(q_1, b) = \emptyset \quad \forall b \in \Sigma$
 - $r = \emptyset$
 $N_r = (\{q_1\}, \Sigma, \delta, q_1, \emptyset)$
 $\delta(q_1, b) = \emptyset \quad \forall b \in \Sigma$

- Passo induttivo:**
 - Se $r = R_1 \cup R_2$, per induzione
 $\exists M_1, M_2$ t.c. $L(M_1) = L(R_1)$ e $L(M_2) = L(R_2)$
 $\Rightarrow \exists M$ t.c. $L(M) = L(R_1) \cup L(R_2)$ (per chiusura di REG)
Stesso discorso per " $\circ$ " e " * "

Es: Quale è l'NFA corrispondente per **$(ab \cup a)^*$** ?





Ad es:

- Stringhe valide per $(ab \cup a)$ cioè "ab oppure a": "a", "ab"
- Stringhe valide per $(ab \cup a)^*$ cioè "zero o più volte ab o a": "ε", "aa", "ab", "a", "aaababa", "aaaabab", ecc

Caso REG $\subseteq$ L(re) (ogni linguaggio regolare appartiene all'insieme dei linguaggi accettati da una qualsiasi espressione regolare)

Sia $L \in \text{REG}$ (un linguaggio regolare) e N un NFA t.c. $L(N) = L$

Per facilitare la conversione in espressione regolare introduciamo l'**NFA generalizzato (GNFA)** e assumiamo **wlog** (without loss of generality, cioè semplificare un ragionamento con un esempio evitando ripetizioni di esempi equivalenti) che ha una **forma canonica**, ossia rispetta le seguenti proprietà:

- Le etichette degli archi sono **espressioni regolari**
- Lo stato **iniziale** ha **solo archi uscenti** verso tutti gli altri stati
- Lo stato **finale** ha **solo archi entranti**
- Eccetto per lo stato iniziale e finale, c'è **un solo arco** per ogni coppia di stati

In pratica abbiamo un GNFA $G = (Q, \Sigma, \delta, q_{\text{start}}, q_{\text{acc}})$, dove

$$\delta : Q \setminus \{q_{\text{acc}}\} \times Q \setminus \{q_{\text{start}}\} \rightarrow \mathcal{R} = \text{re}(\Sigma)$$

dove $\mathcal{R} \equiv$ **insieme delle espressioni regolari su Σ**

Quindi possiamo assumere wlog questa forma canonica perché posso sempre aggiungere **due stati** (iniziale e finale) e gli **archi mancanti** fra le coppie di stati non collegate (usando ϵ -archi)

Ora possiamo usare la funzione **Convert(G)** (che elimina **uno stato alla volta**) per convertire G in un'espressione regolare. Sia k il num. di **stati** in G :

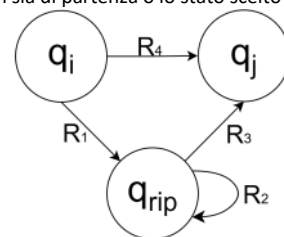
- Se $k = 2$, G avrà solo lo stato iniziale e lo stato finale (q_{start} e q_{acc}) e un singolo arco con etichetta $R \in \mathcal{R}$. Viene restituito R
Es: $q_{\text{start}} \xrightarrow{a^*b} q_{\text{acc}} \Rightarrow \text{Convert}(G) = a^*b$
- Se $k > 2$, scelgo $q_{\text{rip}} \in Q$ dove $q_{\text{rip}} \neq q_{\text{start}}, q_{\text{acc}}$ e definisco $G' = (Q', \Sigma, \delta', q_{\text{start}}, q_{\text{acc}})$, dove:
 - $Q' = Q \setminus \{q_{\text{rip}}\}$ (senza lo stato scelto q_{rip})
 - $\delta' : Q' \setminus \{q_{\text{acc}}\} \times Q' \setminus \{q_{\text{start}}\} \rightarrow \mathcal{R}$ (togliamo gli archi che passano per q_{rip})
 - $\forall q_i \in Q' \setminus \{q_{\text{acc}}\}, q_j \in Q' \setminus \{q_{\text{start}}\}$ (cioè q_i che non sia il finale o lo stato scelto q_{rip} e q_j non sia di partenza o lo stato scelto q_{rip}), abbiamo:

$$\delta'(q_i, q_j) = (R_1)(R_2)^*(R_3) \cup R_4$$

(cioè tutti i modi per andare da q_i a q_j , cioè passando per q_{rip} o saltandolo tramite R_4)

Dove:

- $R_1 = \delta(q_i, q_{\text{rip}})$ (ingresso in q_{rip})
- $R_2 = \delta(q_{\text{rip}}, q_{\text{rip}})$ (rimango in q_{rip})
- $R_3 = \delta(q_{\text{rip}}, q_j)$ (esco da q_{rip})
- $R_4 = \delta(q_i, q_j)$ (salto q_{rip} direttamente se l'arco esiste)



Ora devo dimostrare che **Convert(G)** è **equivalente a G**

- Caso Base:** per $k = 2$ è sicuramente vero poiché $\text{Convert}(G)$ restituisce l'espressione regolare R che descrive tutte le stringhe che portano G da q_{start} a q_{acc}
- Passo induttivo:** suppongo che $\text{Convert}(G)$ è equivalente a G per $k-1$ stati e faccio vedere che è **vero anche per k stati**.

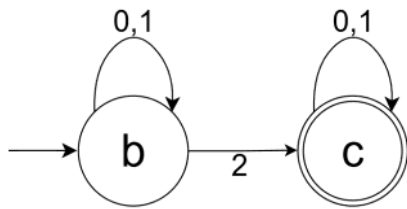
Basta mostrare che $L(G) = L(G')$, così la prova segue dall'ipotesi induttiva siccome il num. di stati di G' è $k-1$

Ma se G accetta w , allora esiste un ramo di computazione t.c. G percorre gli stati $\{q_{\text{start}}, q_1, \dots, q_{\text{acc}}\}$ e se la sequenza non contiene q_{rip} , allora banalmente $L(G) = L(G')$, perché le nuove espressioni regolari contengono le vecchie espressioni come l'unione.

Se c'è q_{rip} , avrò ad es $\{q_1, q_{\text{rip}}, q_2\}$ e quindi una nuova sequenza di stati per G' , ma gli stati adiacenti a q_{rip} , cioè (q_1, q_2) , hanno in G' un arco con un'espressione regolare che tiene conto di tutti i modi per andare da q_1 a q_2 direttamente o passando per q_{rip}

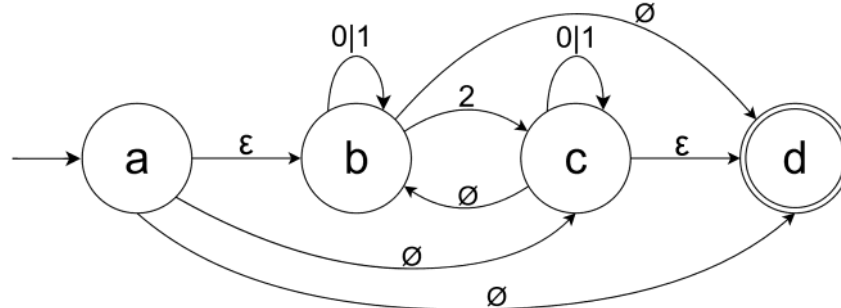
Quindi $L(G) = L(G')$

Esempio: Trovare l'espressione regolare associata a:



Costruiamo la GNFA, trasformando l'automa in forma canonica:

- Trasformiamo le etichette in espressioni regolari (es "0,1" → "0|1" oppure "0∪1")
- Poiché lo stato **iniziale** deve avere **solo archi uscenti** verso tutti gli altri stati, aggiungiamo un nuovo stato iniziale "a" collegato solo a "b" (lo stato iniziale precedente)
- Poiché lo stato **finale** deve avere **solo archi entranti**, aggiungiamo un nuovo stato d'accettazione "d" (poiché "c" ha un arco uscente verso se stesso)
- C'è un solo arco per ogni coppia di stati diversi da quello iniziale e quello finale (nel nostro caso solo "b" e "c"), quindi aggiungiamo l'arco $\delta(b, c) = \emptyset$



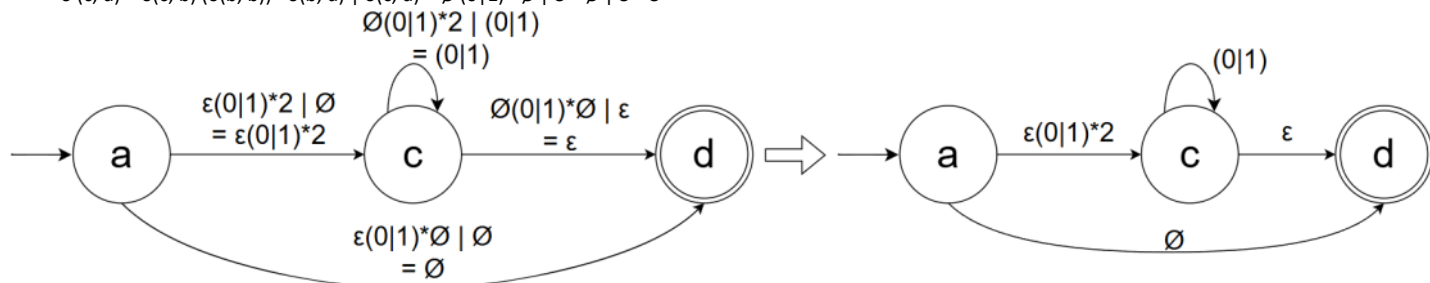
Rimuoviamo b, però dobbiamo aggiornare gli archi che collegano i vari stati usando la formula:

$$\delta'(q_i, q_j) = (R_1)(R_2)^*(R_3) \cup R_4, \quad \forall q_i \in Q \setminus \{d, b\}, q_j \in Q \setminus \{a, b\}$$

dove $R_1 = \delta(q_i, b)$ cioè "entriamo in b", $R_2 = \delta(b, b)$ cioè "rimaniamo in b", $R_3 = \delta(b, q_j)$ cioè "usciamo da b", $R_4 = \delta(q_i, q_j)$ cioè "collego direttamente q_i a q_j "

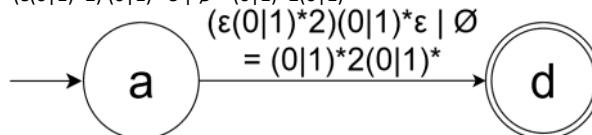
Quindi $\delta'(q_i, q_j) = \delta(q_i, b) (\delta(b, b))^* \delta(b, q_j) \mid \delta(q_i, q_j)$. Quindi:

- $\delta'(a, c) = \delta(a, b) (\delta(b, b))^* \delta(b, c) \mid \delta(a, c) = \epsilon (0|1)^* 2 \mid \emptyset = \epsilon (0|1)^* 2$
- $\delta'(a, d) = \delta(a, b) (\delta(b, b))^* \delta(b, d) \mid \delta(a, d) = \epsilon (0|1)^* \emptyset \mid \emptyset = \emptyset \mid \emptyset = \emptyset$
- $\delta'(c, c) = \delta(c, b) (\delta(b, b))^* \delta(b, c) \mid \delta(c, c) = \emptyset (0|1)^* 2 \mid (0|1) = \emptyset \mid (0|1) = (0|1)$
- $\delta'(c, d) = \delta(c, b) (\delta(b, b))^* \delta(b, d) \mid \delta(c, d) = \emptyset (0|1)^* \emptyset \mid \epsilon = \emptyset \mid \epsilon = \epsilon$



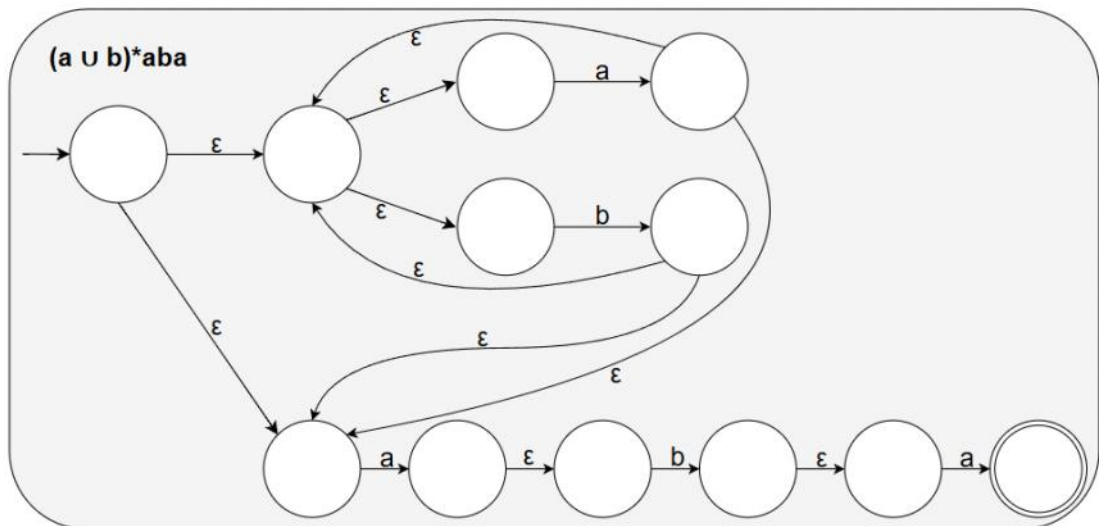
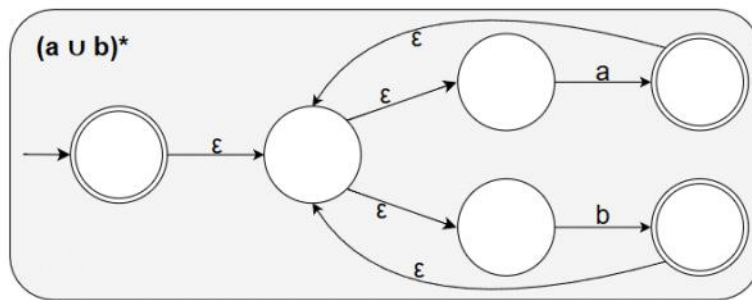
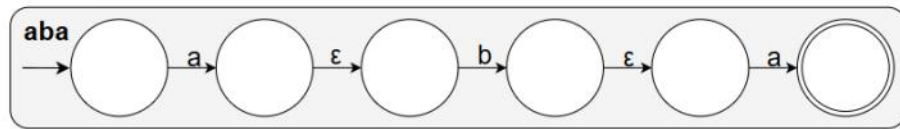
Rimuoviamo c, allo stesso modo di prima:

- $\delta'(a, d) = \delta(a, c) (\delta(c, c))^* \delta(c, d) \mid \delta(a, d) = (\epsilon (0|1)^* 2) (0|1)^* \epsilon \mid \emptyset = (0|1)^* 2 (0|1)^*$



Quindi $r = (0|1)^* 2 (0|1)^*$

Altro esempio di costruzione di un NFA corrispondente a $(a \cup b)^* aba$



Linguaggi Non Regolari

giovedì 9 ottobre 2025 15:30

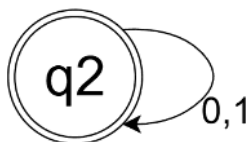
Pumping Lemma

Non tutti i linguaggi sono regolari.

Es: $L = \{0^n 1^n : n \geq 0\} = \{\epsilon, 01, 0011, 000111, \dots\}$ **non è regolare**

poiché un automa dovrebbe contare quanti "0" ha visto e poi controllare che ci siano lo stesso numero di "1",

ma gli automi finiti hanno un **numero finito di stati**, quindi non possono contare fino a **n arbitrariamente grande**



Il **pumping lemma** è un modo per dimostrare che un linguaggio **non è regolare**

Teorema Pumping Lemma:

Il **Pumping Lemma** fornisce una condizione che è soddisfatta da ogni linguaggio L che è **regolare**.

Se L è regolare, allora esiste un val. p t.c. presa $w \in L$ con $|w| \geq p$, allora w può essere scomposta in $w = xyz$, in modo che:

- 1) $\forall i \geq 0, xy^i z \in L$ (y viene ripetuto almeno una volta)
- 2) $|y| > 0$ (y deve esistere, altrimenti non c'è nulla da ripetere)
- 3) $|xy| \leq p$ (il ciclo si trova entro p stati)

z e x possono anche essere vuoti

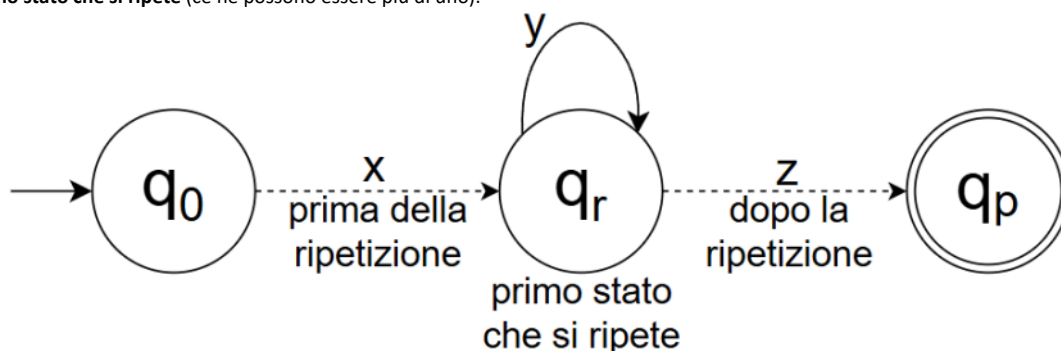
Quindi se L è regolare allora ogni $w \in L$ sufficientemente lunga ha una **parte centrale y** che si può ripetere all'infinito

Intuizione:

$p = \#$ stati automa M t.c. $L(M) = L$ (siccome L è REG esiste un DFA M che legge L) e siccome $|w| \geq p$ (più lunga del num. di stati) allora **almeno uno stato è ripetuto** (La ripetizione non deve per forza essere nello stato stesso ma si può anche avere tornando allo stato q_r dopo aver visitato altri stati)

Siccome y deve esistere per forza allora esiste uno stato che viene ripetuto almeno una volta.

Chiamiamo q_r il **primo stato che si ripete** (ce ne possono essere più di uno).



Dimostrazione:

Sia $M = (Q, \Sigma, \delta, q_1, F)$ t.c. $L(M) = L$ e sia $p = |Q|$ ($\#$ stati di M). Consideriamo $w = w_1 w_2 \dots w_n$ (stringa lungh. n) t.c. $n \geq p$ (la stringa è più lunga del num. di stati)

Sia $r_1, \dots, r_{n+1}$ la **sequenza di stati attraversati** su input w ($r_1 = q_1$ e $r_{n+1} \in F$ poiché l'ultimo stato accetta L)

Quindi r_1 legge w_1 , r_2 legge w_2 , ..., r_{n+1} legge w_n (poiché è l'ultimo carattere e quindi accetta), cioè:

$$\delta(r_i, w_i) = r_{i+1} \text{ con } i = 1, \dots, n$$

Ovviamente gli stati passati sono $n+1 \geq p+1$ (poiché $n \geq p$), quindi per il PHP (Pigeon Hole Principle) (cioè $n+1$ stati visitati ma solo p stati distinti $\Rightarrow$ almeno uno stato si ripete) nella sequenza considerata c'è almeno uno stato che si ripete (ci sono almeno due stati uguali).

Sia questo r_j nella **1ª apparizione** e r_l nella **2ª** dello stesso stato ($r_j = r_l$ con $l > j$)

Ovviamente abbiamo che $l \leq p+1$ perché r_l si presenta tra le prime $p+1$ posizioni della sequenza di transizioni

A questo punto **scomponiamo la stringa $w = xyz$** :

- $x = w_1, \dots, w_{j-1}$ (tutto quello che viene **prima del 1º stato che si ripete**)
- $y = w_j, \dots, w_{l-1}$ (tutto quello che viene **prima della ripetizione**)
- $z = w_l, \dots, w_n$ (tutto il resto **dopo la ripetizione**)

Quindi possiamo **verificare** le condizioni del Pumping Lemma:

- **Condizione 1:** siccome x porta M da $r_1 = q_1$ a r_j , y porta M da r_j a $r_l = r_j$ con $j \neq l$ (c'è la ripetizione), z porta M da $r_l = r_j$ a $r_{n+1} \in F$, allora $\forall i \geq 0, xy^i z \in L(M)$ (se xyz è accettata, **pompando y** (aumentando i di y) (si rega si dice così) allora avremmo che anche xy^2z è accettata e anche xy^3z e così via)
- **Condizione 2:** siccome $j \neq l$, allora $|y| > 0$ (perché c'è una ripetizione)
- **Condizione 3:** siccome $l \leq p+1$, allora $|xy| = l-1 \leq p$ (il pezzo **xy** (che arriva fino al punto in cui lo stato si ripete per la prima volta) si trova **prima o al massimo dopo p simboli letti**)

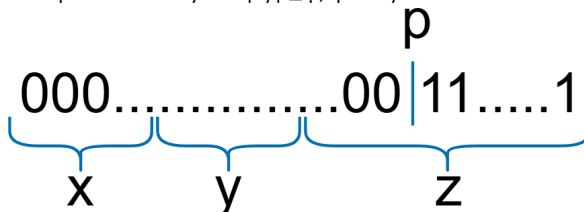
Per contraddire il Pumping Lemma devo trovare **almeno una stringa che scomposta** in modo legale **non appartiene a L**

Esempio 1

$L = \{0^n 1^n : n \geq 0\}$ non è regolare

Scelgo $w = 0^p 1^p$ dove p è il val. di pumping che esisterebbe se il linguaggio fosse regolare (stiamo provando per contraddizione)

Abbiamo $|w| \geq p$. Siccome $w = 0^p 1^p$ per qualsiasi scomposizione $w = xyz$ t.c. $|xy| \leq p$, quindi y contiene solo 0



(y non deve essere per forza l'ultimo "0" in posizione p)

Quindi per $i \geq 2$: $\hat{w} = xy^iz = 0^i 1^p$ t.c. $q > p$

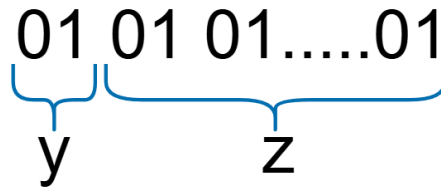
Questo contraddice il pumping perché non possiamo scomporre w in modo che $xy^iz \in L$, perché y contiene solo "0" e se aggiungiamo altri "0" dopo p (perché y si ripete) significa che ho più "0" rispetto al num. di "1"

Esempio 2

$L = \{w \in \{0, 1\}^* : \#_0 w = \#_1 w\}$ non è regolare (ovvero w ha uguale num. di "0" e "1") (possono essere in vario ordine basta che sono lo stesso numero di "0" e "1")

Suppongo che sia regolare: $\exists p$ t.c. vale **pumping lemma**. Devo scegliere $w \in L$, es $w = (01)^p$ con $|w| = 2p \geq p$

Posso scomporla in:



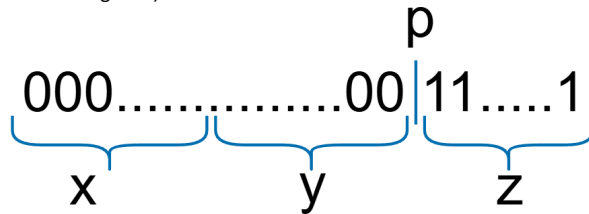
(con x vuota)

In questo caso il linguaggio è regolare poiché siccome stiamo aggiungendo "0" e "1" in coppia il loro numero rimarrà sempre uguale quindi anche se pompiamo y rimarremo sempre con lo stesso numero di "0" e "1". Questo esempio quindi non va bene per contraddire il pumping lemma

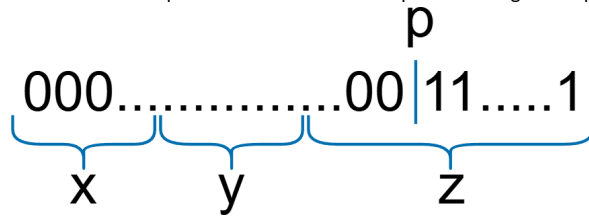
Usando l'esempio di prima proviamo con $w = 0^p 1^p \in L$ e $|w| = 2p$. Ora provo a scomporre $w = xyz$ in tutti i modi legali (cioè $|xy| \leq p$, $|y| > 0$)

Come prima, $|xy| \leq p$, allora y è fatta solo di "0"

- Caso con $|xy| = p$ ($|xy| = |z| = p$; xy comprende tutti gli "0")



- Caso con $|xy| < p$ ($|xy| < p < |z|$; xy comprende meno "0" rispetto al loro totale e z comprende tutti gli "1" e parte degli "0")



Quindi pompando y il num di "0" supererà quello di "1"

Più precisamente ci sono due casi:

- $|y| = k > 0$; $k \leq p$; $|x| = p - k$ e $|z| = p$ (quindi $|xy| = p$ comprende tutti gli "0" e $|z| = p$ comprende tutti gli "1")

Tuttavia $|xy^2z| = (p - k) + 2k + p = 2p + k$. Però:

- $\#_0 = (p - k) + 2k = p + k$
- $\#_1 = p < p + k$

- $|y| = k > 0$; $|z| = p + l$ e $|x| = p - k - l$; $l > 0$ (quindi z comprende anche una parte degli "0" oltre a tutti i p "1")

Tuttavia $|xy^2z| = (p - k - l) + 2k + (p + l) = 2p + k$. Come prima:

- $\#_0 = (p - k - l) + 2k + l = p + k$
- $\#_1 = p < p + k$

Quindi in entrambi i casi il num. di "0" è maggiore del num di "1"

Grammatiche Acontestuali

mercoledì 15 ottobre 2025 17:02

Introduciamo un modello di computazione più potente, utile in diverse applicazioni (es: parser per i compilatori):

Le **grammatiche acontestuali**

Vedremo che le grammatiche coincidono anche con un **diverso tipo di automa (PDA)** e vedremo anche linguaggi che non sono riconoscibili da nessuno di questo tipo di automi

Pero sono utili per dare facilmente una grammatica che genera stringhe tipo $0^n 1^n$ con $n \geq 0$ (tipi di linguaggi non regolari)

Grammatiche Acontestuali (CFG) (Context Free Grammar)

Una **grammatica** è una sequenza di **sostituzioni e produzioni**.

Es di **regole di una grammatica G**:

- **R1**: $A \rightarrow 0A1$
- **R2**: $A \rightarrow B$
- **R3**: $B \rightarrow \#$

Dove A, B sono **variabili**, "0, 1, #" sono detti **terminali** e " $\rightarrow$ " è il simbolo di **sostituzione**

La grammatica genera stringhe:

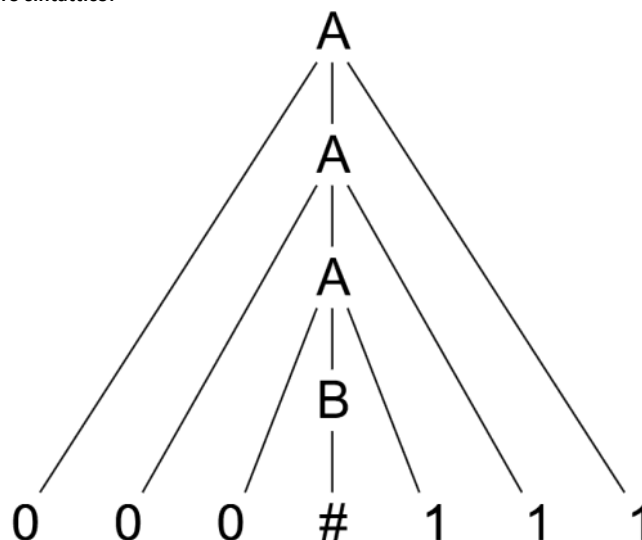
- 1) **Scrivo variabile iniziale**
- 2) **Sostituisco variabili con altre espressioni** applicando le regole della grammatica
- 3) **Ripeto finchè non ci sono più variabili**

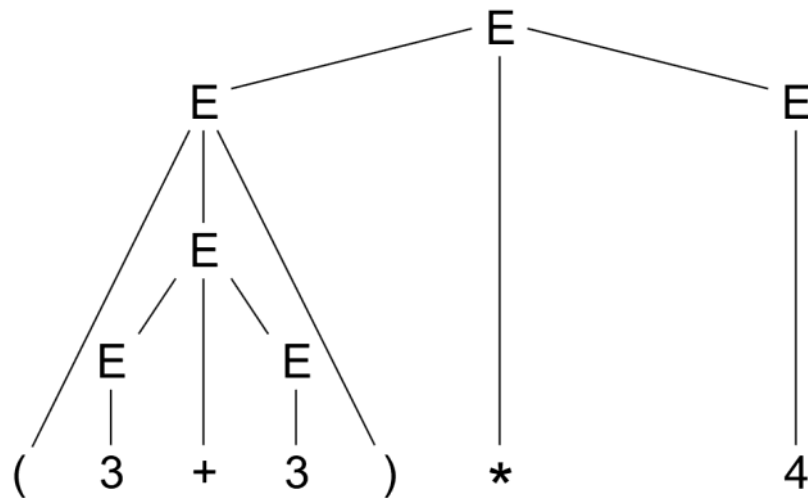
Es:

$A \rightarrow (R1) 0A1 \rightarrow (R1) 00A11 \rightarrow (R1) 000A111 \rightarrow (R2) 000B111 \rightarrow (R3) 000\#111$

(possiamo applicare le regole quando vogliamo)

Per ogni derivazione posso associare un **albero sintattico**:





Definizione di CFG

Una **CFG** è una tupla (V, Σ, R, S) dove:

- V = insieme finito di **variabili**
- Σ = insieme finito **terminale** ($V \cap \Sigma = \emptyset$)
- R = insieme di **regole**
- $S \in V$ è la **variabile iniziale**

Se:

- $u, v, w \in (V \cup \Sigma)$ (possono essere variabili o terminali)
- $(A \rightarrow w) \in R$ (c'è la regola $A \rightarrow w$)

allora diremo che "**uAv**" **produce** "**uvw**" e lo scriviamo come

$$uAv \rightarrow uvw$$

Diciamo anche che **u deriva v**, ovvero $u \rightarrow^* v$ se:

- $u = v$, oppure
- $\exists u_1, \dots, u_k$ con $k \geq 1$ t.c. $u \rightarrow u_1 \rightarrow u_2 \rightarrow \dots \rightarrow u_k \rightarrow v$

Questo ci permette di definire il **linguaggio associato ad una grammatica CFG**

Sia $G = (V, \Sigma, R, S)$ allora $L(G) = \{w \in \Sigma^* : S \rightarrow^* w\}$ (insieme di stringhe terminali che posso produrre partendo dalle variabili iniziali)

Esempio:

$G = (V = \{S\}, \Sigma = \{a, b\}, R, S)$

- **$R: S \rightarrow aSb \mid SS \mid \epsilon$** (il simbolo " $\mid$ " indica "oppure", cioè possiamo sostituire S con " aSb " oppure " SS " oppure " ϵ ")

Quindi avremmo:

- $ab \in L(G): S \rightarrow aSb \rightarrow a\epsilon b = ab$
- $aabb \in L(G): S \rightarrow aSb \rightarrow aaSbb \rightarrow aa\epsilon bb = aabb$
- $abab \in L(G): S \rightarrow SS \rightarrow aSbS \rightarrow aSbaSb \rightarrow a\epsilon baSb \rightarrow a\epsilon ba\epsilon b = abab$

Controlliamo che ab siano **ben annidati** (quindi non possiamo avere mai " ba " o solo " a ") come i compilatori

Costruzione di Grammatiche

Iniziamo a vedere alcune tecniche per **costruire grammatiche**:

- 1) **Unione di grammatiche**
- 2) **Da DFA a grammatiche**
- 3) Sfruttare la **ricorsione** (per linguaggi di lunghezza **infinita** con un certo pattern, tipo 0^n1^n)

È spesso utile la ricorsione nei CFG poiché regole del tipo $R \rightarrow OR1$ consentono di **ricordare "informazione limitata"** (modo per generare stringhe con lo stesso num. di "0" e "1").

Rispetto agli automi, per le grammatiche non vale la chiusura per tutte le operazioni (es: intersezione, complemento)

Unione di Grammatiche

Supponiamo di avere $G_i = (V_i, \Sigma_i, R_i, S_i)$ tutte CFG $\forall i \in [k]$ ($[k] = 1, 2, 3, \dots, k$)

Vogliamo una CFG $G = (V, \Sigma, R, S)$ t.c.

$$L(G) = \bigcup_i L(G_i)$$

Cioè il linguaggio generato da G è l'unione dei linguaggi generati da tutte le G_i

Idea:

- S = nuova variabile (siccome una grammatica deve avere **una sola** var iniziale (ogni G_i ha la sua S_i))

$$V = \bigcup_i (V_i) \cup \{S\}$$

wlog, possiamo assumere $\forall i \cap V_j \neq \emptyset \quad \forall i, j \in [k]$ t.c. $i \neq j$ (le variabili sono disgiunte)

$$\Sigma = \bigcup_i \Sigma_i$$

$$R = \bigcup_i (R_i) \cup \{S \rightarrow S_1 \mid \dots \mid S_k\}$$

Oltre ad unire insieme le regole delle varie grammatiche, aggiungiamo la regola che partendo da S , possiamo transizionare su qualsiasi grammatica G_i

Correttezza: $\bigcup_i L(G_i) = L(G)$ (cioè l'unione dei linguaggi generati dai vari G_i è il linguaggio generato da G)

Bisogna dimostrare che $\bigcup_i L(G_i) \subseteq L(G)$ e $L(G) \subseteq \bigcup_i L(G_i)$

⇒ Dimostro $\bigcup_i L(G_i) \subseteq L(G)$

Sia $w \in \bigcup_i L(G_i) : \exists i \in [k] \text{ t.c. } w \in L(G_i)$ (w è generabile a partire dalle sostituzioni tramite la grammatica di G_i)

Ovvero $S_i \rightarrow_{G_i}^* w$

Ma allora, per definizione:

$S \rightarrow S_i \rightarrow_{G_i}^* w$

(siccome $S \rightarrow S_1 \mid S_2 \mid \dots \mid S_k$ da S posso sempre scegliere una grammatica G_i tramite $S \rightarrow S_i$)

Siccome partiamo da S abbiamo che $w \in L(G)$, quindi $\bigcup_i L(G_i) \subseteq L(G)$

⇐ Dimostro $L(G) \subseteq \bigcup_i L(G_i)$

Sia $w \in L(G)$, quindi $S \rightarrow_G^* w$, quindi per definizione $\exists i \in [k] \text{ t.c.}$

$S \rightarrow S_i \rightarrow_{G_i}^* w$

Siccome le variabili di G_i sono disgiunte da tutte le altre variabili degli altri G_j (con $i \neq j$), allora:

$S \rightarrow S_i \rightarrow_{G_i}^* w$

(scelto S_i le transizioni avvengono solo tramite le regole di G_i)

Quindi $w \in L(G_i)$, perciò $L(G) \subseteq \bigcup_i L(G_i)$

Da DFA a CFG

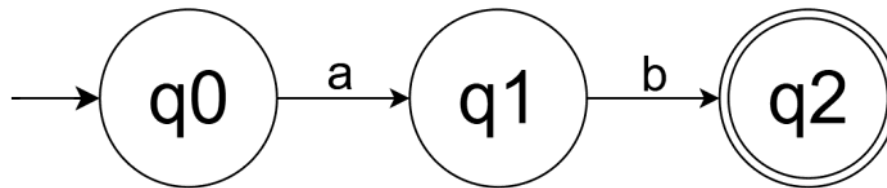
Dato DFA $D = (Q, \Sigma, \delta, q_0, F)$ voglio definire CFG $G = (V, \Sigma, R, S)$ t.c. $L(G) = L(D)$ (La grammatica riconosce lo stesso linguaggio dell'automa)

Idea:

- $V = \{V_q : q \in Q\}$ (ad ogni variabile è associato uno stato dell'automa, quindi $|V| = |Q|$)
- $S = V_{q_0}$
- Aggiungo la regola $V_q \rightarrow aV_p \forall p, q \in Q, a \in \Sigma \text{ t.c.}$
 $\delta(q, a) = p$
- $\forall q \in F$, aggiungo $V_q \rightarrow \epsilon$ (se q è finale allora lo stato V_q con cui siamo arrivati alla stringa con tutti terminali + V_q può essere rimosso per lasciarci la stringa finale)
(Es $abcV_q \rightarrow abce = abc$)

Esercizio: Dimostrare la correttezza in generale

Esempio:



$w = ab \in L(D)$.

Creiamo un CFG $G = (\{V_{q0}, V_{q1}, V_{q2}\}, \Sigma, R, \{V_{q0}\})$ con le regole:

- R1:** $S = V_{q0} \rightarrow aV_{q1}$
 - R2:** $V_{q1} \rightarrow bV_{q2}$
 - R3:** $V_{q2} \rightarrow \epsilon$
- $S = V_{q0} \rightarrow aV_{q1} \rightarrow abV_{q2} \rightarrow ab\epsilon = ab$

Pumping Lemma per CFG

Non tutti i linguaggi sono acontestuali.

Intuizione: siccome le CFG generano stringhe di lunghezza infinita ma $|V|$ è finita, ci saranno derivazioni di lunghezza variabile che si ripetono

Dimostrazione:

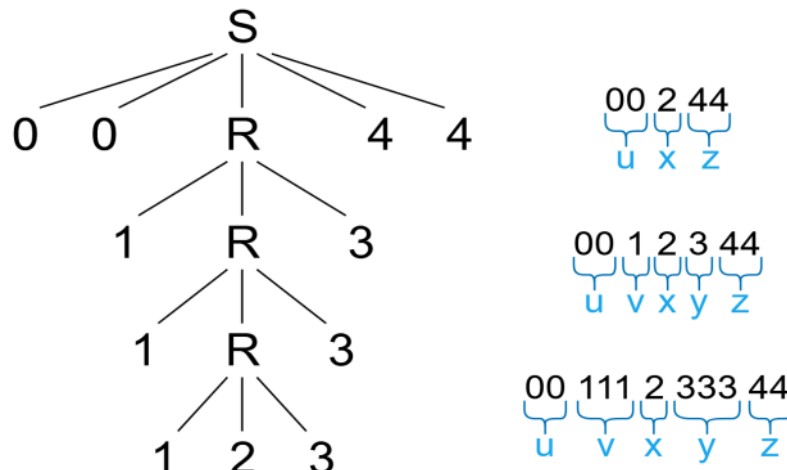
Sia L un CFG, esiste p (lunghezza pumping) t.c. se $w \in L$ con $|w| \geq p$, allora è possibile scomporre $w = uvxyz$ in modo che valgano le proprietà:

- $\forall i \geq 0 : uv^i xy^i z \in L$
- $|vy| > 0$ (vy esistono)
- $|vxy| \leq p$

Es: $L = \{001^n 23^n 44 : n \geq 0\}$

- $S \rightarrow 00R44$
- $R \rightarrow 1R3 \mid 2$

La variabile R si deve ripetere



Ovvero $uv^i xy^i z$ è nel linguaggio $\forall i \geq 0$

Siccome la stringa è infinita, se la stringa è abbastanza lunga, ci sarà una ripetizione nelle variabili. Questa ripetizione sarà quella che ci permetterà di definire p (come nel pumping dei DFA il pumping aumenta gli stati, qui il pumping aumenta le variabili)

Dimostrazione: Wlog assumiamo G in **forma normale di Chomsky (CNF)**, quindi l'albero di derivazione sarà binario

Affermazione: ogni albero generato da G , il cui cammino più lungo ha **lunghezza i** , allora questo albero genera stringhe di lunghezza $\leq 2^{i-1}$

Dimostrazione affermazione: Per induzione sulla lunghezza del cammino.

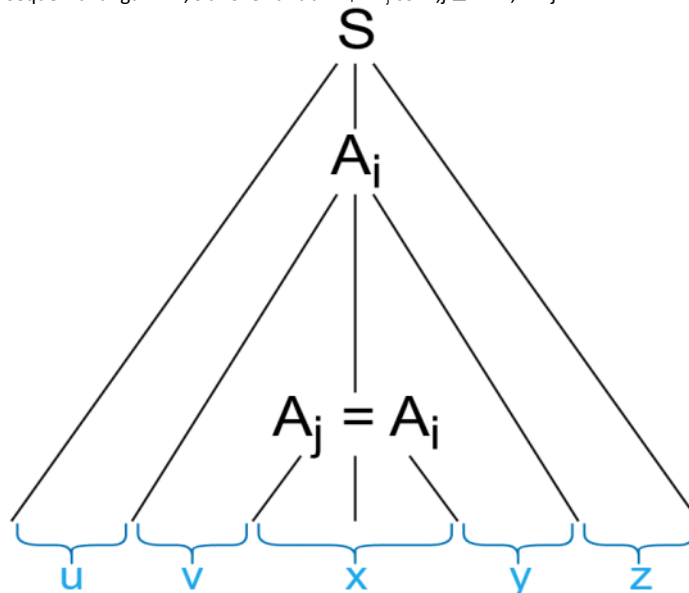
- **Caso base:** Se il cammino è lungo 1, abbiamo $S \rightarrow a$ dove $a \in \Sigma$. Infatti $|a| = 2^{1-1} = 2^0 = 1$
- **Passo induttivo:** Sia vero per cammini di lunghezza $i-1$ e supponiamo il cammino più lungo sia di lunghezza $i > 1$.
Per questo cammino la **prima sostituzione** sarà $S \rightarrow AB$, perché G è in forma normale e il cammino è lungo almeno 2.
I sottoalberi radicati in A e B avranno lunghezza $i-1$ e quindi generano cammini di lunghezza $i-1$, quindi quei cammini per induzione generano stringhe lunghe al più $2^{i-1-1} = 2^{i-2}$. Quindi S genera una stringa lunga $\leq 2 * 2^{i-2} = 2^{i-1}$

- **Dimostrazione proprietà 1: si può scomporre w in $uv^i xy^j z$**

Sia $|V| = m$ (num. variabili) in G , pongo $p = 2^m$. Segue che una stringa w t.c. $|w| \geq p$, e poiché se un cammino è di lunghezza i può generare stringhe di lunghezza almeno 2^{i-1} , quindi il cammino che genera w deve essere almeno $m+1$

Questo cammino ha **$m+2$ nodi** di cui uno è terminale e ci sono **$m+1$ variabili**, quindi almeno **una variabile si deve ripetere** (siccome abbiamo solo m variabili e $m+1$ nel cammino)

Scegliamo la **prima ripetizione** nella sequenza lunga $m+1$, siano le variabili $A_i = A_j$ con $i, j \leq m+1, i \neq j$



L'albero di radice A_i genera vxy , l'albero di radice A_j genera x , e le stringhe u e z sono la parte terminale sx e dx del cammino da S a A_i

- **Dimostrazione proprietà 2: $vy \neq \epsilon$**

vx non deve essere stringa vuota perché G consente di derivare $A_i \Rightarrow^* A_j$

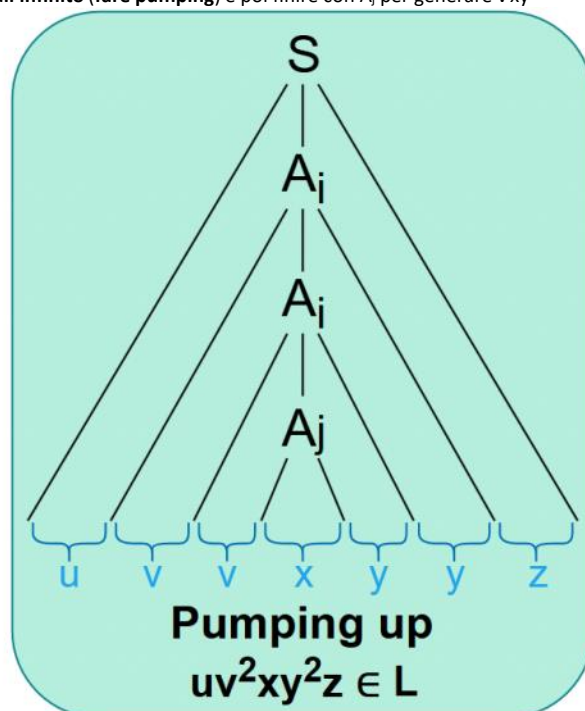
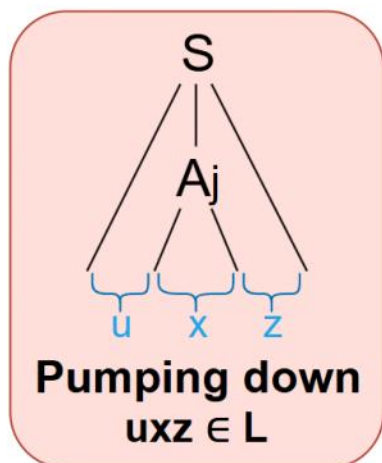
Ma le uniche regole compatibili con questo in G sono del tipo $A_i \rightarrow BC$ t.c. C deriverà A_i e B deriverà v oppure $y \Rightarrow vxy \neq x \Rightarrow |vy| > 0$

- **Dimostrazione proprietà 3: $|vxy| \leq p$**

Per proprietà 1 siccome abbiamo scelto A_i come la prima ripetizione in una sequenza lunga $\leq m+1$, questo cammino genera stringhe lunghe $\leq 2^{m+1-1} = 2^m = p$

Posso **sostituire A_i con A_j** e viceversa e avrò due casi:

- **Pumping down:** scambiando A_i con A_j (siccome sono la stessa variabile) posso generare direttamente x **senza generare vy** (questo non viola la proprietà 2 perché richiede che $vy \neq \epsilon$ nella scomposizione originale e in questo pumping abbiamo $i = 0$, quindi $uv^0xy^0z = uxz$)
- **Pumping up:** possiamo scambiare A_i sempre con A_i in **loop all'infinito (fare pumping)** e poi finire con A_j per generare $v^i xy^i z$



Quindi $\forall i \geq 0, uv^i xy^i z \in L$

Es: $L = \{0^n 1^n 2^n : n \geq 1\}$ non è **CFL** (Context-Free Language)

Applico il **pumping lemma**. Suppongo che L sia CFL e prendo $w = 0^p 1^p 2^p$ con $p = \text{val. pumping}$. Ovviamente $|w| = 3p > p$

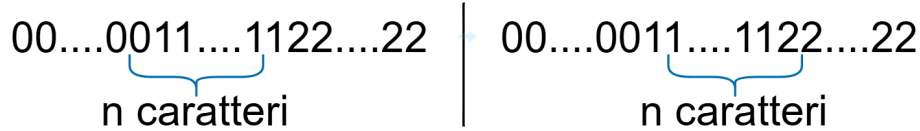
Sia $w = uvxyz$, siccome $|vxy| \leq p$, abbiamo che vxy contiene ≤ 2 simboli tra "0", "1", "2"

Poiché siccome tutti e 3 i simboli sono lunghi p , e $|vxy| \leq p$ allora vxy può solo:

- Stare tutto nel blocco "0" o "1" o "2"
- Coprire al massimo due blocchi adiacenti

Ma **non può mai coprire tutti e tre i blocchi**, poiché $3p > p \geq |vxy|$

Es: $w = 0...01...12...2$ e prendiamo una sottostringa vxy . Se ci sono "0" e "1" non ci sono "2" o se ci sono "1" e "2" non ci sono "0"



Facendo **pumping down** con $i = 0$ e considerando:

uv^0xy^0z

avremmo che:

- Se vxy è **tutto dentro un blocco** (es solo "0") diminuirà il num. di "0" ma non di "1" o "2" perché stiamo togliendo vy dalla stringa
- Se vxy è **tra due blocchi** (es parte di "0" e di "1") diminuiranno insieme il num. di "0" e "1" ma non di "2"

Quindi in entrambi i casi almeno un simbolo ha un num. di caratteri minore rispetto agli altri due quindi $uv^0xy^0z \notin L$, perciò **L non è CFL**

Forma Normale di Chomsky (CNF)

Una CFG è in **forma normale (o canonica)** se ogni regola è del tipo:

$A \rightarrow BC$ oppure

$A \rightarrow a$

Con $A, B, C \in V$ (con $B, C \neq S$, quindi S è solo a sinistra), $a \in \Sigma$

Inoltre è ammessa la regola $S \rightarrow \epsilon$

Teorema: ogni CFG ammette una CFG equivalente in forma normale

Dimostrazione con esempio:

- **Aggiungo var. iniziale** S_0 insieme alla regola $S_0 \rightarrow S$ e usiamo S_0 come variabile iniziale (quindi non si trova a destra)
- **Elimino la ϵ -regola** $A \rightarrow \epsilon$. Per ogni occorrenza di una regola con A a destra, aggiungo nuove regole con l'occorrenza di A cancellata. Es:
 - $R \rightarrow uAv$ aggiungo $R \rightarrow uv$
 - $R \rightarrow A$ aggiungo $R \rightarrow \epsilon$ (che sarà poi da rimuovere)
 - $R \rightarrow uAvAw$ aggiungo
 - $R \rightarrow uvAw$
 - $R \rightarrow uAvw$
 - $R \rightarrow uvw$
- **Elimino le regole unitarie** (del tipo $A \rightarrow B$) e per ogni occorrenza di regole $B \rightarrow u$ ($u \in \Sigma^*$) aggiungo $A \rightarrow u$ (a meno che questa regola non era già stata eliminata)
- Infine trasformo **regole restanti**, tipo:
 - $A \rightarrow u_1, u_2, \dots, u_k$ per $k \geq 3$, farò
 - $A \rightarrow u_1A_1; A_1 \rightarrow u_2A_2; \dots; A_{k-2} \rightarrow u_{k-1}u_k$
 Dove A_i sono nuove variabili. Se u_i è **terminale** lo sostituisco con una var. U_i e aggiungo la regola $U_i \rightarrow u_i$
(Spezzo una regola lunga in più regole binarie, creando nuove variabili intermedie)

Esempio CFG generica a cui abbiamo aggiunto la nuova regola

- **R1:** $S_0 \rightarrow S$
- **R2:** $S \rightarrow ASA \mid aB$
- **R3:** $A \rightarrow B \mid S$
- **R4:** $B \rightarrow b \mid \epsilon$

Elimino le ϵ -regole $B \rightarrow \epsilon$:

- **R1:** $S_0 \rightarrow S$
- **R2:** $S \rightarrow ASA \mid aB \mid a$
- **R3:** $A \rightarrow B \mid S \mid \epsilon$
- **R4:** $B \rightarrow b$

Elimino la ϵ -regola $A \rightarrow \epsilon$:

- **R1:** $S_0 \rightarrow S$
- **R2:** $S \rightarrow ASA \mid SA \mid AS \mid aB \mid a$
- **R3:** $A \rightarrow B \mid S$
- **R4:** $B \rightarrow b$

Elimino la regola unitaria $S \rightarrow S$ e $S_0 \rightarrow S$:

- **R1:** $S_0 \rightarrow ASA \mid SA \mid AS \mid aB \mid a$
- **R2:** $S \rightarrow ASA \mid SA \mid AS \mid aB \mid a$
- **R3:** $A \rightarrow B \mid S$
- **R4:** $B \rightarrow b$

Elimino $A \rightarrow B$:

- **R1:** $S_0 \rightarrow ASA \mid SA \mid AS \mid aB \mid a$
- **R2:** $S \rightarrow ASA \mid SA \mid AS \mid aB \mid a$
- **R3:** $A \rightarrow S \mid b$
- **R4:** $B \rightarrow b$

Elimino $A \rightarrow S$:

- **R1:** $S_0 \rightarrow ASA \mid SA \mid AS \mid aB \mid a$
- **R2:** $S \rightarrow ASA \mid SA \mid AS \mid aB \mid a$
- **R3:** $A \rightarrow b \mid ASA \mid SA \mid AS \mid aB \mid a$
- **R4:** $B \rightarrow b$

Completo le sostituzioni

- **R1:** $S_0 \rightarrow AA_1 \mid SA \mid AS \mid UB \mid a$
- **R2:** $S \rightarrow AA_1 \mid SA \mid AS \mid UB \mid aw$
- **R3:** $A \rightarrow b \mid AA_1SA \mid SA \mid AS \mid UB \mid a$

- **R4:** $B \rightarrow b$
- **R5:** $A_1 \rightarrow SA$
- **R6:** $U \rightarrow a$

Automi a Pila (PDA) (Push-Down Automata)

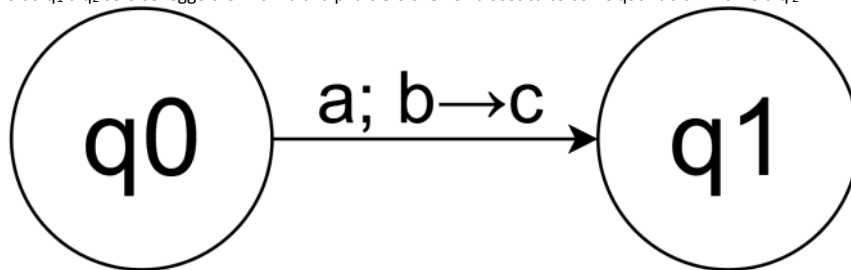
Un PDA è un NFA con una pila **LIFO (o stack)** che permette di riconoscere **linguaggi non regolari** (es $w = 0^n 1^n$). Infatti i PDA sono equivalenti alle CFG. La pila è inizialmente vuota e ad ogni passo di computazione il PDA può **operare sulla cima della pila**:

- **Sostituzione** in cima alla pila, del carattere che è in cima alla pila con un altro carattere
- **PUSH**: **inserimento** del simbolo in cima
- **POP**: **elimino** il simbolo in cima

Non esiste una **pila universale** per tutto l'automa, ma ogni ramo di computazione ha associata una sua pila

Dal punto di vista grafico su un arco abbiamo a sinistra del punto e virgola il carattere che leggiamo e a destra l'operazione che avviene sulla pila

Es sostituzione: "a; b $\rightarrow$ c" = passiamo da q_1 a q_2 solo se leggo a e in cima alla pila c'è b che verrà sostituito con c quando arriviamo a q_2



L'alfabeto di pila Γ_ϵ (gamma epsilon) = $\Gamma \cup \{\epsilon\}$ può essere diverso da Σ_ϵ

Definizione di PDA

Un PDA è una tupla $(Q, \Sigma, \Gamma, \delta, q_0, F)$ dove $Q, \Sigma, \Gamma, \delta, q_0, F$ sono le stesse degli NFA, inoltre:

- Γ è l'**alfabeto finito di pila**
- $\delta: Q \times \Sigma_\epsilon \times \Gamma_\epsilon \rightarrow \mathcal{P}(Q \times \Gamma_\epsilon)$
Che succede in una transizione?
(q, c) $\in \delta(p, a, b)$ dove $p, q \in Q, a \in \Sigma_\epsilon, b, c \in \Gamma_\epsilon$ con i casi:
 - **Sostituzione b con c**: a in lettura, $b \neq \epsilon, c \neq \epsilon$
 - **PUSH di b**: a in lettura, $b = \epsilon, c \neq \epsilon$
 - **POP di b**: a in lettura, $b \neq \epsilon, c = \epsilon$
- **Configurazione** ("foto" del PDA in tempo t): elementi di $Q \times \Sigma^* \times \Gamma^*$

Accettazione: PDA M accetta $w = w_1 \dots w_n$ t.c. $w_i \in \Sigma$ se $\exists$ stati $r_0, \dots, r_m$ con $r_i \in Q$ e stringhe $s_0, \dots, s_m \in \Gamma^*$ (stack) t.c.:

- All'inizio, $r_0 = q_0$ e $s_0 = \epsilon$ (partiamo dal primo stato e lo stack è vuoto)
- $\forall i = 0, \dots, m$ abbiamo:
(r_{i+1}, b) $\in \delta(r_i, w_{i+1}, a)$
dove $s_i = bt$ e $s_{i+1} = at$ con $a, b \in \Gamma_\epsilon, t \in \Gamma^*$ (resto dello stack)
(ho sostituito a con b nel passaggio da r_i a r_{i+1} leggendo w_{i+1})
- $r_m \in F$ (nell'ultimo stato l'automa accetta)

Posso mettere le configurazioni in relazione

$$(p, ax, by) \vdash_M (q, x, cy) \Leftrightarrow (q, c) \in \delta(p, ab)$$

con $q, p \in Q, a \in \Sigma_\epsilon, x \in \Sigma^*, b, c \in \Gamma_\epsilon, y \in \Gamma^*$

(l'automa sta nello stato p, legge ax e sulla cima dello stack c'è b (y è il resto dello stack), allora può andare nello stato q, sostituendo b con c)

Come fatto per i DFA/NFA posso considerare la **chiusura simmetrica e transitiva** " $\vdash_M^*$ ". Pertanto:

$$L(M) = \{w \in \Sigma^* : (q_0, w, \epsilon) \vdash_M^* (q, \epsilon, y) \text{ con } q \in F, y \in \Gamma^*\}$$

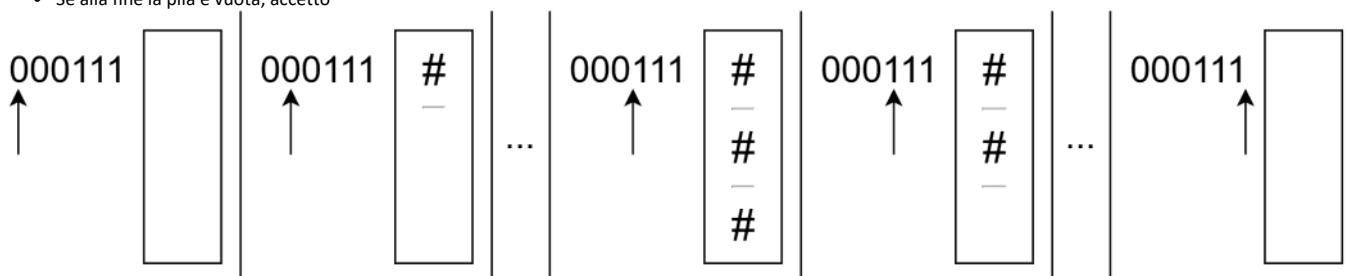
($L(M)$ = tutte le stringhe t.c. la configurazione iniziale è in relazione estesa con lo stato finale con la pila contenente un carattere qualunque (anche vuoto))

Nota: PDA accetta indipendentemente dal contenuto della pila

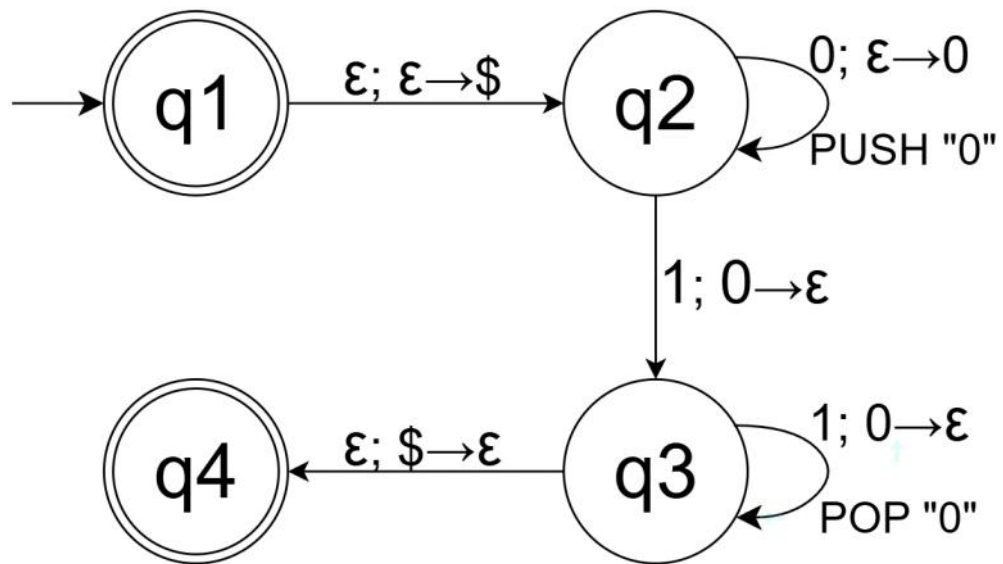
Esempio 1: DFA per $L = \{w = 0^n 1^n : n \geq 0\}$

Idea: $w = 000111$.

- Finchè leggo "0" faccio PUSH di "#"
- Finchè leggo "1" faccio POP di "#"
- Se alla fine la pila è vuota, accetto



- $Q = \{q_1, q_2, q_3, q_4\}$
- $\Sigma = \{0, 1\}, \Gamma = \{0, \$\}$ (\$ serve per capire se ho finito la pila; se ho finito tutto devo leggere \$)
- $F = \{q_1, q_4\}; q_0 = q_1$



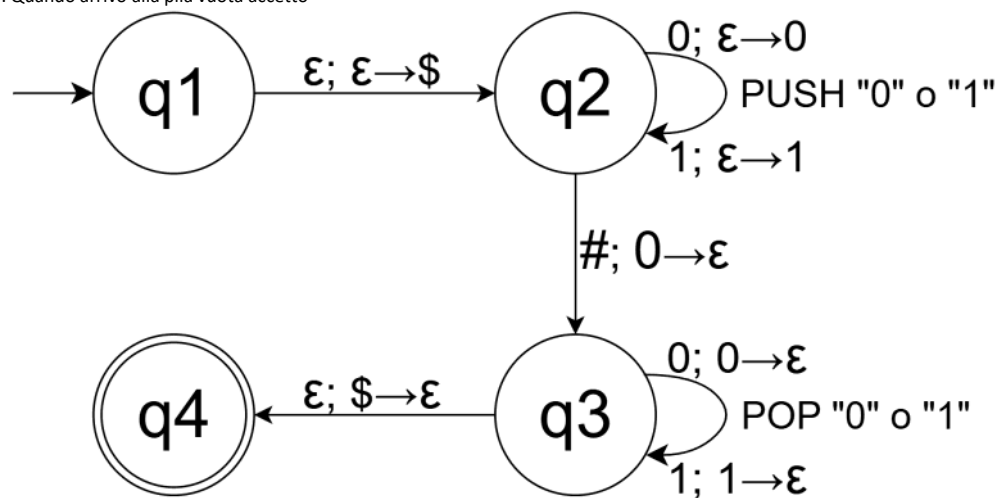
(finiamo solo se c'è \$ rimasto nella pila)

Esempio 2: $L = \{w\#w^R : w \in \{0, 1\}^*\}$ (w e w^R sono separate da #)

- $Q = \{q_1, q_2, q_3, q_4\}$
- $\Sigma = \{0, 1, \#\}$
- $\Gamma = \{0, 1, \$\}$
- $F = \{q_4\}; q_0 = q_1$

(q_1 non è accettante perché c'è "#" in mezzo alla stringa)

Idea: copio i caratteri letti nello stack finché non incontra "#", a questo punto avrò nella pila w^R , quindi dopo aver letto # controllo che ogni carattere che leggo deve essere uguale al carattere in cima alla pila. Quando arrivo alla pila vuota accetto



("ε → ε" è obbligatorio)

Se non ci fosse # allora la stringa vuota è accettante ($q_1 \in F$) e sostituisco la transizione $q_2 \rightarrow q_3$ con " $\epsilon; \epsilon \rightarrow \epsilon$ " perché quando passa da q_2 a q_3 avremmo comunque la stringa rovesciata nello stack

Equivalenza tra Grammatiche e PDA

Un linguaggio è **acontestuale** (generato da CFG) se e solo se esiste un PDA che lo riconosce

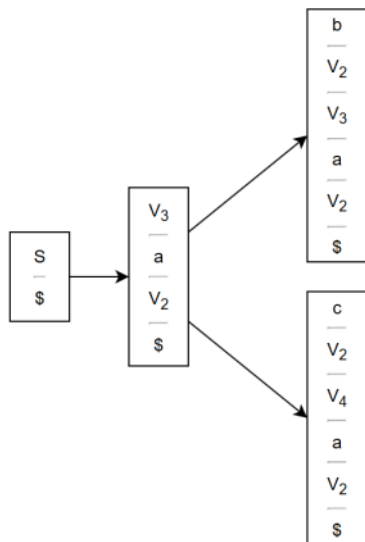
Dimostrazione nelle due direzioni:

- **Lemma 1:** Se L è acontestuale allora $\exists M \in \text{PDA}$ t.c. $L = L(M)$

Intuizione per la dimostrazione: il PDA deve riconoscere l'insieme delle stringhe generabili usando le regole della grammatica

Sia G la grammatica con S la var. iniziale e R le regole. Es:

- **R1:** $S \rightarrow V_1 a V_2$
- **R2:** $V_1 \rightarrow b V_2 V_3 \mid c V_2 V_4$



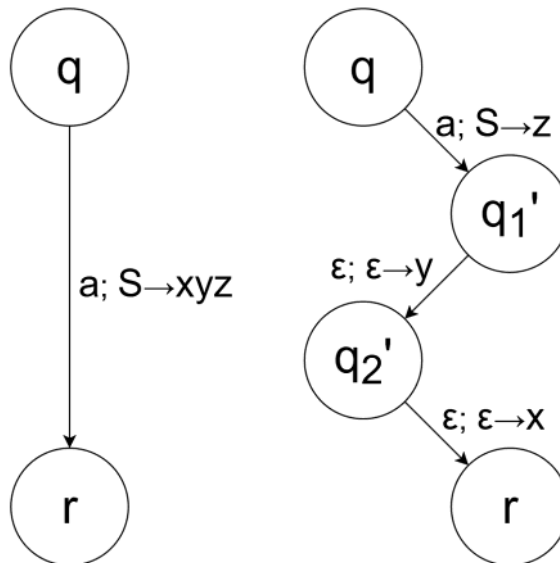
Ad alto livello (ancora sull'idea):

- Inserisce \$ nella pila
- **Ripete:**
 - Se in cima alla pila c'è una **variabile** A seleziona **non-deterministicamente** (prova tanti rami in parallelo) una delle regole di G di tipo $A \rightarrow \dots$ e opera la sostituzione di A
 - Se in cima alla pila c'è un **terminale** a controllo se è il carattere che mi serve per passare al prossimo stato. Altrimenti bisogna **rifiutare** quel ramo di derivazione
- Se arrivo allo stato di accettazione e ho **controllato tutti i caratteri della pila** (quindi rimane solo \$) allora accetto quel ramo e significa che L è **acontestuale**

Dimostrazione del lemma 1:

Sia $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$. Per semplificare la descrizione **abbreviamo le transizioni** in questo modo:

$(r, xyz) \in \delta(q, a, S)$



(nelle transizioni sottointese si inseriscono le var/terminali **in modo inverso nello stack**, cioè prima "z", poi "y" e poi "x", poiché essendo uno **stack LIFO** gli oggetti inseriti per ultimi saranno i primi dello stack. Quindi per avere "x" nella cima bisogna inserirlo per ultimo)

In generale permetto l'azione

$(r, u) \in \delta(q, a, S)$ dove $u = u_1 \dots u_l$

Poiché posso comunque fare:

$(q_1', u_l) \in \delta(q, a, S)$

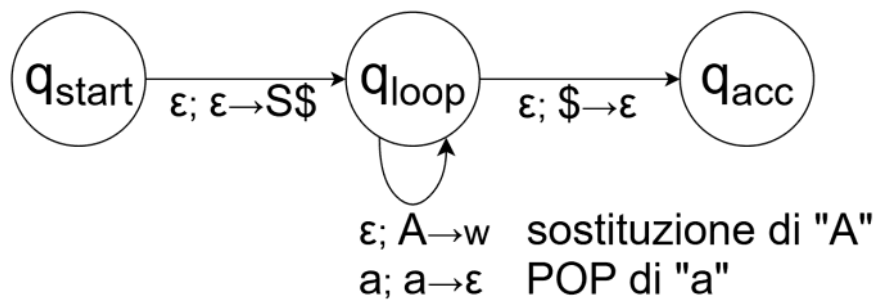
$(q_2', u_{l-1}) \in \delta(q_1', \epsilon, \epsilon)$

...

$(r, u_1) \in \delta(q_{l-1}', \epsilon, \epsilon)$

Definiamo il PDA:

- $Q = \{q_{start}, q_{loop}, q_{acc}\} \cup Q'$ (Q' = stati ausiliari che permettono di abbreviare le transizioni)
 - q_{start} = stato iniziale
 - q_{acc} = unico stato finale
 - q_{loop} = stato in cui rimango per sostituire variabili o leggere terminali in modo non deterministico
- δ :
 - Inizializzazione: inserisco "\$\$" nella pila, ovvero:
$$\delta(q_{start}, \epsilon, \epsilon) = \{(q_{loop}, \$\$)\}$$
 - Nello stato q_{loop} :
 - Se la cima dello stack è una **variabile** $A \in V$, allora
$$\delta(q_{loop}, \epsilon, A) = \{(q_{loop}, w) : A \rightarrow w \text{ in } G\}$$
(se senza leggere niente, prendo tutte le regole in cui A è a sinistra e scrivo w nello stack in modo non deterministico)
 - Se la cima dello stack è un **terminale** $a \in \Sigma$, allora
$$\delta(q_{loop}, a, a) = \{(q_{loop}, \epsilon)\}$$
(se in cima c'è a e leggo a faccio POP di a)
 - Se la cima dello stack è \$, allora
$$\delta(q_{loop}, \epsilon, \$) = \{(q_{acc}, \epsilon)\}$$
(passiamo allo stato di accettazione perché abbiamo finito)



Esempio: Sia G la grammatica

- **R1:** $S \rightarrow aTb \mid b$
- **R2:** $T \rightarrow Ta \mid \epsilon$

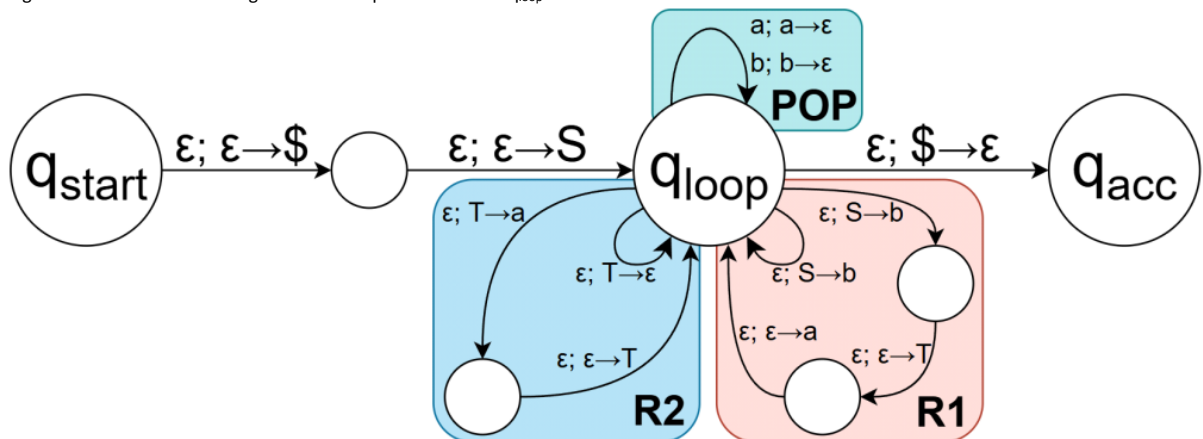
Costruire il PDA equivalente:

- **Nota:** Nella costruzione del PDA **non si possono semplificare le transizioni**, quindi se abbiamo una transizione del tipo " $a; S \rightarrow xyz$ " dovrà essere divisa in 3 stati differenti. Ad esempio la transizione " $\epsilon; \epsilon \rightarrow S\$$ " si deve dividere in due transizioni: prima " $\epsilon; \epsilon \rightarrow \$$ " e poi " $\epsilon; \epsilon \rightarrow S$ ".

Quindi dobbiamo separare:

- $\epsilon; S \rightarrow aTb$ in $\epsilon; S \rightarrow b$, $\epsilon; \epsilon \rightarrow T$, $\epsilon; \epsilon \rightarrow b$
- $\epsilon; T \rightarrow Ta$ in $\epsilon; T \rightarrow a$, $\epsilon; \epsilon \rightarrow T$

- Per ogni terminale dobbiamo eseguire il POP di quel terminale in q_{loop}



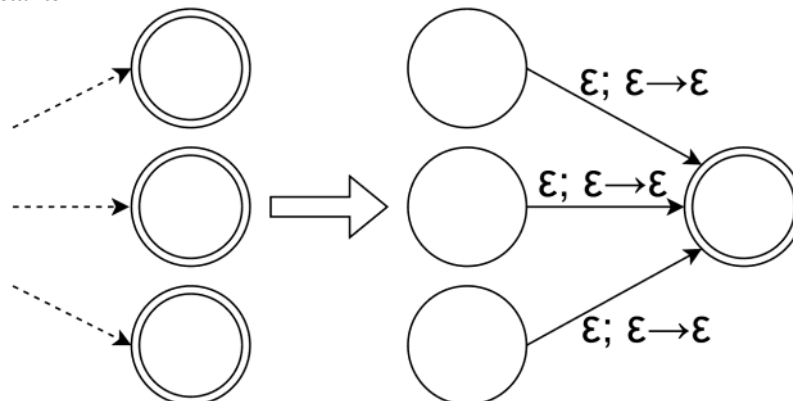
Es: $S \rightarrow aTb \rightarrow aTab \rightarrow aab$

- **Lemma 2:** Se $M \in \text{PDA}$ t.c. $L = L(M)$ allora L è acontestuale

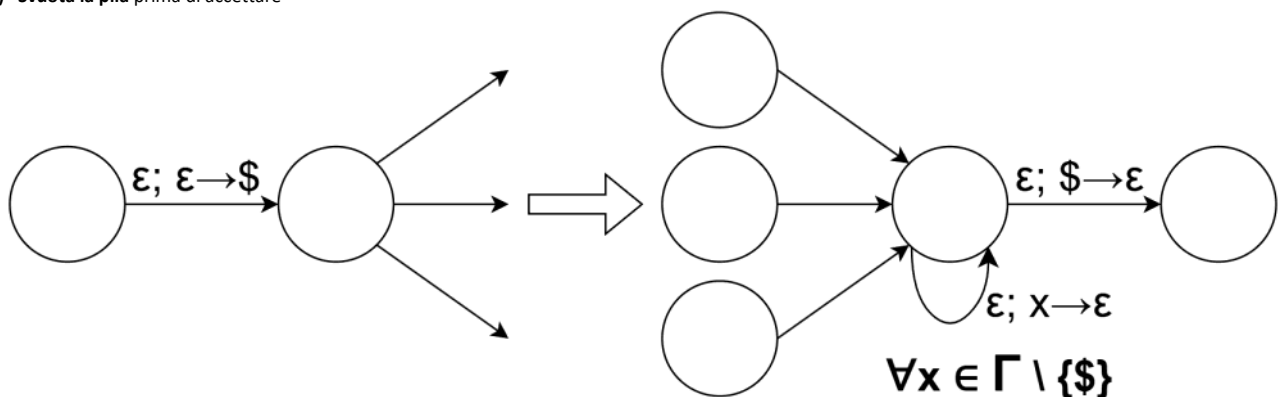
Intuizione per la dimostrazione: bisogna tradurre il PDA in una grammatica equivalente.

Wlog, in questo PDA:

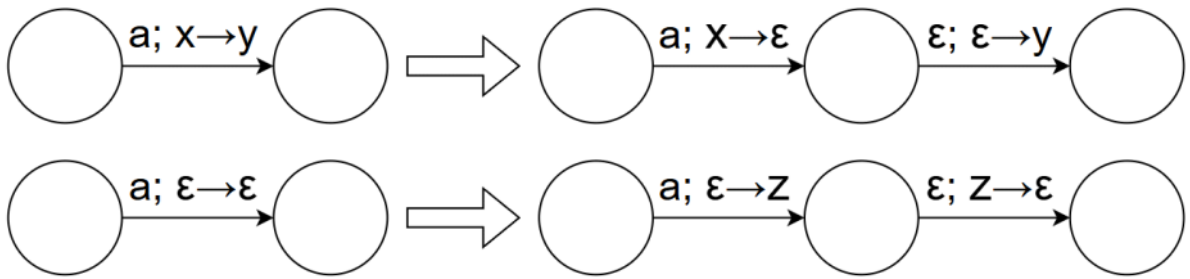
- 1) Abbiamo **un solo stato accettante**



- 2) **Svuota la pila prima di accettare**



- 3) Ogni transizione fa **POP e PUSH**

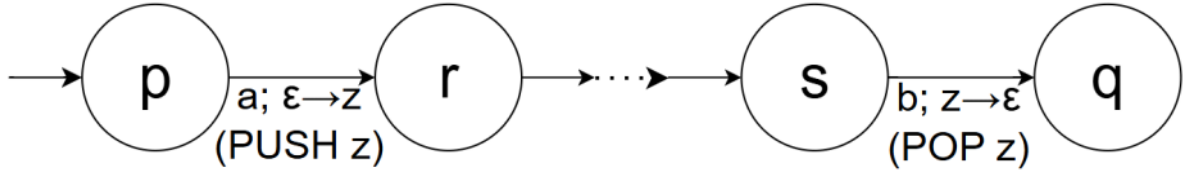


Osservazione: la grammatica deve generare tutte le stringhe che portano M da q_0 a q_{acc} con **pila vuota**

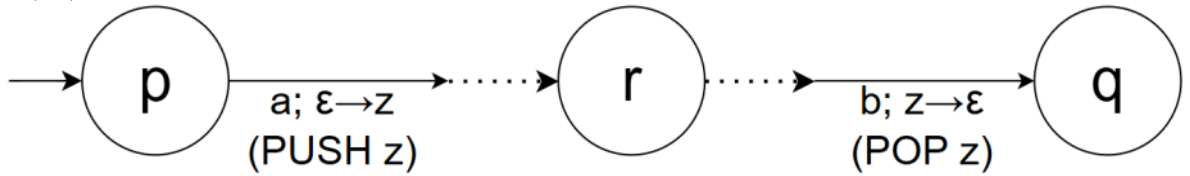
Idea: per ogni $p, q \in Q$ definisco $A_{pq} \in V$ e le regole codificano i modi in cui M può passare da p a q .

Avremmo quindi due casi:

- $A_{pq} \rightarrow a A_{rs} b$



- $A_{pq} \rightarrow A_{pr} A_{rq}$



Se guardiamo l'altezza della pila:

Inoltre posso sempre andare da p a q senza fare passi con $A_{pq} \rightarrow \epsilon$

Dimostrazione del Lemma 2:

Sia $M = (Q, \Sigma, \Gamma, \delta, q_0, \{q_{acc}\})$ e assumo che M sia nella forma di sopra del PDA descritto.

La grammatica G ha $V = \{A_{pq} : p, q \in Q\}$ e $S = A_{q_0 q_{acc}}$

Le regole saranno:

- $\forall p, q, r, s \in Q, u \in \Gamma, a, b \in \Sigma_\epsilon$
se $(r, u) \in \delta(p, a, \epsilon)$
e $(q, \epsilon) \in \delta(s, b, u)$
 $\Rightarrow A_{pq} \rightarrow a A_{rs} b$
- $\forall p, q \in Q, A_{pq} \rightarrow A_{pr} A_{rq}$
- $\forall p \in Q, A_{pp} \rightarrow \epsilon$

Affermazione 1: Se A_{pq} genera x , allora x porta M da p a q con pila vuota ($\forall p, q \in Q$)

Dimostrazione affermazione 1: Per induzione su # passi per derivare x da A_{pq}

- **Caso base:** # = 1 passo. L'unica regola per derivare x in 1 passo è $A_{pp} \rightarrow \epsilon$

Ovviamente ϵ porta M da p a p con pila vuota

- **Passo induttivo:** Sia vero per # passi $\leq k, k \geq 1$.

Assumo $A_{pq} \Rightarrow^* x$ in $k+1$ passi. La prima sostituzione è:

- $A_{pq} \rightarrow a A_{rs} b$
 $x = ayb$ e $A_{rs} \Rightarrow^* y$ in k passi. Quindi M può andare da r ad s con pila vuota. Inoltre
 $(r, u) \in \delta(p, a, \epsilon)$
 $(q, \epsilon) \in \delta(s, b, u)$

Quindi M inizia in p con la pila vuota, legge a e inserisce u nella pila, passa da $r \rightarrow s$ senza modificare la pila, fa POP di u e arriva in q con pila vuota

- $A_{pq} \rightarrow A_{pr} A_{rq}$
 $x = yz$ con $A_{pr} \Rightarrow^* y$ e $A_{rq} \Rightarrow^* z$
Inoltre queste derivazioni hanno lunghezza $\leq k$ quindi portano M da p ad r e da r a q con pila vuota
Quindi M su x va da p a q con pila vuota

Affermazione 2: Se x porta M da p a q con pila vuota allora A_{pq} genera x

$\Rightarrow w \in L(M)$ se solo se $S \Rightarrow^* w$

M su input w passa da q_0 a q_{acc} con pila vuota se e solo se $S = A_{q_0 q_{acc}} \Rightarrow^* w$

Dimostrazione affermazione 2: Per induzione su # passi di M

- **Caso base:** # = 0 passi, quindi $x = \epsilon$ e la grammatica ha la regola $A_{pp} \rightarrow \epsilon$
- **Passo induttivo:** Sia vero per # passi $\leq k, k \geq 1$

Suppongo M su x passi da p a q con pila vuota in $k+1$ passi. Devo mostrare che $A_{pq} \Rightarrow^* x$. Abbiamo due casi:

- La pila è **vuota all'inizio e alla fine**.

Nella prima mossa M fa **PUSH di u** e nell'ultima fa **POP di u**.

Sia a il carattere letto nella prima mossa e b nell'ultima e sia r lo stato dopo la prima mossa e s quello prima dell'ultima

$$(r, u) \in \delta(p, a, \epsilon)$$

$$(q, \epsilon) \in \delta(s, b, u)$$

Ma G contiene $A_{pq} \rightarrow a A_{rs} b$.

Sia $x = ayb$; inoltre M va da r a s senza modificare la pila quindi M su y può passare da r ad s con la pila vuota

M su y esegue $k+1-2 = k-1$ passi. Quindi $A_{rs} \Rightarrow^* y$ e quindi $A_{pq} \Rightarrow^* x$

- La pila **si svuota in mezzo**.

Sia r lo stato in cui la pila si **svuota**. La computazione di M da p ad r e da r ad q ha al più k passi

Sia $x = yz$ dove y corrisponde a $p \rightarrow r$ e z corrisponde a $r \rightarrow q$. Per ipotesi induttiva:

$$A_{pr} \Rightarrow^* y \text{ e } A_{rq} \Rightarrow^* z$$

$$\Rightarrow A_{pq} \Rightarrow^* x$$

Macchina di Turing

mercoledì 29 ottobre 2025 11:29

Abbiamo visto diversi modelli di calcolo, principalmente DFA/NFA e PDA. Sappiamo anche che ci sono linguaggi non riconoscibili da questi automi

Le TM sono state introdotte da Turing nel 1936 e corrispondono al modello astratto del computer

Vedremo che ci sono problemi intrinsecamente **non risolvibili** dai computer indipendentemente dalle risorse

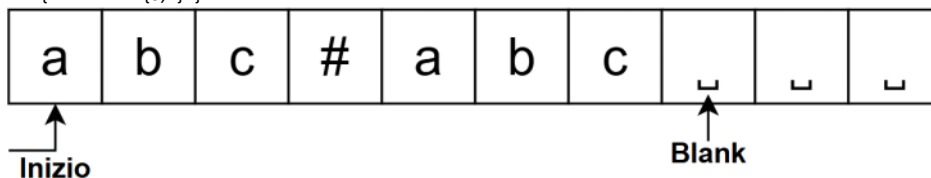
A livello intuitivo:

- La TM ha un **nastro** di lunghezza **infinita** e contiene l'**input all'inizio** e infinite **celle vuote (blank) dopo**
- La TM può **leggere/scrivere** sul nastro e **muovere** la testina a **Sx o Dx** (si muove in maniera arbitraria, non solo da sinistra a destra)

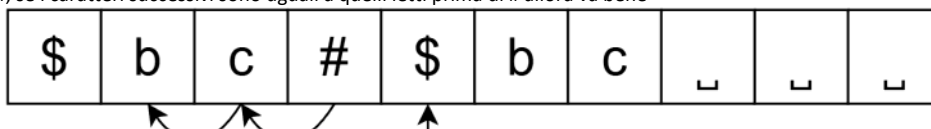
Esistono diverse varianti della TM ma sono tutti equivalenti

Inoltre la TM ha **due stati speciali accettazione/rifiuto** con effetto immediato (non deve controllare prima qualcosa)

Es: vogliamo fare una TM per $L = \{w\#w : w \in \{0,1\}^*\}$



Idea: leggo i caratteri fino a #, se i caratteri successivi sono uguali a quelli letti prima di # allora va bene



Leggo "a" a sinistra e la riscrivo con "\$", se il carattere dopo # è "a" (quello letto) allora lo sovrascrivo con "\$" e torno a sinistra e prendo il primo carattere dopo "\$" (a questo punto "b") e ripeto la lettura/sovrascrittura con "b" poi con "c"

Definizione TM

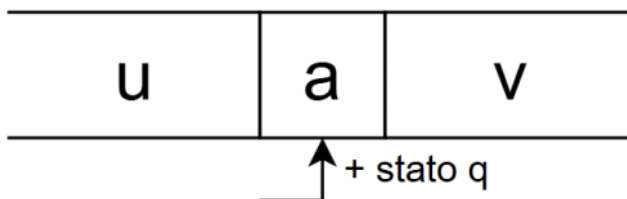
Una TM è una tupla $M = (Q, \Sigma, \Gamma, \delta, q_{start}, q_{acc}, q_{rej})$ dove:

- Q = insieme finito di **stati**
- Σ = insieme finito di **caratteri di input** ($_ \notin \Sigma$)
- Γ = insieme finito di **simboli di nastro** ($_ \in \Gamma, \Sigma \subseteq \Gamma$)
- $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ (L = left, R = right)
- $q_{start}, q_{acc}, q_{rej} \in Q$: stati **iniziali, accettazione e rifiuto** ($q_{acc} \neq q_{rej}$)

Considerare la configurazione: stato, contenuto nastro e posizione testina. Per convenzione si usa la notazione:

$u q a v$

con $u, v \in \Gamma^*, q \in Q, a \in \Gamma$



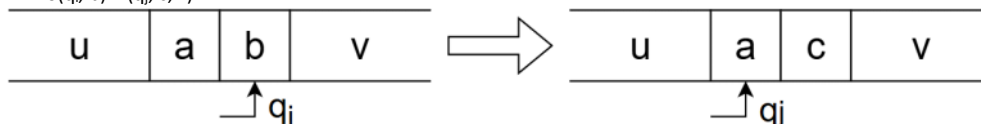
Che significa: sto nello **stato q** e la **testina legge "a"**. Dove a è un carattere del nastro, u è tutto quello che viene prima e v è tutto quello che viene dopo (lo stato q e il carattere a sono **due cose separate**).

Quando si scrive la configurazione, il carattere che stiamo leggendo è sempre dopo lo stato in cui ci troviamo (es in $u q a v$, il carattere letto "a" è dopo lo stato q)

- Configurazione iniziale:** $q_{start} w$, dove w è l'input
- Configurazione accettazione/rifiuto:** lo stato $q \in \{q_{acc}, q_{rej}\}$

Es:

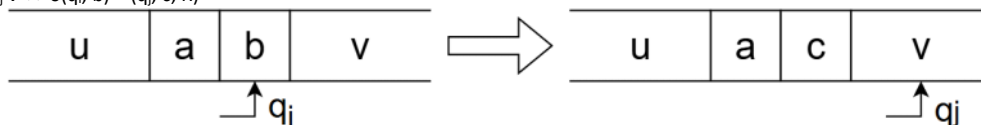
$u a q_i b v \vdash_M u q_j a c v \Leftrightarrow \delta(q_i, b) = (q_j, c, L)$



TM sta in q_i e la testina legge "b", va in q_j , sostituisce "b" con "c" e mi muovo a sinistra

Similmente

$u a q_i b v \vdash_M u a c q_j v \Leftrightarrow \delta(q_i, b) = (q_j, c, R)$



Lo stato è q_i e leggo "b", vado in q_j , rimpiazzo "b" con "c" e mi muovo a destra

Abbiamo che M accetta $w \in \Sigma^* \Leftrightarrow C_{start} = q_{start} w$ è t.c. $C_{start} \vdash_M^* C_{acc}$ (dove C_{acc} è una qualsiasi configurazione di accettazione)

Definizione Linguaggi Turing Riconoscibili

Un linguaggio è **Turing riconoscibili (Turing-Ric)** se esiste **TM M** che lo riconosce, ovvero $\forall w \in L$, abbiamo che M accetta w

Attenzione: se $w \notin L$, $M(w)$ può **rifiutare** o andare in **loop** (continua a transizionare all'infinito tra stati che non sono q_{acc} e q_{rej})
Cioè il riconoscitore è garantito solo a riconoscere i membri del linguaggio. Per chi non appartiene, **non garantisce che si fermi**.

Definizione di Decidibilità

L è **Turing decidibile (Turing-Dec)** se esiste TM M che lo **decide**

Il **decisore** non va mai in loop, ovvero:

- $w \in L \Rightarrow M$ accetta w
- $w \notin L \Rightarrow M$ rifiuta w

Cioè la TM si ferma sempre su ogni stringa, dando una risposta sì/no.

Decidibile: ferma sempre e accetta o rifiuta

Riconoscibile: si ferma solo se accetta ma può anche **non terminare** se la stringa non appartiene al linguaggio

Ogni linguaggio decidibile è anche riconoscibile, ma non viceversa

Varianti di TM

lunedì 10 novembre 2025 18:30

Ci convinciamo che le nostra definizione di TM ha alcune scelte arbitrarie, ma queste sono tutte irrilevanti. Es:

- Nastro infinito in entrambe le direzioni
- TM che muovono a R, L e S ("stay put")
- TM con k-nastri
- TM non deterministiche (può fare più scelte possibili invece di una sola)

Queste TM sono teoricamente più efficienti ma **tutte riconoscono gli stessi linguaggi** della TM normale e possono essere simulate con una TM ad 1 nastro. Vedremo quindi solo l'approccio **ad alto livello**, come nella progettazione di algoritmi

TM con Movimento "Stay Put"

Vediamo il caso della TM con aggiunta S ("stay put") come movimento:

$$\delta': Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, S\}$$

Una TM standard può simulare la TM M' definita sopra, muovendo prima a L e poi a R ogni volta che M' sta ferma.

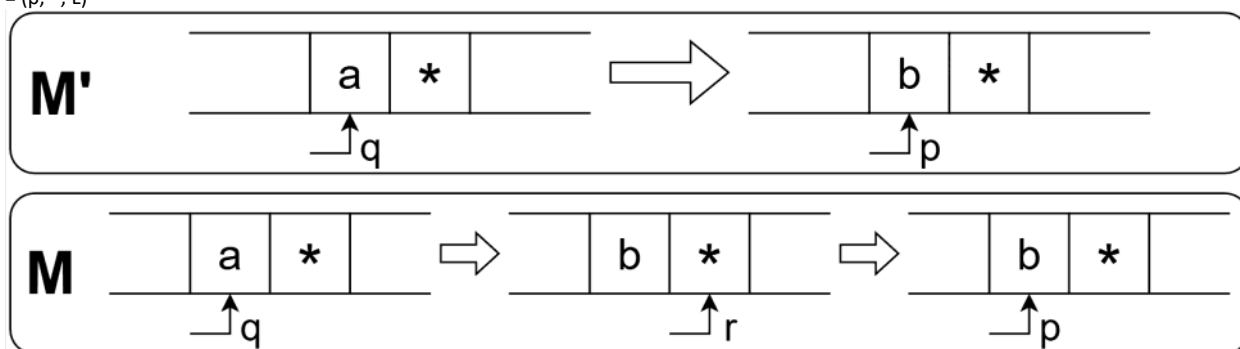
Più nel dettaglio:

$$\delta'(q, a) = (q, b, S)$$

Introduco un nuovo stato r e aggiungo

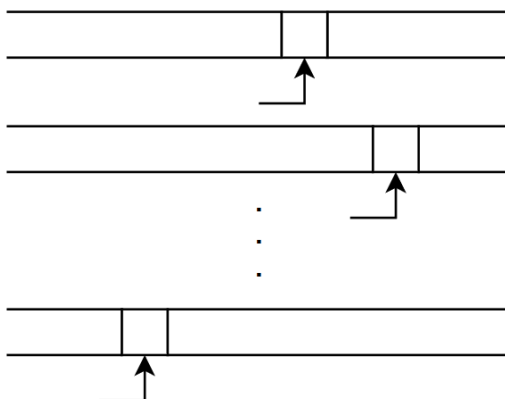
$$\delta(q, a) = (r, b, R)$$

$$\delta(r, *) = (p, *, L)$$



TM con Multinastro

$$\delta': Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, R, S\}^k$$



Teorema: per ogni TM M' multinastro ne esiste una M con singolo nastro equivalente

Correttezza: un linguaggio è **Turing-Dec/Ric** $\Leftrightarrow \exists$ TM multinastro che lo decide/riconosce

Idea: devo simulare sul singolo nastro di M tutti i nastri di M'

La TM M' utilizzerà alcuni **simboli speciali #** per separare i nastri di M'

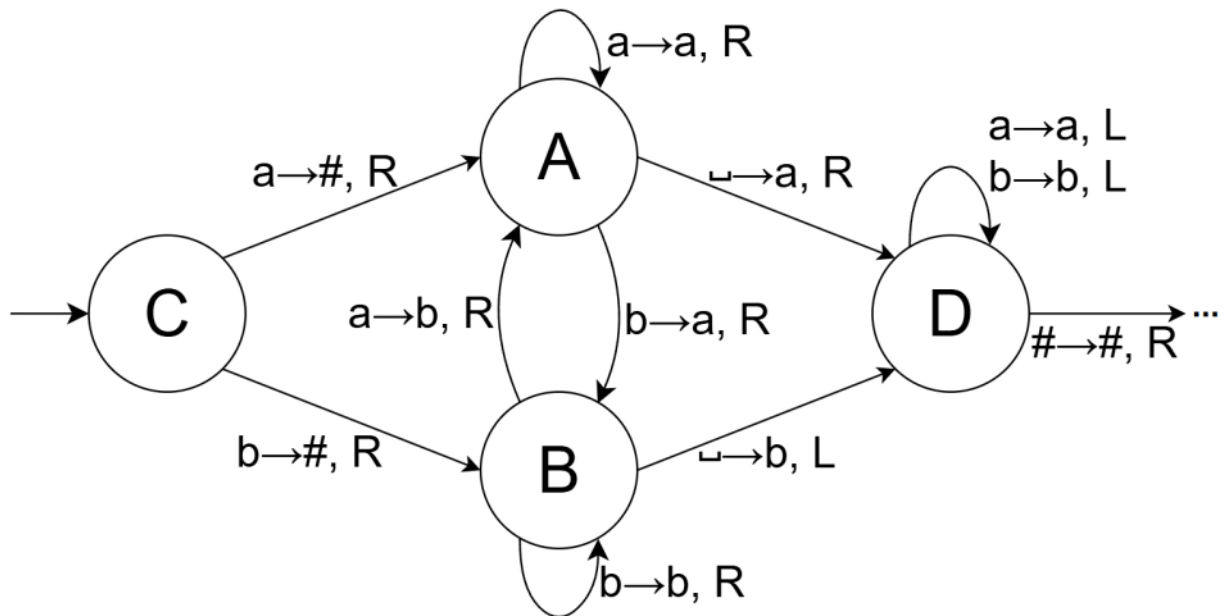
$\forall a \in \Gamma$, avremmo anche $\hat{a}$, cioè la sua **versione marcata** per tracciare le posizioni delle k testine di M'

Codice di M:

- All'inizio, M transiziona in questa configurazione
 $\# \hat{w}_1 \dots \hat{w}_n \# \hat{\sqcup} \# \dots \# \hat{\sqcup} \# \sqcup$
- Simulo una mossa di M':
 - Scorriamo il nastro per leggere i simboli marcati in ognuno dei k nastri
 - Torno indietro al primo #
 - Scansiono nuovamente andando ad aggiornare i simboli marcati in accordo alla δ'
 - Se M' costringe a muovere la testina emulata di uno dei nastri su #, M sosta tutto il nastro a R di una posizione e inserirà il simbolo $\hat{\sqcup}$

A livello implementativo, marcare i simboli è una semplice sostituzione. Per spostare il nastro o riconoscere l'inizio del nastro:

Sia $\Sigma = \{a, b\}$



TM Non Deterministica (NTM)

$$\delta': Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\})$$

Come per i PDA e i DFA. La NTM accetta se e solo se esiste un cammino di computazione accettante.

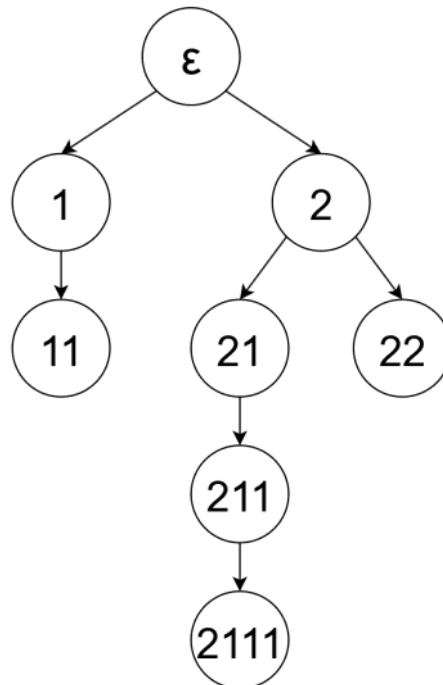
Gli altri cammini possono rifiutare o non fermarsi mai

Teorema: per ogni NTM N esiste una TM M equivalente

Correttezza: un linguaggio è **Turing-Dec/Ric** $\Leftrightarrow \exists$ NTM che lo decide/riconosce

Idea: la TM M prova tutte le **possibili scelte non-deterministiche** di N. Se ne esiste un insieme che accetta, M accetta

La nostra NTM avrà un albero di computazione:



(grado di non-determinazione 2: cioè ad ogni transizione può produrre **al massimo 2 possibili mosse**)

Wlog. Assumo M con 3 nastri:

- **1° nastro (input):** contiene $w = w_1 \dots w_n$, usato per rileggere i simboli di input ogni volta che simuliamo un nuovo cammino
- **2° nastro (di lavoro):** simula lo stato interno e il contenuto della NTM, tenendo traccia della computazione corrente
- **3° nastro (contatore):** contatore per ricordarmi l'indirizzo del nodo dell'albero in cui mi devo fermare nella simulazione di un cammino per N.

Es figura sopra: siccome la NTM ha grado di non-determ 2, codifichiamo le scelte con 1 e 2. Quindi il cammino "211" significa "1° scelta = 2, 2° = 1, 3° = 1"

Importante: esploro l'albero di N in **ampiezza**: cioè prima i nodi con indirizzo corrispondente a profondità 1, poi 2, poi 3, ...

Così non vado mai in loop e prima o poi se N accetta w , trovo un cammino che accetta e quindi appena lo trovo accetto in M

Altrimenti **incremento l'indirizzo** e ricomincio con la simulazione fino al prossimo nodo

Indecidibilità

mercoledì 12 novembre 2025 11:59

Problemi Decidibili

Mostreremo che come ci sono linguaggi **non regolari e non acontestuali**, ci sono anche linguaggi **non Turing-decidibili/riconoscibili**

Nel 1900 Hilbert pose il problema "processo in base al quale il problema può essere risolto in un numero finito di passi" per determinare se un polinomio ha radici intere

Es: $6x^3 + yz^2 + 3xy^2 - x^3 - 10$ Sol: $x = 5; y = 3, z = 0$

In termini di calcolabilità è come dire che:

$L = \{ \langle p \rangle : p \text{ polinomio con radici intere} \}$

è decidibile/riconoscibile. (nel 1970 è stato dimostrato che L è riconoscibile ma non decidibile)

Dove la notazione $\langle x \rangle$ indica una **qualsiasi codifica** di un polinomio (i suoi coefficienti)

Nell'ottica della tesi **Church-Turing** significa che esistono problemi che **nessun computer può mai risolvere** (nemmeno con infinite risorse)

Iniziamo con esempi di problemi decidibili:

Esempio 1

Stabilire se DFA D accetta una stringa. Il problema corrisponde al linguaggio:

$A_{DFA} = \{ \langle D, w \rangle : D \in DFA, D \text{ accetta } w \}$ (A sta per "accept")

La codifica $\langle D, w \rangle$ contiene tutte le informazioni necessarie per costruire sia la macchina che le stringhe.

In altri termini devo codificare $(Q, \Sigma, \delta, q_0, F)$ ma la **parte più importante di $\langle D, w \rangle$ è δ** poiché le altre componenti servono solo a rappresentare δ

$\delta(q, a) = p$

Es: una rappresentazione di δ è l'elenco di tutte le righe della tabella per δ separate da "#"

Una codifica può essere **valida** o meno se rispetta le regole per permettono di interpretarla come una coppia (D, w) , e nel caso in cui non sia valida, rifiuta.

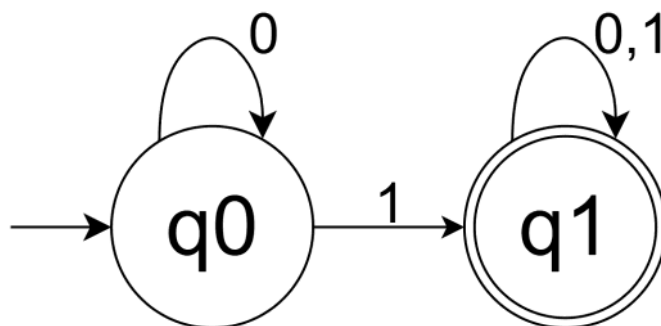
Vediamo come decidere A_{DFA} :

- Su input $\langle D, w \rangle$ controllo che sia una codifica valida di $D \in DFA$. Se non lo è, rifiuto
- Se è valida, **simulo un'esecuzione di D su w**
Wlog. Assumo la TM ha:
 - Un nastro contenente δ
 - Un nastro per **input w**
 - Un nastro per tracciare lo **stato**Tutti inizialmente vuoti, tranne per la codifica del DFA
- Se la simulazione **termina** su stato accettato, accetta. Altrimenti rifiuta

Es:

Supponiamo di avere un DFA $D = (Q, \Sigma, \delta, q_0, F)$ con:

- $Q = \{q_0, q_1\}$
- $\Sigma = \{0, 1\}$
- $F = \{q_1\}$
- Transizioni:
 - $\delta(q_0, 0) = q_0$
 - $\delta(q_0, 1) = q_1$
 - $\delta(q_1, 0) = q_1$
 - $\delta(q_1, 1) = q_1$



e vogliamo codificare D insieme a $w = 01$.

$\langle D, w \rangle$ si scrive descrivendo prima D e poi w dividendo i vari componenti con "#" (o altro):

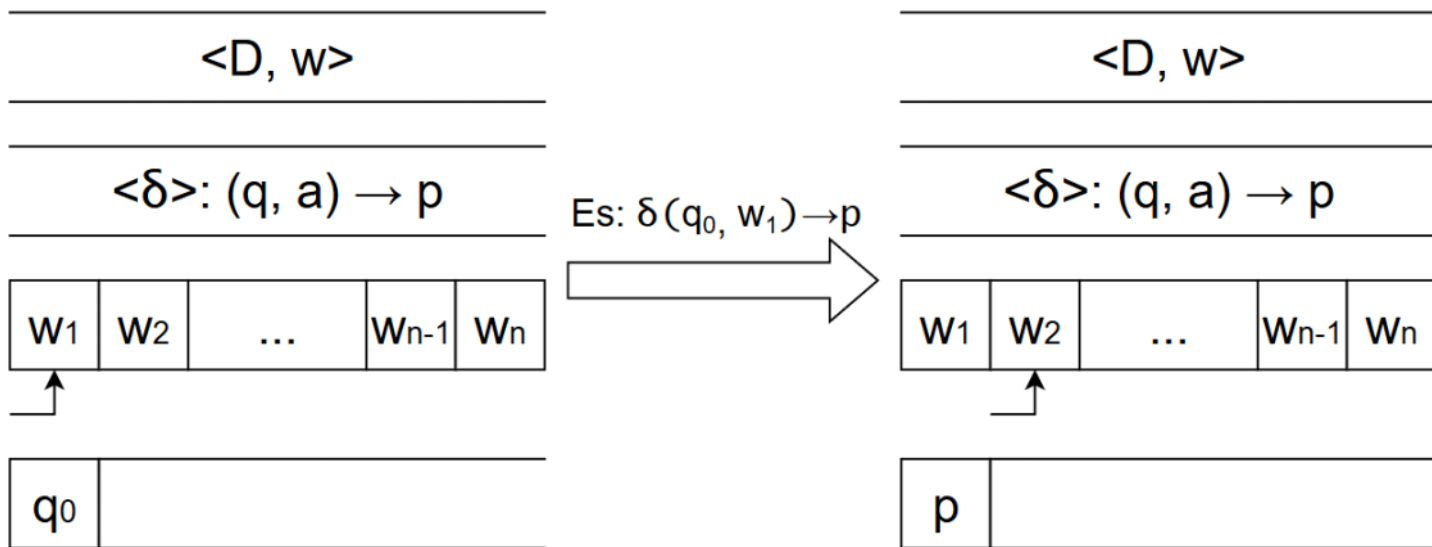
$\#Q\#\Sigma\#q_0\#F\#\delta\#w\#$

e $\langle \delta \rangle$ può essere in formato " $q\#a\#p$ " o anche " $q+a \rightarrow p$ " o anche " (q, a, p) " $\forall q \in Q, \forall a \in \Sigma$, e le varie transizioni possono essere divise da ","

Quindi con la TM a 4 nastri:

- 1° nastro:** $\#q_0,q_1\#0,1\#q_0\#q_1\#(q_0,0,q_0);(q_0,1,q_1);(q_1,0,q_1);(q_1,1,q_1)\#01\#$
- 2° nastro:** $(q_0,0,q_0);(q_0,1,q_1);(q_1,0,q_1);(q_1,1,q_1)$
- 3° nastro:** 01
- 4° nastro:** q_0

Altro esempio:



Sta in q_0 e legge w_1 e per la regola della δ sovrascrivo q_0 con p e passo al prossimo carattere

Supponiamo di avere un alfabeto $\Gamma = \{0, 1\}$ e ho $|Q|$ stati, per codificare $q \in Q$ necessito di $\log_2(|Q|)$ simboli. Poiche non possiamo scrivere direttamente il nome dello stato sul nastro a meno che non definiamo una TM diversa con Γ che contiene i nomi veri e propri degli stati, ma comunque è solo un diverso tipo di codifica dei nomi e non cambia la struttura della TM

Esempio 2

$A_{NFA} = \{ \langle N, w \rangle; N \in NFA, N \text{ accetta } w \}$

Abbiamo diversi modi per definire se questo linguaggio è decidibile, ad es:

- Generalizzo la definizione del DFA dato sopra per descrivere l'NFA
- Uso una NTM (TM Non Deterministica)
- ecc

Per decidere A_{NFA} con la generalizzazione del DFA dato sopra:

- Prima traduco $\langle N, w \rangle$ in $\langle D, w \rangle$ t.c. $D \in DFA, L(D) = L(N)$. Devo ridefinire la δ
- Dopo simulo la TM per A_{NFA}

Stessa cosa per:

$A_{REG} = \{ \langle R, w \rangle; R \text{ espressione regolare, } R \text{ genera } w \}$

A_{REG} è decidibile perche data la codifica dell'espressione regolare, basta tradurre $\langle R, w \rangle$ in $\langle D, w \rangle$ t.c. $L(D) = L(R)$ (poiche le espressioni regolari possono essere trasformate sempre in DFA)

Esempio 3

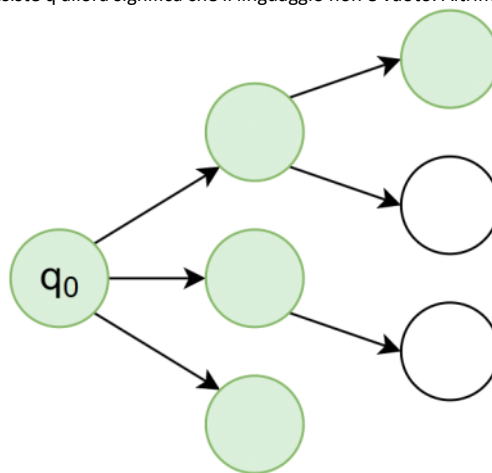
$E_{DFA} = \{ \langle D \rangle; D \in DFA, L(D) = \emptyset \}$ (la DFA non accetta nessuna stringa)

Problema: la TM non può provare tutti i $w \in \Sigma^*$

Tuttavia posso ridurre questo problema a quello di cercare un cammino $q_0 \rightarrow q$ t.c. $q \in F$ nel diagramma di stato per D .

Siccome la TM è un computer semplice posso applicare vari algoritmi, come ad esempio questo algoritmo di marcatura:

- Dato $\langle D \rangle$ rifiuto se non è una codifica valida
- Altrimenti ripeto questi passi fino a che non vengono marcati nuovi stati
 - Marco tutti gli stati che hanno transizione proveniente da stato marcato
- Se esiste $q \in F$ marcato allora rifiuto, poiche se esiste q allora significa che il linguaggio non è vuoto. Altrimenti accetto



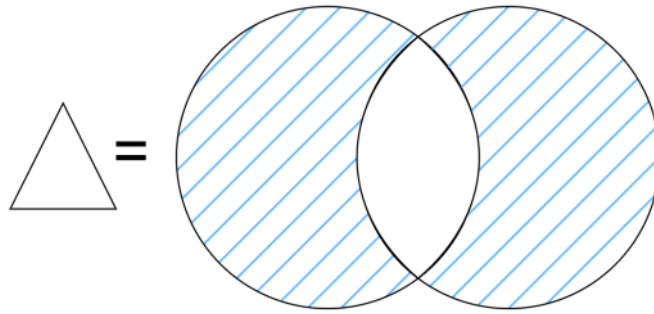
Esempio 4

$E_{QDFA} = \{ \langle A, B \rangle; A, B \in DFA, L(A) = L(B) \}$ (dati due DFA, mi dice se quelli hanno lo stesso linguaggio)

$L(A) = L(B) \Leftrightarrow L(A) \Delta L(B) = \emptyset$

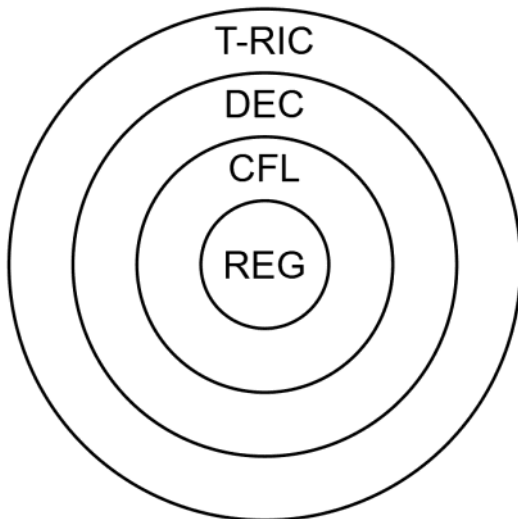
Dove $\Delta = (L(A) \cap L(B)^c) \cup (L(A)^c \cap L(B))$

Cioe solo le parti laterali, escludendo l'intersezione $(L(A) \cup L(B))$ dove le stringhe vengono accettate sia da A che da B



Per decidere EQ_{DFA} definisco C t.c. $L(C) = L(A) \Delta L(B)$
 Questo esiste sempre perché i DFA sono chiusi per unione e intersezione
 A questo punto uso la TM per decidere se $\langle C \rangle \in E_{DFA}$

Quindi ora abbiamo che



$REG \subset CFL \subset DEC \subset T-RIC$

Tecnica di Diagonalizzazione

Vediamo che esistono linguaggi che **non sono Turing-Dec/Ric** usando la tecnica della **diagonalizzazione**

Teorema: $A_{TM} = \{ \langle M, w \rangle : M \in TM \text{ e } M(w) = \text{accetta} \}$ **non è decidibile** (nessun algoritmo risolve questo linguaggio)

Prima di dimostrarlo, vediamo che A_{TM} è **turing-riconoscibile**. Infatti considero la seguente TM U:

- Su input $\langle M, w \rangle$ con $M \in TM$, $w \in \Sigma^*$
- Simula M su input w
- Se M accetta allora, allora U accetta l'input. Altrimenti rifiutiamo

Se M va in loop, anche U va in loop (riconoscibile). Se U potesse capire se M va in loop o meno (**halting problem**) allora sarebbe decisore ma vedremo che anche questo è indecidibile.

Una TM può prendere in input la descrizione di un'altra TM e simularlo su qualsiasi input, come facciamo per la DFA. Solo che la DFA non va in loop, mentre la TM si quindi per questo è solo riconoscibile

Bisogna chiarire come funziona U (**TM universale**, perché simula un'altra TM)

Es implementazione: Uso due nastri:

- **1° nastro:** salvo $\langle M, w \rangle$
- **2° nastro:** tengo traccia della configurazione $\langle a, q, b \rangle$ di M, dove q è lo stato, (a, b) è il nastro e la testina punta sul primo carattere di b

Esempio della codifica $\langle M, w \rangle$:

Sia $M = (\Sigma, \Gamma, Q, \delta, q_0, q_{acc}, q_{rej})$ e sia $m = |\Sigma|$ e $m = |\Gamma|$, $s = |Q| + 3$ (perché metto q_0, q_{acc}, q_{rej} nelle posizioni finali)

Denotiamo con **(i)** la **codifica binaria di i** e $\langle \delta \rangle$ la codifica di δ come sequenza di regole $\langle R \rangle$ separate da un marcatore " , "

$R = ((q, a), (r, b, Z))$ con $Z = \{L, R\}$

Quindi abbiamo le codifiche:

$\langle M, w \rangle = (n)_2, "(m)_2, "(s)_2, "<\delta>", "w$

$\langle \delta \rangle = \langle R_1 \rangle, "<R_2>", "...$

La simulazione sarà:

- **1° nastro:** contiene $\langle M, w \rangle$
- **2° nastro:** vuoto inizialmente e poi ci scrivo $\langle q_0, w \rangle$

Al passo t:

- **1° nastro:** contiene $\langle M, w \rangle$
- **2° nastro:** configurazione generica $\langle a, q, b_1, b_2, \dots, b_k \rangle$

Cerco sul nastro 1 una regola del tipo:

$\langle (q, b_1), (r, y, Z) \rangle$

e aggiorno il nastro 2 con la nuova configurazione

Ad ogni passo controllo se $r = q_{acc}$ e se lo è accetto, se $r = q_{rej}$ allora rifiuto, altrimenti va in loop

A questo punto dimostriamo che A_{TM} è **indecidibile**

Dimostrazione: per contraddizione supponiamo che H sia una TM che decide A_{TM} , ovvero $H(\langle M, w \rangle)$:

- Accetta se M accetta w
- Rifiuta se M rifiuta w

Costruisco una nuova TM D che usa H, quindi fa:

- Su input $\langle M \rangle$ esegue H su input $\langle M, \langle M \rangle \rangle$ ($\langle M \rangle = w$ = codifica della macchina di turing)
 - Se H accetta, D rifiuta
 - Altrimenti se H rifiuta, D accetta

Considerando tutte le possibili macchine di Turing, troveremmo anche D e considerando tutte le codifiche delle possibili TM, avremmo anche la codifica di D

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...	D
M_1	ACC					
M_2		ACC				
M_3			REJ			
M_4				REJ		
...					...	
D	REJ	REJ	ACC	ACC		???

Se D prende su input M_1 , siccome M_1 accetta la codifica $\langle M_1 \rangle$ e siccome D fa il contrario della macchina che prende in input, allora D rifiuta $\langle M_1 \rangle$

Da una parte $D(\langle M \rangle)$:

- Accetta $\Leftrightarrow$ H rifiuta $\langle M, \langle M \rangle \rangle$
- Accetta $\Leftrightarrow$ H rifiuta $\langle M \rangle$
- Rifiuta $\Leftrightarrow$ H accetta $\langle M, \langle M \rangle \rangle$
- Rifiuta $\Leftrightarrow$ H accetta $\langle M \rangle$

Quando $M = D$, abbiamo una contraddizione, poiche $D(\langle D \rangle)$:

- Accetta $\Leftrightarrow$ D rifiuta $\langle D \rangle$
- Rifiuta $\Leftrightarrow$ D accetta $\langle D \rangle$

D è un **decisore che contraddice se stesso**. L'unico modo per uscire dalla contraddizione è se **H non esiste**, ovvero **A_{TM} non è decidibile**

Esercizio: Dimostrare che $A_{CFG} = \{ \langle G, w \rangle : G \in CFG, w \in L(G) \}$ è decidibile

Esercizio: Dimostrare che $E_{CFG} = \{ \langle G \rangle : G \in CFG, L(G) = \emptyset \}$ è decidibile

Riduzione

martedì 18 novembre 2025 16:55

Abbiamo mostrato che questo linguaggio **non è decidibile**:

$$A_{TM} = \{ \langle M, w \rangle : M \in TM, M(w) = ACC \}$$

Vedremo che esistono anche

Teorema: L è **decidibile** $\Leftrightarrow L$ è **Turing-Ric** e **CoTuring-Ric** (L è CoTuring-Ric se L^C è Turing-Ric)

Cioè L è decidibile $\Leftrightarrow L$ e L^C sono Turing-Ric

Corollario: Se L **non è decidibile**, allora almeno uno tra L e L^C **non è Turing-Ric**

Corollario: A_{TM}^C **non è Turing-Ric**

Dimostrazione:

$\Rightarrow$ Se L è **decidibile**, allora anche L^C è **decidibile** (perché posso fare una TM che

Quindi anche L e L^C sono **Turing-Ric**

$\Leftarrow$ Date $M_1, M_2 \in TM$ t.c. M_1 **riconosce** L e M_2 **riconosce** L^C , costruisco $M \in TM$ che accetta quando l'altra rifiuta e rifiuta quando l'altra accetta)

Idea: M simula i **singoli passi** di esecuzione di M_1 e M_2 in **parallelo**, ad es. usando 2 nastri

- Se M_1 **accetta** ($x \in L$) allora M **accetta**
- Se M_2 **accetta** ($x \notin L$) allora M **rifiuta**

$\forall x: x \in L \vee x \in L^C$

Ovvero $M(x) = ACC \Leftrightarrow x \in L$ e M è **decisore**

Tecnica di Riduzione

La **tecnica della riduzione** ci serve per dimostrare che esistono linguaggi che **non sono né decidibili né riconoscibili**

Per capire la tecnica bisogna estendere la nostra definizione di TM aggiungendo un **nastro di output** (per scrivere gli output degli algoritmi) (Wlog)

Diremo che una TM **calcola** una funz. su un input se inizia con input w sul nastro di **input** e termina con $f(w)$ sul nastro di **output**

Definizione: $f: \Sigma^* \rightarrow \Sigma^*$ è **calcolabile** se $\exists TM$ che **calcola** $f \forall w \in \Sigma^*$

Es: trasformazioni di codifiche che abbiamo usato nella scorsa lezione

Definizione (Mapping Reductions):

A è **riducibile tramite funzione** a B ($A \leq_m B$), con $A, B \in$ linguaggi

se $\exists f: \Sigma^* \rightarrow \Sigma^*$ calcolabile t.c. $\forall w \in \Sigma^*$:

$$w \in A \Leftrightarrow f(w) \in B$$

$$(w \in A \Rightarrow f(w) \in B)$$

$$(w \notin A \Rightarrow f(w) \notin B)$$

La funzione è definita su tutto l'alfabeto (istanze sì e istanze no, quindi w o sta in A o non ci sta)

Teorema: Se $A \leq_m B$ e B **decidibile**, allora A **decidibile**

Se B è decidibile, calcolo la mapping reduction per A , creando una TM che dato input accetta se e solo se input è in A

Corollario: Se $A \leq_m B$ e A è **indecidibile** (non esiste TM che lo decida), allora B è **indecidibile** (è la negazione del teorema)

Dimostrazione (Cor): Se B fosse **decidibile** allora A **sarebbe decidibile** per il teorema

Dimostrazione (Teo): Sia M la TM che decide B e sia F la TM che calcola la **mapping reduction** f da A a B

Decisore M' per A :

- Su input w calcola $f(w)$ usando F
- Poi lancia M su $f(w)$ e **accetta $\Leftrightarrow M$ accetta**

M' è **decisore** perché M lo è e F termina sempre (non va in loop). Inoltre:

$$M'(w) = ACC \Leftrightarrow M(f(w)) = ACC$$

$$\Leftrightarrow f(w) \in B \text{ (per definizione di Mapping Reduction)}$$

$$\Leftrightarrow w \in A$$

Applicazioni

Esempio 1:

$HALT_{TM} = \{ \langle M, w \rangle : M \in TM, M(w) \neq LOOP \}$ (M non va in loop) è **indecidibile**

Dimostrazione: dimostro esibendo una **mapping reduction** $A_{TM} \leq_m HALT_{TM}$

Esibisco una $f: \Sigma^* \rightarrow \Sigma^*$ **calcolabile** t.c. per ogni input

$$\langle M, w \rangle \in A_{TM} \Leftrightarrow f(\langle M, w \rangle) \in HALT_{TM}$$

Definisco la seguente f :

- Su input $\langle M, w \rangle$
- Definisce un'altra TM M' :
 - Su input $x \in \Sigma^*$
 - Esegue $M(x)$
 - Se $M(x)$ accetta $\Rightarrow M'(x)$ accetta e termina
 - Altrimenti, **va in loop** (es: muove indefinitivamente a DX)
- **Output:** $\langle M', w \rangle$

La funzione f è **calcolabile** ($\exists$ TM F che la calcola per ogni input)

Correttezza:

$\Rightarrow$ Sia $\langle M, w \rangle \in A_{TM}$

Allora $\langle M', w \rangle$ termina perché $M'(w) = M(w) = \text{ACC}$.

Ovvero $\langle M', w \rangle \in \text{HALT}_{TM}$

$\Leftarrow$ Sia $\langle M, w \rangle \notin A_{TM}$

Allora $M(w) = \text{REJ}$ o LOOP . In questo caso abbiamo che $M'(w) = \text{LOOP}$ (per definizione di f)

Ovvero $\langle M', w \rangle \notin \text{HALT}_{TM}$

Esempio 2:

$E_{TM} = \{\langle M \rangle : M \in TM, L(M) = \emptyset\}$ (linguaggio vuoto) è **indecidibile**

In questo caso è utile fare la mapping reduction per il **complemento** E^c_{TM} (il linguaggio non è vuoto) per avere $A_{TM} \leq_m E^c_{TM}$

Definisco $f: \Sigma^* \rightarrow \Sigma^*$ t.c.

$\langle M, w \rangle \in A_{TM} \Leftrightarrow f(\langle M, w \rangle) \notin E^c_{TM}$

La funzione f :

- Dato input $\langle M, w \rangle$
- Output: $\langle M', w \rangle$ t.c. M' su input x :
 - Se $x \neq w$ **rifiuta**
 - Se $x = w$, **simula M su w e accetta $\Leftrightarrow M$ accetta**

Cioè se x è diverso da w rifiuta. Altrimenti accetta solo se M accetta w

$\Rightarrow \langle M, w \rangle \in A_{TM}$, guardiamo M' :

- Se $x \neq w$, rifiuta x
- Se $x = w$, lancia $M(w)$; ovvero M' accetta w

Quindi M' accetta un input e quindi $L(M') \neq \emptyset$ e quindi $\langle M' \rangle \in E^c_{TM}$

$\Leftarrow \langle M, w \rangle \notin A_{TM}$, guardiamo M' :

- Se $x \neq w$, rifiuta
- Se $x = w$, $M(w)$ rifiuta o loop, quindi M' rifiuta

Ovvero $\langle M' \rangle \notin E^c_{TM}$ perché $L(M') = \emptyset$

Esempio 3:

$\text{REG}_{TM} = \{\langle M \rangle : M \in TM, L(M) \in \text{REG}\}$ (il linguaggio della TM è regolare) è **indecidibile**

Eseguo mapping reduction $A_{TM} \leq_m \text{REG}_{TM}$

La funzione f :

- Dato input $\langle M, w \rangle$
- Output $\langle M' \rangle$ t.c. M' su input x :
 - Se $x = 0^n 1^n$ accetta
 - Se $x \neq 0^n 1^n$, simula $M(w)$ e accetta $x \Leftrightarrow M(w) = \text{ACC}$ (perché se w fosse stato un altro linguaggio irregolare allora $M(w) = \text{REJ}$)

$\Rightarrow$ Se $\langle M, w \rangle \in A_{TM}$, allora $M(w) = \text{ACC}$, quindi $L(M') = \Sigma^*$

Ovvero $\langle M' \rangle \in \text{REG}_{TM}$

$\Leftarrow$ Se $\langle M, w \rangle \notin A_{TM}$, allora $M(w) = \text{REJ}/\text{LOOP}$, quindi $L(M') = \{0^n 1^n : n \geq 0\}$ (saranno solo irregolari)

Ovvero $\langle M' \rangle \notin \text{REG}_{TM}$

Esempio 4:

$\text{EQ}_{TM} = \{\langle M_1, M_2 \rangle : M_1, M_2 \in TM, L(M_1) = L(M_2)\}$ è **indecidibile**

Per riduzione a $E_{TM} = \{\langle M \rangle : M \in TM, L(M) = \emptyset\}$

Mostriamo che $E_{TM} \leq_m \text{EQ}_{TM}$. Devo costruire $f: \Sigma^* \rightarrow \Sigma^*$ t.c.:

$\langle M \rangle \in E_{TM} \Leftrightarrow f(\langle M \rangle) \in \text{EQ}_{TM}$

Definiamo la TM F che computa f :

- Input $\langle M \rangle$
- Costruisce $\langle M' \rangle$ t.c. su input x M' **rifiuta sempre** (quindi $L(M') = \emptyset$)
- Output $\langle M, M' \rangle$

$\Rightarrow$ Se $M \in E_{TM}$, allora $L(M) = \emptyset$

Per costruzione, $L(M') = \emptyset$, quindi siccome $L(M) = L(M') = \emptyset$ allora $\langle M, M' \rangle$ e quindi $\langle M, M' \rangle \in \text{EQ}_{TM}$

$\Leftarrow$ Se $\langle M, M' \rangle \in \text{EQ}_{TM}$, allora $L(M) = L(M') = \emptyset$. Ovvero $\langle M \rangle \in E_{TM}$

Riduzione per Turing-Riconoscibilità

Vediamo come applicare le stesse tecniche alla **Turing-Riconoscibilità**

Teorema: Se $A \leq_m B$ e B è **turing-ric** allora A è **turing-ric**

Corollario: Se A **non è turing-ric** (irriconoscibile) allora B **non è turing-ric**

Dimostrazione: identica al caso di decidibilità

Applicazione: posso dimostrare la non turing-ric per **riduzione** ad altri linguaggi **non turing-ric** (es A_{TM}^c)

Siccome A_{TM}^C non è facile da maneggiare, possiamo ragionare sul **complemento**

LEMMA: Se $A \leq_m B$, allora $A^C \leq_m B^C$

Dimostrazione: Basta considerare la **stessa f mapping reduction** da A a B (le istanze sì di A saranno le istanze no di A^C)

Pertanto se A^C **non è turing-ric** e $A \leq_m B$ allora B^C **non è turing-ric**. Quindi basta avere A^C non turing-ric e fare la mapping reduction con $A \leq_m B$

Esempio:

EQ_{TM} **non è turing-ric**

Pongo $A^C = A_{TM}^C$ ($A = A_{TM}$) e $B^C = EQ_{TM}$ ($B = EQ_{TM}^C$). Pertanto basta mostrare che $A_{TM} \leq_m EQ_{TM}^C$

Definisco $f: \Sigma^* \rightarrow \Sigma^*$ computabile t.c.

$$\langle M, w \rangle \in A_{TM} \Leftrightarrow f(\langle M, w \rangle) \in EQ_{TM}^C$$

TM F per calcolare f:

- Su input $\langle M, w \rangle$
- Output $\langle M_1, M_2 \rangle$ t.c. M_1 **rifiuta tutto** ($L(M_1) = \emptyset$) e M_2 su input x esegue M su w e accetta x $\Leftrightarrow$ M accetta w

$\Rightarrow$ Se $\langle M, w \rangle \in A_{TM}$, allora $M(w) = ACC$, quindi M_2 accetta ogni x (M accetta sempre)

Quindi $L(M_2) = \Sigma^* \neq L(M_1) = \emptyset$, quindi

$$\langle M_1, M_2 \rangle \in EQ_{TM}^C$$

$\Leftarrow$ Se $\langle M_1, M_2 \rangle \in EQ_{TM}^C$ allora $L(M_1) \neq L(M_2)$

Siccome M_1 rifiuta tutto allora M_2 **deve accettare almeno un input** (così il suo linguaggio non è vuoto).

Ma questo è possibile solo se $M(w) = ACC$ (poiché così $L(M_2) = \Sigma^*$). Ovvero

$$\langle M, w \rangle \in A_{TM}$$

Incompletezza di Goedel

mercoledì 26 novembre 2025 08:19

Teoremi di Incompletezza di Goedel

Si può dimostrare usando solamente la teoria della calcolabilità

Nasce dall'esigenza di formalizzare la **Good Old Real Math (GORM)**

Euclide ha iniziato il processo di formalizzazione delle correttezza delle prove matematiche come sequenza di derivazione logiche a partire da **assiomi**:

- 1) Congiungendo 2 punti qualsiasi si ottiene un segmento
- 2) Per un punto esterno a una retta passa una sola retta parallela
- 3) ...

I primi approcci formali di formalizzare la GORM risalgono al 1800 tramite la **logica di primo ordine** e il **calcolo deduttivo**

Si possono esprimere enunciati del tipo: "Alice ha il papà più figo di tutti"

$\forall x (\neg(x = a) \rightarrow \text{IsCooler}(\text{Father}(a), \text{Father}(x)))$

Connettori: $\wedge, \vee, \neg, \rightarrow$

Quantificatori: $\exists, \forall, =$

Funzioni: $\text{Father}(\cdot)$

Relazioni: $\text{IsCooler}(\cdot, \cdot)$

Questo permette di rappresentare gli enunciati matematici e le prove come stringhe, che permettono di verificare se l'enunciato segue dagli assiomi

Esempi: assiomi di Trotsky per la geometria piana. Aritmetica di Peano su $\mathbb{N}$. Gli assiomi di Zermelo-Fraenkel (ZFC)

Aritmetica di Peano:

- **Costante:** 0
- **Funzioni:** $\text{Successor}(x)$, $\text{Plus}(x, y)$, $\text{Times}(x, y)$ (prodotto)
- **Assiomi:**
 - $\forall x: \neg(\text{Successor}(x) = 0)$
 - $\forall x, y: (\text{Successor}(x) = \text{Successor}(y)) \Rightarrow (x = y)$
 - $\forall x: \text{Plus}(x, 0) = x$
 - $\forall x: \text{Times}(x, 0) = 0$
 - ...

Qualsiasi enunciato su $\mathbb{N}$ lo posso **dimostrare** solo usando le regole della logica

Fatto: alcuni di questi sistemi assiomatici (tipo Peano e ZFC) sono abbastanza espressivi che si possono formalizzare enunciati tipo "M(w) Termina" dove $M \in \text{TM}$
A questo punto proviamo a formalizzare la nostra branca preferita della matematica.

Introduciamo un sistema di prova π t.c.:

- Per ogni **affermazione Vera o Falsa** esiste una sua rappresentazione come **stringa <x>** di lunghezza finita
- Per ogni **dimostrazione** esiste una rappresentazione come stringa $\langle \pi \rangle$ di lunghezza finita
- $\exists \text{ TM } V \text{ t.c.}$

$V(\langle x, \pi \rangle) = \text{ACC} \Leftrightarrow \pi$ è una prova valida per x

Diciamo: x **dimostrabile** se $\exists \pi$ t.c.

$V(\langle x, \pi \rangle) = \text{ACC}$

π deve essere **computabile**: gli assiomi sono **riconoscibili** (definire se è sono assiomi o meno)

Più precisamente:

$\{A : A \text{ è assioma in } \pi\}$ è **decidibile**

Dato il verificatore V , posso definire il **dimostratore** P : (se esiste un verificatore esiste un dimostratore)

TM $P(\langle x \rangle)$:

- 1) Per $k = 1, 2, 3, \dots$
- 2) Per ogni stringa w di lunghezza k (w può essere la codifica di π)
- 3) Se $V(\langle x, w \rangle) = \text{ACC}$, ritorna w

(brute force su V che controlla ogni stringa di lunghezza finita)

Posso anche considerare:

$L_{\text{PROVABLE}} = \{\langle x \rangle : x \text{ è dimostrabile in } \pi\}$

La seguente TM prova a decidere L_{PROVABLE} :

TM $R(\langle x \rangle)$:

- 1) Per $k = 1, 2, 3, \dots$
- 2) Per ogni stringa w di lunghezza k
- 3) Se $V(\langle x, w \rangle) = \text{ACC}$, ritorna **ACC**
Se $V(\langle \neg x, w \rangle) = \text{ACC}$, ritorna **REJ**

Ritorna il linguaggio di tutti gli x che sono dimostrabili (gli x veri sono dimostrabili e gli x falsi non lo sono)

A seconda delle proprietà di π , R potrebbe essere o meno un **decisore**.

Definizione: Sia π un sistema di prova. Diremo che:

- π è **consistente**, se $\forall x$ al più uno tra x e $\neg x$ è **dimostrabile**
($\forall x, x \text{ dimostrabile} \Rightarrow \neg x \text{ non dimostrabile e viceversa}$) (non possiamo dimostrare x e il suo contrario)
- π è **valido**, se $\forall x$, se x è **dimostrabile** allora x è **Vera**
($x \text{ dimostrabile} \Rightarrow x \text{ vera}$)
- π è **completo**, se $\forall x$ almeno uno tra x e $\neg x$ è **dimostrabile**
($\forall x, x \text{ dimostrabile} \vee \neg x \text{ dimostrabile}$) (per ogni x è possibile dimostrare o x o il suo contrario, altrimenti se x e $\neg x$ non sono dimostrabili, π non è completo)
Se è **incompleto**, $\exists x$ detta **affermazione indipendente**, cioè **esiste un enunciato che non puoi dimostrare né x né $\neg x$** (ne confutare, ne dimostrare)

Osservazioni:

- Se progetto π , **meglio** che sia **consistente**, altrimenti ogni THM è dimostrabile (tutto è banale).
Esempio:
THM: Bla Bla Bla

Dim: Supponiamo non sia vero

- Assiomi $\Rightarrow x$
- Assiomi $\Rightarrow \neg x$

Contraddizione tra x e $\neg x$

- Se π è **valido** è anche **consistente**. Perché:

x **dim** $\Rightarrow x$ **Vero**

Poiché se $\neg x$ **dim** $\Rightarrow \neg x$ **Vero** $\Rightarrow x$ **Falso** $\rightarrow$ **contraddizione**

- Se π è **consistente e completo**, $\forall x$ solo uno tra x e $\neg x$ è **dimostrabile**
- Se π è **valido e completo**, $\forall x$ solo uno tra x e $\neg x$ è **dimostrabile** e quindi **Vero**
 $\Rightarrow$ **Tutto solo ciò che è Vero è dimostrabile**

Esistono sistemi che sono validi e completi (es assiomi di Tarski) ma il **teorema di incompletezza di Godel e Turing** ci dimostrano che è impossibile formalizzare tutta la matematica

Torniamo alla TM $R(\langle x \rangle)$, che $\forall$ stringa w di lung. k ($k \in [1, 2, 3, \dots]$):

- **ritorna ACC** se $V(\langle x, w \rangle) = \text{ACC}$
- **ritorna REJ** se $V(\langle \neg x, w \rangle) = \text{ACC}$

Possiamo dire che:

- Se π è **valido**, quando R **termina** e fornisce la **risposta corretta** (non è detto che termini)
- Se π è **completo**, R è un **decisore**. Infatti se $R(\langle x \rangle) = \infty$ (**loop**) allora né x né $\neg x$ sono **dimostrabili**
- Se π è **valido e completo** allora R è un **decisore che dice sempre la verità**

Cioè t.c. $\forall x$:

- Se x è **Vera**, $R(\langle x \rangle) = \text{ACC}$
- Se x è **Falsa**, $R(\langle x \rangle) = \text{REJ}$

Supponiamo che π sia **valido e completo** (se π dimostra x , allora x è vera).

Costruisco TM per decidere:

$\text{HALT}_{\text{TM}} = \{ \langle M, w \rangle : M \in \text{TM}, M(w) \neq \infty (\text{loop}) \}$

TM $D_{\text{HALT}}(\langle M, w \rangle)$:

- Ritorna $R(\langle "M(w) \neq \infty" \rangle)$ ($\langle "M(w) \neq \infty" \rangle$ = codifica dell'informazione di $M(w)$ non va in loop)

Unica assunzione: π deve poter esprimere " $M(w) \neq \infty$ " (deve capire se M non va in loop)

Se π è valido e completo, D_{HALT} **decide** $\text{HALT}_{\text{TM}} \rightarrow$ **contraddizione** (siccome R è un decisore che dice sempre la verità)

Teorema: ZFC (Peano) non può essere valido e completo

Conseguenza: se ZFC è **valido**, allora $\exists x$ che pur essendo **Vera non può essere dimostrata (affermazione indipendente)** $\Rightarrow$ **ZFC è incompleto**

Tentativo: siccome x è **Vera** la aggiungo agli assiomi ma il sistema ottenuto soddisfa ancora le assunzioni del thm di incompletezza e quindi esiste un'altra x che non è dimostrabile ma è Vera

Ma poi il sistema di assiomi **non è computabile**

Secondo Teorema di Incompletezza

Come può essere **x Vera se non è dimostrabile?** Il concetto di verità non è influenzato dalla logica di π

Teorema: ZFC non può essere consistente e completo (non parla del concetto di verità)

Dimostrazione: Il punto di partenza è che ci sono affermazioni così semplici che sono certamente vere

Es: ci sono 25 num. primi < 100 (puoi controllare perché puoi provarli tutti)

Lemma: se una TM ha una traccia di esecuzione per $t \geq 1$ passi, allora c'è una prova di questa affermazione in ZFC

Basta controllare quando M è nella **configurazione iniziale**, ..., dopo t passi M è nella configurazione t (banale)

Quindi se $M \in \text{TM}$ e $M(w) \neq \infty$, allora c'è una prova di " $M(w)$ termina" in ZFC (poiché in t passi accetterà/rifiuterà w)

Consideriamo la seguente TM $D(\langle M \rangle)$:

- Per $k = 1, 2, 3, \dots$
- Per ogni stringa w di lung. k
- (prova a trovare una dimostrazione per l'affermazione " $M(w)$ termina")
- Se w è una dimostrazione per " $M(\langle M \rangle) \neq \infty$ " entra nello stato "muovi sempre a DX" (se M non va in loop $\rightarrow D$ va in loop)
- Se w è una dimostrazione per " $M(\langle M \rangle) = \infty$ " allora **termina** (se M va in loop $\rightarrow D$ termina)

Che succede se $D(\langle D \rangle)$?

Assumiamo ZFC **consistente**:

al più una tra " $D(\langle D \rangle)$ termina" oppure " $D(\langle D \rangle)$ loop" è **dimostrabile**

In realtà però con la **tecnica della diagonalizzazione** vediamo che " $D(\langle D \rangle)$ va in loop" e " $D(\langle D \rangle)$ termina" **non sono dimostrabili**:

- Supponiamo che " $D(\langle D \rangle)$ va in **loop**" sia **dimostrabile**.
Allora quando lancio $D(\langle D \rangle)$, troverò un w che è la dimostrazione per " $D(\langle D \rangle)$ termina"
Pertanto per il lemma esiste una prova in ZFC per " $D(\langle D \rangle)$ **termina**"
Questo **contraddice la consistenza**
- Supponiamo che " $D(\langle D \rangle)$ **termina**" sia **dimostrabile**.
Quando lancio $D(\langle D \rangle)$, troverò un w che entra nello stato "muovi sempre a DX"
Per il lemma esiste una prova in ZFC per " $D(\langle D \rangle)$ va in **loop**"
Questo **contraddice di nuovo la consistenza**

Pertanto **ZFC è incompleto**

Bonus

Ma che fa $D(\langle D \rangle)$? Termina o Loop?

Siccome $D(\langle D \rangle)$ è indipendente (non puoi dimostrare che termina o va in loop) allora $D(\langle D \rangle)$ non può trovare **nessuna delle due $\Rightarrow$ va in loop**

Se codifico questo in ZFC ho una dimostrazione per l'affermazione " $D(\langle D \rangle)$ **loop**"

In realtà **non c'è nessuna contraddizione** perché quello che abbiamo dimostrato in precedenza è vero solo se ZFC è **consistente**

Quindi abbiamo dimostrato:

"Se ZFC è **consistente**, allora $D(\langle D \rangle)$ **va in loop**"

Ovvero una affermazione del tipo " $A \rightarrow B$ "

Se assumiamo che ZFC sia consistente abbiamo visto che B non è dimostrabile in ZFC

Cioè ZFC non può dimostrare la sua consistenza (cioè non possiamo dimostrare in ZFC l'affermazione "ZFC è consistente")

Teorema: Sia ZFC **consistente**

Allora ZFC è **incompleto** e l'affermazione "ZFC è consistente" **non è dimostrabile**

Esempi e Esercizi

mercoledì 26 novembre 2025 18:22

Esempio 1

Mostriamo che

$A_{CFG} = \{ \langle G, w \rangle : G \in CFG \text{ e } w \in L(G) \}$ è **decidibile**

Siccome ci possono essere **infinite derivazioni** usiamo una certa proprietà:

Se G è un **CNF (forma normale di Chomsky)**, dove ad ogni passo:

- O sostituiamo una variabile con due variabili ($A \rightarrow BC$)
- O sostituiamo una variabile con un terminale ($A \rightarrow a$)

Siccome ogni nodo produce solo due figli (regola $A \rightarrow BC$), l'albero è **binario**

In un albero **binario pieno** (ogni nodo interno ha 2 figli) abbiamo n foglie (terminali) e $n - 1$ nodi interni (dove applichiamo $A \rightarrow BC$)

Quindi facciamo $n - 1$ espansioni $A \rightarrow BC$ e n produzioni terminali $A \rightarrow a$, con un totale di:

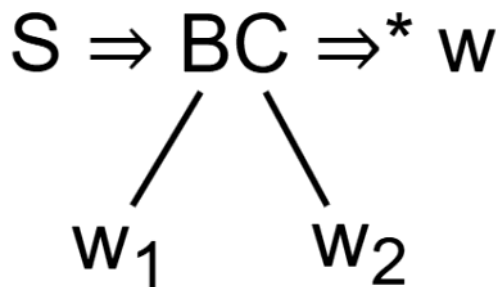
$$(n - 1) + n = 2n - 1 \text{ passi}$$

Allora $\forall w \in L(G)$ t.c. $|w| = n$ sono richiesti **$2n - 1$ passi** per ogni sua derivazione

Dimostriamo per induzione:

- **Caso Base:** se $n = 1$, allora c'è la regola $S \rightarrow w$, con $w \in \Sigma$ e infatti $2 \cdot 1 - 1 = 1$
- Se $n \geq 2$, assumo vero per lunghezze $< n$ e sia $w \in L(G)$ lunga n

Sicuramente:



t.c. $|w_1| = i < n$ e $|w_2| = n - i < n$

Per ipotesi induttiva

$$\# \text{ passi per } w = 1 + 2i - 1 + 2(n - i) - 1 = 2i + 2n - 2i - 1 = 2n - 1$$

Una TM prova tutte le derivazioni di lunghezza $2n - 1$. Quindi A_{CFG} è **decidibile**

Esempio 2

$E_{CFG} = \{ \langle G \rangle : G \in CFG, L(G) = \emptyset \}$ è **decidibile**

Non posso provare a generare tutte le stringhe perché possono essere infinite. Usiamo invece l'**algoritmo di marcatura** dei grafi.

- Su input $\langle G \rangle$, marca tutti i terminali
- Ripeti fino a che abbiamo finito le variabili da marcare:
Marca le variabili A t.c. G usa la regola $A \rightarrow U_1 \dots U_k$ e $U_1 \dots U_k$ sono stati già marchati
- Accetta $\Leftrightarrow S$ non è marcata

Esempio 3

$L = \{ \langle M \rangle : M \in TM \text{ t.c. } M \text{ accetta tutte e solo le stringhe di lunghezza dispari in } \{0, 1\}^* \}$ **non è decidibile**

Per **riduzione**. $A_{TM} \leq_m L$. Ovvero definisco $f: \Sigma^* \rightarrow \Sigma^*$ t.c.

per ogni input $\langle M, w \rangle \in A_{TM} \Leftrightarrow f(\langle M, w \rangle) \in L$

f calcolabile restituisce $\langle M' \rangle$:

- Su input x , se $|x|$ è **pari** allora **rifiuta**
- Altrimenti eseguo M su w e accetta $x \Leftrightarrow M$ accetta w (quindi accetto solo se x è dispari e M accetta w)

$\Rightarrow$ Se $\langle M, w \rangle \in A_{TM}$, $M(w) = \text{ACC}$

Ora $L(M') = \{ x \in \{0, 1\}^* : x \text{ ha lungh. dispari} \}$ ovvero $\langle M' \rangle \in L$

$\Leftarrow$ Se $\langle M, w \rangle \notin A_{TM}$. Allora M **rifiuta/loop**.

Ma allora $L(M') = \emptyset$ e $\langle M' \rangle \notin L$

Esempio 4

$L = \{ \langle M \rangle : M \in TM, L(M) = \{ 0^n 1^n 0^n : n \geq 0 \} \}$ non è decidibile

Sempre per riduzione: $A_{TM} \leq_m L$. Definisco $f: \Sigma^* \rightarrow \Sigma^*$ calcolabile t.c.

$\langle M, w \rangle \in A_{TM} \Leftrightarrow f(\langle M, w \rangle) \in L$

f restituisce $\langle M' \rangle$:

- Su input x simula M su w
- Se $M(w) = \text{ACC}$ e $x = 0^n 1^n 0^n$ per $n \geq 0$ allora $M'(x) = \text{ACC}$
Altrimenti se $x \neq 0^n 1^n 0^n$, $M'(x) = \text{REJ}$
- Se $M(w) = \text{REJ}$, $M'(x) = \text{REJ}$

$\Rightarrow$ Se $\langle M, w \rangle \in A_{\text{TM}}$, allora $M(w) = \text{ACC}$

In questo caso $L(M') = \{0^n 1^n 0^n : n \geq 0\}$. Quindi $\langle M' \rangle \in L$

$\Leftarrow$ Se $\langle M, w \rangle \notin A_{\text{TM}}$, $M(w) = \text{rifiuta/loop}$

In questo caso $L(M') = \emptyset$, quindi $\langle M' \rangle \notin L$

Proposta alternativa:

- Su input x , se $x \neq 0^n 1^n 0^n$ allora $M'(x) = \text{REJ}$
- Altrimenti simula $M(w)$ e accetta $M'(x) = \text{ACC} \Leftrightarrow M(w) = \text{ACC}$

$\Rightarrow$ Se $\langle M, w \rangle \in A_{\text{TM}}$, allora $M(w) = \text{ACC}$

Allora $L(M') = \{0^n 1^n 0^n : n \geq 0\}$

$\Leftarrow$ Se $\langle M, w \rangle \notin A_{\text{TM}}$, $M(w) = \text{rifiuta/loop}$

Allora $L(M') = \emptyset$.

Esercizio

Dimostrare:

Se A è Turing-Ric e $A \leq_m A^c$ allora A è **decidibile**

Idea: Se A è Turing-Ric e Co-Turing-Ric, allora A è **decidibile**.

Ma inoltre $A \leq_m A^c$ implica $A^c \leq_m A$ con la stessa mapping reduction f

Siccome A è Turing-Ric, allora pure A^c lo è

Complessità

mercoledì 26 novembre 2025 08:19

La **complessità** si interroga sull'**efficienza degli algoritmi (TM)** nella risoluzione dei problemi (decisione di linguaggi)

Efficienza in termini di **risorse**:

- **Tempo e spazio** (memoria)
- Randomness
- # processori per calcolo parallelo

Esempio:

PATH = $\{ \langle G, s, t \rangle : \exists s \rightarrow t \}$ (esiste un cammino da s a t nel grafo G)

Quanto tempo e quanto spazio richiede?

Esistono anche tantissimi problemi aperti:

- P = NP? Trovare una soluzione è tanto veloce quanto verificarne una?
- P = PSPACE? Se puoi risolvere un problema in poco spazio, puoi risolverlo in poco tempo?
- P = BPP? Ogni algoritmo efficiente può essere derandomizzato?

Complessità di Tempo

Caso deterministico

Sia M una TM decisore. La **complessità di tempo** di M è

$T: \mathbb{N} \rightarrow \mathbb{N}$ t.c.:

$$T(n) = \max_{\substack{x \in \Sigma^* \\ |x|=n}} \{ \# \text{passi richiesti da } M(x) \}$$

Siamo interessati a come questa funzione **si comporta** in termini della **lunghezza dell'input**

Classe P e Satisfiability

mercoledì 26 novembre 2025 15:50

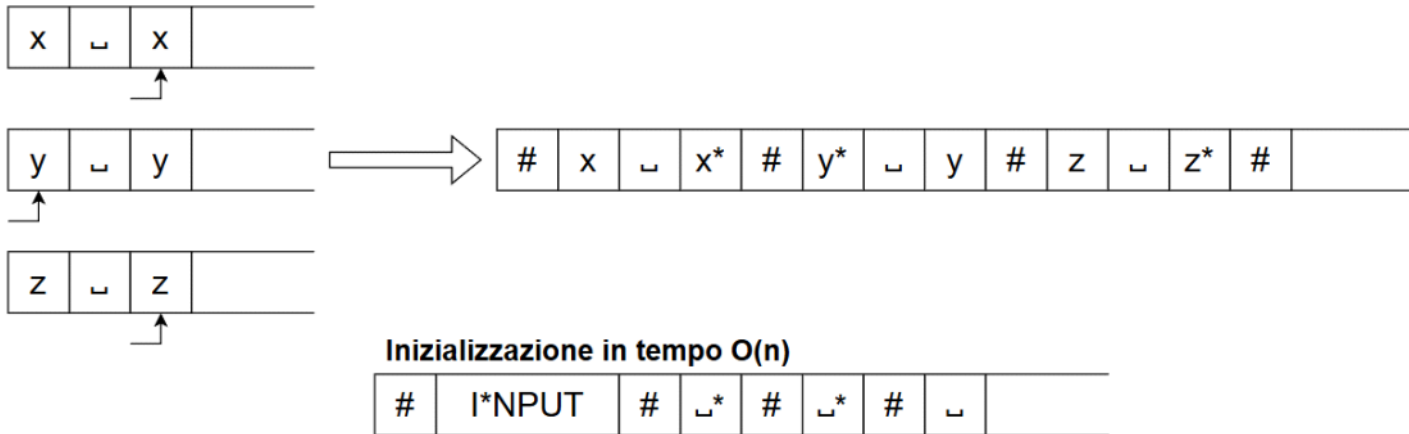
Classe DTIME (Deterministic Time)

La classe è un insieme di linguaggi

Esempi relativi alla complessità di tempo

Varianti della TM (tutte equivalenti):

- La TM M' che può fare $\{L, R, S\}$ (può fermarsi) si può simulare con TM M classica, dove invece di fermarci facciamo un passo a Dx e uno a Sx, e la complessità di tempo sarà $\leq 2T(n) = O(T(n))$ (max overhead di fattore 2, poiché fa 2 passi in più quando si ferma)
- La TM M' con k nastri, si può simulare con TM M classica e la complessità di tempo sarà $O(T^2(n))$:



Separo i nastri dalla testina e marco i simboli che vengono indicati sui 3 nastri

Marcare i simboli ha overhead costante (rimpiazzo con versione marcata)

Andare nella configurazione iniziale costerà $O(n)$ perché l'input è n

Nella simulazione, ad ogni passo:

- Devo scansionare il nastro
- Ricordare i simboli marcati
- Aggiornare nastri e posizione testine
- Se necessario, per liberare una cella di un nastro dobbiamo traslare tutto a Dx di 1 posizione

Il tempo limita lo spazio, poiché se ho t passi non potrò mai superare t celle visitate sul nastro

Quindi in questo caso la lunghezza del nastro è $3T(n) = O(T(n))$

- 1 passo $\Rightarrow O(T(n))$
- $T(n)$ passi $\Rightarrow O(T^2(n))$

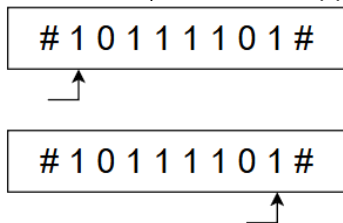
Totale: $O(n) + O(T^2(n)) = O(T^2(n))$ (poiché $T(n) \gg n$)

Nella pratica, il modello di TM può avere un impatto importante.

Es: consideriamo il linguaggio PALINDROMES delle parole palindromo su $\{0, 1\}^*$

PALINDROMES è decidibile da TM singolo nastro in tempo $O(n^2)$, poiché per ogni carattere che leggiamo dobbiamo controllare dalla parte opposta del nastro se il carattere palindromo è uguale a quello letto

Se abbiamo due nastri possiamo farlo in $O(n)$. In $O(n)$ copiamo la parola dal 1° al 2° nastro e posso leggere in contemporanea il primo carattere con l'ultimo, ecc



Teorema: ogni TM singolo nastro necessita $\Omega(n^2)$ per decidere PALINDROMES

Per questo corso, le differenze tra $O(n)$, $O(n^2)$, ecc non contano

Definizione: sia $t: \mathbb{N} \rightarrow \mathbb{R}^+$, definiamo:

$DTIME(t(n)) = \{L: \exists \text{ TM che decide } L \text{ in tempo } O(t(n))\}$

Tutti i linguaggi decidibili in tempo $O(t(n))$

PALINDROMES $\in DTIME(n^2)$ ma PALINDROMES $\notin DTIME(n)$

Per quanto riguarda la complessità i fattori costanti non importano perché vengono "assorbiti" da O grande

Classe P (Tempo Polinomiale)

Definizione: Nella classe P ci sono tutti i linguaggi che sono decidibili da TM in tempo polinomiale (efficiente):

$$P = \bigcup_{k \in \mathbb{N}} DTIME(n^k)$$

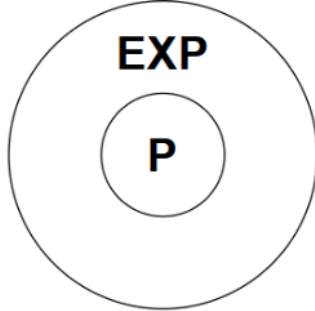
k può essere anche altissimo ma sta sempre in P

Questa definizione è robusta perché è invariante rispetto al modello di TM

Definizione: La classe EXP è la classe di linguaggi decidibili da TM in tempo esponenziale:

$$EXP = \bigcup_{k \in \mathbb{N}} DTIME(2^{n^k})$$

Una cosa importante che mostreremo:



$\exists L \text{ t.c. } L \notin P \text{ ma } L \in EXP$
 $\Rightarrow P \subset EXP$

Teorema di gerarchia di tempo (T.H.T)

La classe EXP è molto grande, ma ci sono linguaggi banalmente in EXP che si dimostrano essere in P usando strategie più "smart"

Esempio 1

$PATH = \{ \langle G, s, t \rangle : \exists s \rightarrow t \text{ in } G \}$ (esiste un cammino da s a t nel grafo G diretto)

$\langle G \rangle$ è una qualunque codifica, es la matrice di adiacenza

Tipicamente in questo caso è più comodo contare la complessità in funzione di $n = |V|$ e $m = |E|$ dove $G = (V, E)$

$s \rightarrow t: v_0, v_1, \dots, v_l \text{ t.c.}$

$v_0 = s, v_l = t, (v_i, v_{i+1}) \in E$

La lunghezza del cammino è $\leq n$ e il # cammini sarà $\leq n^n = 2^{n \log(n)}$ (tutti i possibili cammini)

Ovvero **PATH $\in$ EXP**

In realtà usando l'**algoritmo di marcatura**:

- Marco s
- Ripeto:
 - Scansiono tutti gli archi (u, v)
 - Se u è marcato e v no, marco v (marco ogni vicino dei vicini)
 - Mi fermo quando smetto di marcare
- Se t è marcato, **accetto**

In tempo $O(m \cdot n) = O(n^3)$, perché $m = O(n^2)$

Quindi **PATH $\in$ P**

Esempio 2

$2COL = \{ \langle G \rangle : G \text{ è 2-colorabile} \}$

Devo assegnare ai nodi 2 colori, es "Blu" e "Rosso", in modo che 2 nodi vicini devono avere colori diversi

possibili 2-coloring = $O(2^n) \Rightarrow 2COL \in EXP$, perché ogni nodo ha 2 possibili colori

Ma in realtà $2COL \in P$:

- Prendo un vertice qualsiasi e lo coloro Blu
- Coloro i suoi vicini Rossi
- Coloro i vicini di quei nodi Blu
- ...
- Se trovo **contraddizione** durante la colorazione **rifiuto**.
Altrimenti ripeto per ogni componente connessa in G
- Se ho **colorato tutto, accetto**

Nota: $3COL \in EXP$ ma nessuno sa se $3COL \in P$

Il miglior algoritmo decide $3COL$ in tempo $O(1.333^n)$

Esempio 3

CLIQUE: tutti i grafi indiretti che contengono un **triangolo** (ciclo di lunghezza 3)

CLIQUE $\in P$, perché # triangoli = $O(n^3)$ e controllare se è triangolo costa $O(1)$ (controllare se 3 nodi sono vicini)

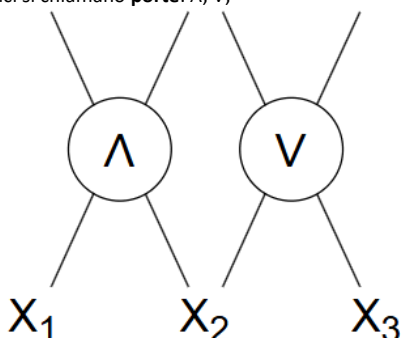
Nessuno sa se CLIQUE si può decidere in $O(n^2)$

Satisfiability

Problema fondamentale in teoria della complessità. Ci sono tante varianti:

Definizione: un **circuito booleano** C è un **grafo diretto aciclico** con **n input** $x_1, \dots, x_n$ e **1 output** (che può essere 0 o 1)

I vertici si chiamano **porte**: $\wedge, \vee, \neg$



In generale un circuito può avere un FANOUT **arbitrario**

FAN-IN: num. di cavi che **entrano** in una determinata porta

FAN-OUT: num. di cavi che **escono** da una determinata porta

Se FAN-OUT = 1 (True), C è una **formula**

Definizione: CIRCUIT-EVAL = $\{ \langle C, x \rangle : C(x) = 1 \}$, dove:

$\langle C \rangle$: una qualsiasi codifica in funzione di # input, # porte

$|\langle C \rangle| = O(m \log(m))$

CIRCUIT-EVAL $\in P$

Definizione: CIRCUIT-SAT = $\{ \langle C \rangle : \exists x \in \{0, 1\}^n \text{ t.c. } C(x) = 1 \}$

cioè un circuito è **soddisfacibile** se esiste un assegnazione dei val. delle variabili t.c. l'output è 1

Ovviamente, **CIRCUIT-SAT $\in \text{EXP}$** perché posso deciderlo in tempo **$O(\text{poly}(n) \cdot 2^n)$**

Nessuno sa se CIRCUIT-SAT $\in P$ (spoiler: CIRCUIT-SAT $\in P \Leftrightarrow P = NP$)

Varianti:

- **FORMULA-SAT**: C è una **formula**
- **CNF-SAT**: C è una **CNF** cioè un grande " $\wedge$ " di clausole (" $\vee$ " di letterali), dove:
 - **Letterale**: variabile booleana semplice (es: x o $\neg x$)
 - **Clausola**: OR di letterali (es: $(x \vee y \vee \neg z)$)
- **K-SAT**: tutte le clausole hanno k letterali

Es. 3-SAT con $k = 3$.

Ovviamente queste varianti appartengono a EXP (ma anche a P se e solo se $P = NP$)

Migliori algoritmi:

- 3-SAT $\in \text{DTIME}(1.34^n)$
- 4-SAT $\in \text{DTIME}(1.5^n \cdot \text{poly}(n))$
- ...

Teorema: 2-SAT $\in P$

Es 2-SAT: $(x \vee y) \wedge (\neg z \vee x) \wedge \dots$

Prova: Sia $\phi(x_1, \dots, x_n)$ la nostra formula 2-SAT con **n variabili** e **m clausole**

Faremo corrispondere alla formula un grafo

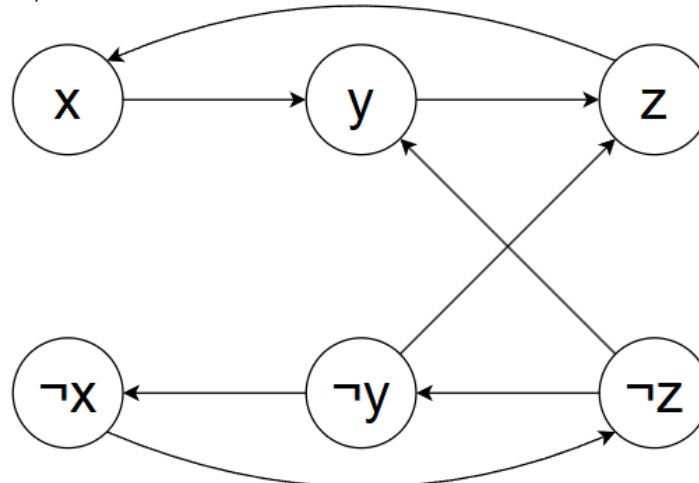
Per ogni **clausola** $x \vee y$, considero 2 **implicazioni logiche equivalenti** " $\neg x \rightarrow y$ " e " $\neg y \rightarrow x$ "

x	y	$x \vee y$	$x \rightarrow y$	$\neg x \rightarrow y$	$\neg y \rightarrow x$
0	0	0	1	0	0
0	1	1	1	1	1
1	0	1	0	1	1
1	1	1	1	1	1

Se x è 1, $x \rightarrow y$ assume il val. di y , se x è 0, $x \rightarrow y$ assume il val 1

Per ogni $l_1 \vee l_2$ in ϕ costruisco grafo G con nodi $\{x_1, \neg x_1, \dots, x_n, \neg x_n\}$ aggiungendo gli archi $\neg l_1 \rightarrow l_2$ e $\neg l_2 \rightarrow l_1$

Es: $\phi(x) = (\neg x \vee y) \wedge (\neg y \vee z) \wedge (x \vee \neg z) \wedge (y \vee z)$



LEMMA: ϕ è **soddisfacibile** $\Leftrightarrow$ nessuna componente fortemente connessa di G contiene una variabile e la sua negazione

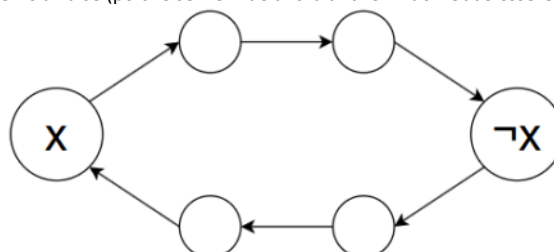
Fortemente connesso: ogni nodo raggiungibile da qualsiasi altro nodo in quella componente.

Un grafo si può dividere in **componenti fortemente connesse** e ordinare in **modo topologico** per usare l'algoritmo che decide 2-SAT

$\Rightarrow$ Sia ϕ **soddisfacibile** e sia ab un arco in G

Questo implica che se $a = \text{True}$ anche $b = \text{True}$, perché se c'è l'arco ab significa che in ϕ c'è la clausola $(\neg a \vee b)$

Se nel grafo c'è il cammino $x \rightarrow \neg x$ è necessario che x sia False (poiché se x è True allora anche $\neg x$ dovrebbe essere True ma non è possibile)



Ma allora non può esserci allo stesso tempo il cammino $x \rightarrow \neg x$ e $\neg x \rightarrow x$,
 però essendo una componente fortemente connessa se abbiamo $x \rightarrow \neg x$ abbiamo anche $\neg x \rightarrow x$
 Supponiamo ci sia il cammino $x \rightarrow \neg x$ allora non c'è il cammino $\neg x \rightarrow x$ e viceversa
 Questo perché, per lo stesso ragionamento $\neg x$ dovrebbe essere False, ovvero x è True, quindi abbiamo una **contraddizione**
 $\Rightarrow$ ogni componente fortemente connessa **non contiene** x e $\neg x$

$\Leftarrow$ Assumiamo che nessuna componente connessa comprende x , $\neg x$.

Ordiniamo le componenti fortemente connesse:

$C_1, \dots, C_t$

Assegnamento:

$\forall x$, setto $x = \text{True} \Leftrightarrow x$ appare **dopo** di $\neg x$

Altrimenti $x = \text{False}$

Affermazione: per nessun arco ab di G , il vertice a è assegnato True e b è assegnato False

Questo implica che la formula ϕ è **soddisfatta**

Supponiamo che esiste l'arco ab con $a = \text{True}$ e $b = \text{False}$ e che $a \in C_i$

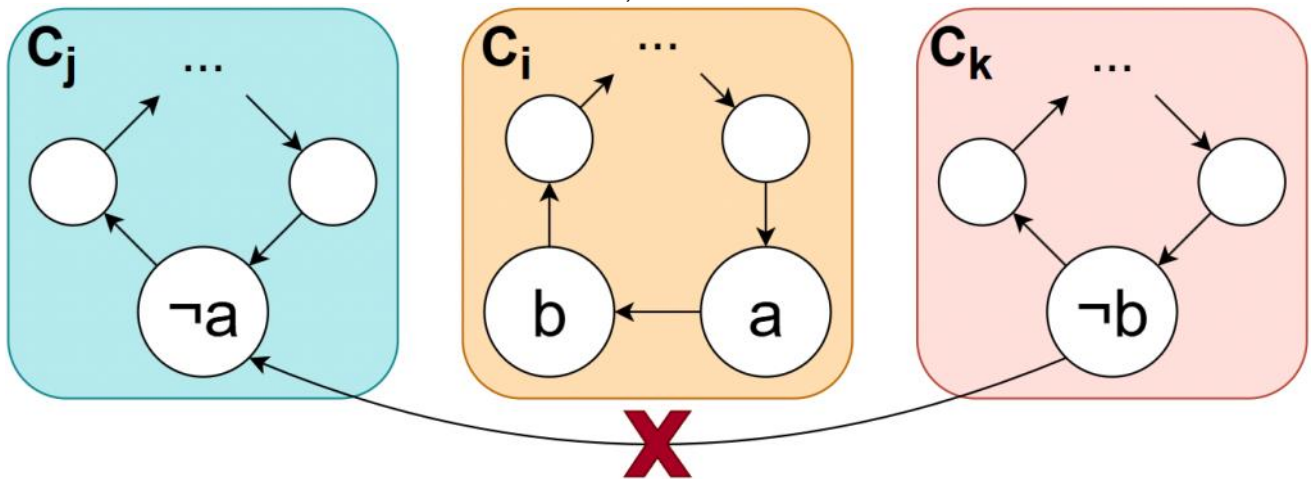
L'arco ab è presente perché c'è la clausola $(\neg a \vee b)$ che ha pure generato l'altro arco $\neg b \rightarrow a$
 (ogni clausola genera due implicazioni)

Siccome $a = \text{True}$ e $\neg a = \text{False}$, allora $a \in C_j$ **appare dopo** di $\neg a$ ovvero $\neg a \in C_j$ con $j < i$

Analogamente, siccome $b = \text{False}$ e $\neg b = \text{True}$ allora $\neg b$ **appare dopo** $b \in C_i$ (siccome c'è l'arco ab , b appartiene a C_i) ovvero $\neg b \in C_k$ per $k > i$

$\Rightarrow$ L'arco ab **contraddice l'ordine delle componenti**:

- Abbiamo che $\neg a \in C_j$ è prima di $a \in C_i$ che sono a loro volta prima di $\neg b \in C_k$
- Però poiché esiste l'arco $\neg b \rightarrow a$ abbiamo che esiste un arco tra C_k e C_i , **contraddicendo l'ordine**



Classe NP

venerdì 5 dicembre 2025 08:19

La domanda principale è se $SAT \in P$ o $3SAT \in P$, poiché è importante per la NP-completezza, ovvero $3SAT \in P \Leftrightarrow P = NP$

Intuizione per la classe NP: tanti problemi hanno la caratteristica di avere un **num. esponenziale** di **soluzioni** ma **controllare se** una soluzione è **corretta è facile** (polinomialmente)

Quindi problemi con num. **esponenziale** di **soluzioni** che si possono **risolvere** in tempo **polinomiale**

Es:

- **3COL:** il num. di 3-colorazioni è **esponenziale** in n , ma controllare se una 3-colorazione è **corretta** richiede tempo **polinomiale**
 - **Istanza:** tutti i grafi 3-colorabili
 - **Soluzione o certificato:** correttezza della 3-colorazione
- **3SAT, e CLIQUE:** stesso discorso
- **2SAT, PATH, 2COL:** sono sia in NP sia in P

Quindi esistono problemi sia in NP che in P ma non sappiamo se esistono problemi al di fuori di questi due insiemi

In generale, per i problemi in NP:

- Le soluzioni **candidate** sono codificabili con stringhe di lunghezza **poly**
- **Verificare** una sol è possibile in tempo **polinomiale**

Verificatore

Una TM V è un **verificatore** per un linguaggio L se:

- V ha input $\langle x, y \rangle$ (x = istanza, y = certificato)
- $\forall x: x \in L \Leftrightarrow \exists y$ t.c. $V(\langle x, y \rangle) = ACC$ (preso un qualsiasi linguaggio, x appartiene al linguaggio solo se esiste un certificato tale che il verificatore accetta)

Nota: questo significa:

- **YES CASE:** $x \in L \Rightarrow \exists y$ t.c. $V(\langle x, y \rangle) = ACC$
- **NO CASE:** $x \notin L \Rightarrow \forall y V(\langle x, y \rangle) = REJ$ (perché l'istanza appartiene al linguaggio)

V ha tempo di esecuzione **polinomiale** se il suo tempo di esecuzione è $O(|x|^k)$ per $k \in \mathbb{N}$

Conseguenza: $|y| = poly(n)$, perché V deve almeno leggere y

Verifier-NP (una definizione di NP)

NP è l'insieme di linguaggi L t.c. L ha un **verificatore** di tempo **polinomiale**

Es:

THM: $3COL \in NP$

DIM: basta considerare il seguente verificatore V

- Su input $\langle x, y \rangle$ dove $x = G$
- Interpreta $y = \langle c_1, c_2, \dots, c_n \rangle$ (sequenza di colori) dove $n = \# \text{vertici in } G$ e $c_i \in \{R, Y, B\}$ (uno dei tre colori per ogni vertice)
- $\forall (i, j) \in E$ (archi tra i e j) **REJ** $\Leftrightarrow c_i = c_j$ (rifiuta se due nodi adiacenti hanno lo stesso colore)
- Se non ho mai rifiutato, **ACC**

Innanzitutto, V ha tempo **polinomiale** in $|x|$ (deve controllare in tempo costante se quei nodi hanno lo stesso colore) (tempo proporzionale in base al num. di archi)

Inoltre:

- **YES CASE:** $G \in 3COL$, allora $\exists$ 3-colorazione $y = \langle c_1, \dots, c_n \rangle$ t.c. $\forall (i, j) \in E$ allora $c_i \neq c_j$. Allora $V(\langle x, y \rangle) = ACC$
- **NO CASE:** Se $V(\langle x, y \rangle) = ACC$ per qualche y allora y è una 3-colorazione. Ovvero $G \in 3COL$

Esercizio per casa:

$3SAT \in NP$

Relazione P e NP

Quale è la relazione tra P e NP?

THM: $P \subseteq NP \subseteq EXP$

Ricordiamo che $P \neq EXP$ (T.H.T)

Ma allora $P \neq NP$ o $NP \neq EXP$ o tutte e due

Perché P sta in NP?

DIM: Sia $L \in P$: $\exists M$ t.c. $L(M) = L$ e M ha tempo polinomiale

Basta considerare il verificatore che:

- Su input $\langle x, y \rangle$
- Ignora y e $ACC \Leftrightarrow M(x) = ACC$

Se $x \in L$ allora con $y = \emptyset$ (certificato vuoto), allora V accetta

Altra inclusione: se $L \in NP$, esiste verificatore V di tempo polinomiale

Questo significa, come già detto, che $|y| = O(|x|^k)$ per $k \in \mathbb{N}$ e quindi $\exists M$ che decide L in tempo esponenziale, provando tutti gli y

Altra Definizione di NP

L'altra definizione di NP è

Non-deterministic Polynomial Time

Già conosciamo la TM non-deterministica (NTM)

La complessità di tempo di una NTM è il **massimo** tempo di esecuzione su **tutti i rami** di computazione della NTM

DEF NTIME: in generale **NTIME($f(n)$)** (Non-deterministic Time) è l'insieme di linguaggi L t.c. $\exists$ NTM N per cui $L(N) = L$ e N ha tempo **$O(f(n))$**

DEF NP:

$$NP = \bigcup_{k \in \mathbb{N}} NTIME(n^k)$$

DEF NEXP:

$$NEXP = \bigcup_{k \in \mathbb{N}} NTIME(2^{n^k})$$

THM: 3SAT $\in$ NP

DIM: Su input x , considero la NTM N che su input $\langle x \rangle = \langle \phi \rangle$ una 3-SAT formula:

- Prova **non-deterministicamente** tutti i possibili assegnamenti $x_1, \dots, x_n$ e controlla $\phi(x_1, \dots, x_n) = 1$
Se **almeno** un ramo soddisfa la formula, allora N accetta

Verifier-NP = NP

THM: Verifier-NP = NP (le due definizioni di NP sono uguali)

DIM:

$\Rightarrow$ Sia L con verificatore V di tempo polinomiale in $|x|$. Devo definire una NTM N t.c. $L(N) = L$

Come sappiamo, $V(\langle x, y \rangle)$ dove $|y| = O(|x|^k)$, ovvero $\leq c \cdot x^k$ per qualche costante c

La NTM farà:

- Indovina non-determ y
- Esegue determ $V(\langle x, y \rangle)$ in ogni ramo e $ACC \Leftrightarrow V(\langle x, y \rangle) = ACC$ (V ha tempo polinomiale)

Chiaramente:

$$x \in L \Leftrightarrow \text{almeno un ramo } ACC \Leftrightarrow \exists y \text{ t.c. } V(\langle x, y \rangle) = ACC \Leftrightarrow N(x) \text{ ACC}$$

$\Leftarrow$ Sia $L \in NP$, ovvero $\exists NTM N$ t.c. $L(N) = L$ e N ha tempo polinomiale (ogni ramo è verificabile in tempo polinomiale)

Devo definire il verificatore V per L di tempo polinomiale

Idea: il certificato y sarà l'insieme delle scelte non-determ di N

Questo è rappresentabile con una stringa di lunghezza polinomiale perché N ha tempo polinomiale

Il verificatore farà:

- Su input $\langle x, y \rangle$
- Simula determ $N(x)$ usando le scelte non-determ contenute in y (ogni volta che N deve fare una scelta non-determ e **sceglie uno dei due rami possibili**)

Chiaramente:

$$x \in L \Leftrightarrow N(x) \text{ ACC} \Leftrightarrow \exists y = \text{insieme di scelte non-determ per cui } N \text{ ACC nel relativo ramo di computazione. Questo è vero} \Leftrightarrow \exists y \text{ t.c. } V(\langle x, y \rangle) = ACC$$

Quindi Verifier-NP e NP sono la stessa cosa

NP-Completezza

venerdì 5 dicembre 2025 08:33

Abbiamo visto che esistono linguaggi $L \in NP$ (es 3SAT) che non sembrano essere in P

Questi linguaggi sono **NP-completi**: $L \in P \Leftrightarrow P = NP$

Questo si può vedere tramite le **riduzioni**

DEF poly-time mapping reduction: siano A, B linguaggi.

Diremo che $A \leq_m^P B$ (esiste una **poly-time mapping reduction** da A a B) (riduzione a tempo polinomiale) se esiste **poly-time** $R: \Sigma^* \rightarrow \Sigma^*$ t.c. $\forall x \in \Sigma^*: x \in A \Leftrightarrow R(x) \in B$

Nota: è identica alla precedente definizione di mapping reduction ma ora R è **poly-time computable**

THM: se $A \leq_m^P B$ e $B \in P$, allora $A \in P$

DIM: dato una TM M_B t.c. $L(M_B) = B$ e M_B è **poly-time**, considero la TM:

$$M_A(\cdot) = M_B(R(\cdot))$$

Dove R è la **mapping reduction** per $A \leq_m^P B$

Chiaramente M_A è **poly-time** poiché R e M_B sono poly-time, quindi con complessità $\text{poly}(\text{poly}(n)) = \text{poly}(n)$ (**composizione di due polinomio è ancora un polinomio**)

Inoltre:

$$x \in A \Leftrightarrow R(x) \in B \text{ (definizione mapping reduction)} \Leftrightarrow M_B(R(x)) = \text{ACC} \Leftrightarrow M_A(x) = \text{ACC} \text{ (per definizione di } M_A)$$

Similmente

THM: Se $A \leq_m^P B$ e $B \in NP$, allora $A \in NP$

DIM: identica a prima ma sostituiamo M_A e M_B con le rispettive NTM N_A, N_B

Osservazione: " $\leq_m^P$ " è **transitiva**, cioè:

$$\text{Se } A \leq_m^P B \text{ e } B \leq_m^P C \Rightarrow A \leq_m^P C$$

Infatti esistono R_{AB}, R_{BC} t.c. $\forall x$

$$x \in A \Leftrightarrow R_{AB}(x) \in B$$

$$x \in B \Leftrightarrow R_{BC}(x) \in C$$

Quindi, ponendo $R_{AC}(\cdot) = R_{BC}(R_{AB}(\cdot))$ avremmo che:

$$x \in A \Leftrightarrow R_{AC}(x) \in C$$

e R_{AC} è **poly-time**

Esempi

Esempio 1

THM: $4COL \leq_m^P SAT$

DIM: dato un grafo $G = (V, E)$ 4-colorabile, devo costruire una formula ϕ_G t.c. **ϕ_G -soddisfacibile $\Leftrightarrow G$ è 4COL**

Se l'istanza è in 4COL allora la formula ϕ_G è soddisfacibile

La formula avrà **2n variabili**: $x_1, x_1', \dots, x_n, x_n'$ dove $n = |V|$

Ad ogni nodo associo due variabili, dove ogni coppia di variabili x_i e $x_i' \in \{0, 1\}$ codifica un colore:

x_i	x_i'	Colore
F	F	R
F	V	B
V	F	Y
V	V	W

Quindi con una coppia di variabili possiamo definire i 4 colori

Nella formula, per ogni arco $(i, j) \in E$, codifico la regola di **avere colori diversi**:

$$"(x_i, x_i') \neq (x_j, x_j')" \Leftrightarrow \neg(x_i \leftrightarrow x_j \wedge x_i' \leftrightarrow x_j') \text{ (} \leftrightarrow \text{biezione logica, } x_i \text{ implica } x_i' \text{ e } x_i' \text{ implica } x_i)$$

con " $\leftrightarrow$ ":

x	y	$x \leftrightarrow y$
F	F	V
F	V	F
V	F	F
V	V	V

Poi traduco " $\rightarrow$ " con " $\vee$ ", " $\wedge$ ", " $\neg$ " e ottengo una formula ϕ_G

$$\phi_G(\cdot) = \bigwedge_{(i,j) \in E} \phi_{ij}(\cdot)$$

La correttezza è immediata:

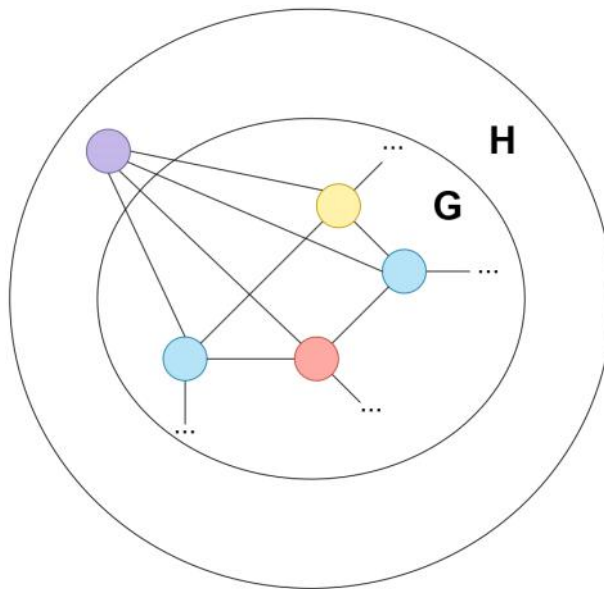
$$\phi_G \in SAT \Leftrightarrow G \in 4COL$$

(DA RIVEDERE)

Esempio 2

THM: $3COL \leq_m^P 4COL$

DIM: Devo definire $R(\langle G \rangle) = \langle H \rangle$ t.c. $G \in 3COL \Leftrightarrow H \in 4COL$



Quindi **aggiungo un nodo** ad H e lo **collego a tutti** i nodi di G
 Ora se $G \in 3COL$ allora $H \in 4COL$, perché al nuovo nodo di H posso colorarlo con il quarto colore
 Se $H \in 4COL$ allora il nuovo nodo deve essere colorato diverso da tutti e quanti i nodi di G, quindi $G \in 3COL$
 Quindi $G \in 3COL \Leftrightarrow H \in 4COL$

Conseguenza:

$$3COL \leq_m^P 4COL \leq_m^P SAT$$

ovvero

$$3COL \leq_m^P SAT$$

Si può anche dimostrare che $SAT \leq_m^P 3COL$ (più complicato)

Definizione NP-Hardness

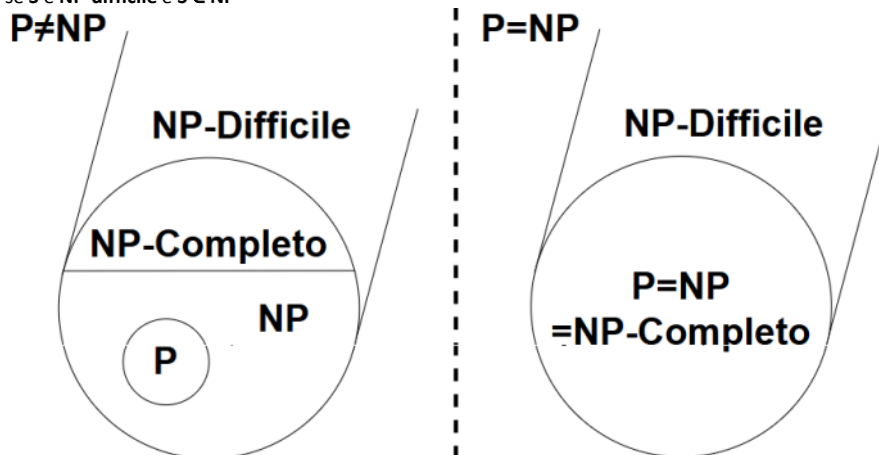
Un linguaggio S è **NP-difficile** (NP-Hard) se $\forall L \in NP$ abbiamo $L \leq_m^P S$

Tutti i linguaggi in NP si **riducono** ad S (es SAT)

I linguaggi NP-difficili non sono per forza in NP (sono almeno difficili quanto i problemi in NP ma possono essere molto più complicati)

Definizione NP-Completezza

Un linguaggio S è **NP-completo** se S è NP-difficile e $S \in NP$



L'insieme NP-difficile è aperto perché non sappiamo se possono esserci altri insiemi a cui è associato

THM: Se S è NP-completo, allora $S \in P \Leftrightarrow P = NP$

DIM:

$$\Rightarrow \forall L \in NP, L \leq_m^P S$$

Se $S \in P$ allora $L \in P$ e quindi $P = NP$

$\Leftarrow$ Se $P = NP$ e S è NP-completo allora $S \in NP$, ovvero $S \in P$ (poiché $NP = P$)

Siccome $3COL \leq_m^P SAT$, la dimostrazione $SAT \leq_m^P 3COL$ ci dimostra che anche 3COL è NP-completo (anche 4COL, CLIQUE, 3SAT, ecc)

Teorema di Cook Levin (SAT è NP-Completo)

Il **teorema di Cook Levin** ci dice che **SAT è NP-completo** ($SAT \in P \Leftrightarrow P = NP$)

DIM: In primis, $SAT \in NP$ (già dimostrato). Dobbiamo mostrare che SAT è NP-difficile

Ovvero: Dato $L \in NP$ devo far vedere

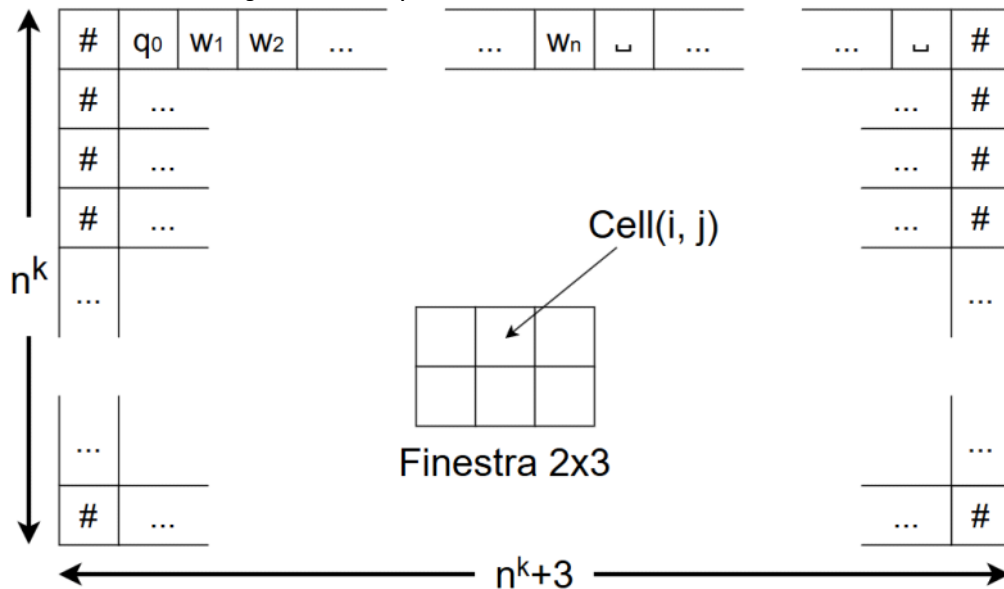
$$L \leq_m^P SAT$$

L'unica cosa che sappiamo è che $\exists NTM N$ poly-time t.c. $L(N) = L$

Supponiamo in tempo n^k per qualche $k \in \mathbb{N}$

N è **decisore** (non va mai in loop) quindi ogni suo ramo decisionale **o rifiuta o accetta**

Definiamo il concetto di **Tableau associato** ad un **singolo ramo di computazione** determ. di N:



Dimensione Tableau = $O(n^k \times n^k)$

La **1° riga** è la **configurazione iniziale**

La TM ha nastro infinito ma avendo tempo n^k esplorerà solo n^k celle, quindi delinearono il nastro usabile tra due caratteri #

Esiste una **finestra** all'interno del Tableau di grandezza 2x3 che delimita la finestra di movimento della testina, poiché, siccome la testina si può muovere solo a Sx o Dx, se nel passo i (riga i) si trova nella cella al centro (i, j) allora al passo successivo i+1 la testina sarà o in j-1 o in j+1

Stabilire se $w \in L$ è equivalente a stabilire se $\exists$ **Tableau accettante** (contiene una configurazione di accettazione)

Devo costruire una formula ϕ_N che è **soddisfacibile** $\Leftrightarrow$ N accetta w

Le **variabili** di ϕ_N : Sia Q insieme degli stati e Γ l'alfabeto di nastro. Pongo:

$$C = Q \cup \Gamma \cup \{\#\}$$

Per ogni $i, j \in [n^k]$ (per un elemento del Tableau in posizione i, j) associo la variabile

$$x_{i,j,s}^{SEC} = 1 \Leftrightarrow Cell(i, j) = s$$

La formula ϕ_N sarà:

$$\phi_N = \phi_{cell} \wedge \phi_{start} \wedge \phi_{move} \wedge \phi_{accept}$$

Dove:

- ϕ_{cell} : dice che una cella porta con se **un solo simbolo**. Si può ottenere così:

$$\phi_{cell}(x) = \bigwedge_{i,j \in [n^k]} \left[\left(\bigvee_{s \in C} x_{i,j,s} \right) \wedge \left(\bigwedge_{\substack{s,t \in C \\ s \neq t}} (\neg x_{i,j,s} \vee \neg x_{i,j,t}) \right) \right]$$

Per ogni coppia di variabili almeno una assume valore F

- ϕ_{start} : dice che la prima riga deve contenere la configurazione iniziale $\#q_0 w_1 \dots w_n \#$

$$\phi_{start}(x) = x_{1,1,\#} \wedge x_{1,2,q_0} \wedge x_{1,3,w_1} \dots \wedge x_{1,n^k,\#}$$
- ϕ_{move} : assicura che ogni riga del tableau sia consistente rispetto alla δ_i a partire dalla riga precedente
 Per assicurare questo, usiamo la finestra 2x3. Diremo che una finestra è **lecita** se **non viola la regola di δ_i**
 Se ad es $\delta_n(q_0, w_1) = (p, s, R)$ è **lecita**

#	q ₀	w ₁	...
#	s	p	w ₂ ...

- ϕ_{accept} : esiste una configurazione accettante

$$\phi_{accept}(x) = \bigvee_{i,j \in [n^k]} x_{i,j,q_{acc}}$$

Osservazione: se la prima riga è la configurazione iniziale e ogni finestra del tableau è lecita, allora ogni riga segue dalla precedente in accordo alla δ_N
 È chiaramente vero, segue dalla definizione di finestra lecita

$$\phi_{move}(x) = \bigwedge_{i,j \in [n^k]} ("finestra(i, j) \text{ lecita}")$$

finestra(i, j) ha cell (i, j) in alto al centro

Inoltre:

$$"finestra(i,j) \text{ lecita}" = \bigvee_{a_1, \dots, a_6} (x_{i,j-1,a_1} \wedge x_{i,j,a_2} \wedge x_{i,j+1,a_3} \wedge x_{i+1,j-1,a_4} \wedge x_{i+1,j,a_5} \wedge x_{i+1,j+1,a_6})$$

a1	a2	a3
a4	a5	a6

f

domenica 8 febbraio 2026 18:27

F

Definizioni Utili

venerdì 3 ottobre 2025 18:31

Definizione della **funzione estesa**:

- $\delta^*(p, \epsilon) = p$
- $\delta^*(p, xa) = \delta(\delta^*(p, x), a)$

Costruire DFA da NFA

Dato NFA $N = (Q_N, \Sigma, \delta_N, q_0^N, F_N)$, devo costruire DFA $D = (Q_D, \Sigma, \delta_D, q_0^D, F_D)$ t.c. $L(D) = L(N)$

Parte teorica:

- $Q_D = \mathcal{P}(Q_N)$
- $F_D = \{R \in Q_D : R \cap F_N \neq \emptyset\}$
- Se **non ci sono ϵ -archi**, è già quasi un DFA. Costruiamo D:
 - $q_0^D = \{q_0^N\}$
 - Dati $a \in \Sigma, R \in Q_D$, vale:

$$\delta_D(R, a) = \bigcup_{r \in R} \delta_N(r, a)$$
- Se **ci sono ϵ -archi**, definiamo la funzione E:
 - Per ogni $q_i \in Q_D$, $E(q_i) = \{q_j \in Q_D : q_j \text{ può essere raggiunto da } q_i \text{ attraverso } \geq 0 \text{ } \epsilon\text{-archi}\}$ (insieme di stati raggiungibili da q_i muovendosi con ϵ -archi)
(Es: NFA con $\delta(q_0, \epsilon) = q_1$ e $\delta(q_1, \epsilon) = q_2$, allora $E(\{q_0\}) = \{q_0, q_1, q_2\}$, $E(\{q_1\}) = \{q_1, q_2\}$ e $E(\{q_2\}) = \{q_2\}$)
 - $q_0^D = E(\{q_0^N\})$
 - Dati $a \in \Sigma, R \in Q_D$, vale:

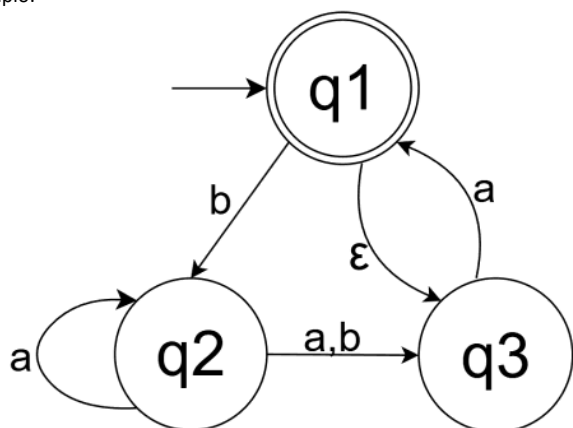
$$\delta_D(R, a) = \bigcup_{r \in R} E(\delta_N(r, a))$$

Parte pratica:

È utile creare una tabella in cui le colonne sono ogni simbolo $a \in \Sigma$:

- Se **non ci sono ϵ -archi**:
 - Iterazione tabella partendo da q_0 :
 - Uniamo insieme **tutti** gli stati in cui arriviamo da q leggendo il simbolo a . Questo è un nuovo stato (es se da q_0 arriviamo sia a q_1 che a q_2 leggendo "0", allora nella tabella in riga q_0 colonna "0" scriviamo $q_{1,2}$)
 - Se troviamo nuovi stati (es $q_{1,2}$) li aggiungiamo alla prossima riga e ripetiamo il processo su quello stato
 - Se non troviamo più stati creiamo il DFA e gli **stati finali** sono quelli che contengono almeno uno stato finale dell'NFA (es se nell'NFA avevamo come stato finale q_2 e nel DFA abbiamo nuovi stati $q_{1,2}, q_{2,4}$ allora quelli saranno i nuovi stati finali siccome sono l'unione con q_2)
- Se **ci sono ϵ -archi**:
 - I primi stati del DFA sono calcolati con $E(\{q_0\})$, quelli raggiungibili anche con ϵ -archi
 - Iterazione tabella partendo da q_0 :
 - Prendiamo l'insieme di stati in cui arriviamo partendo da q e leggendo il simbolo a
 - Applichiamo la formula E sugli stati ottenuti.
 - Se troviamo nuovi stati li aggiungiamo alla prossima riga e ripetiamo il processo
 - Se non troviamo più stati creiamo il DFA e gli **stati finali** sono quelli che contengono almeno uno stato finale dell'NFA

Esempio:



Costruisco DFA D:

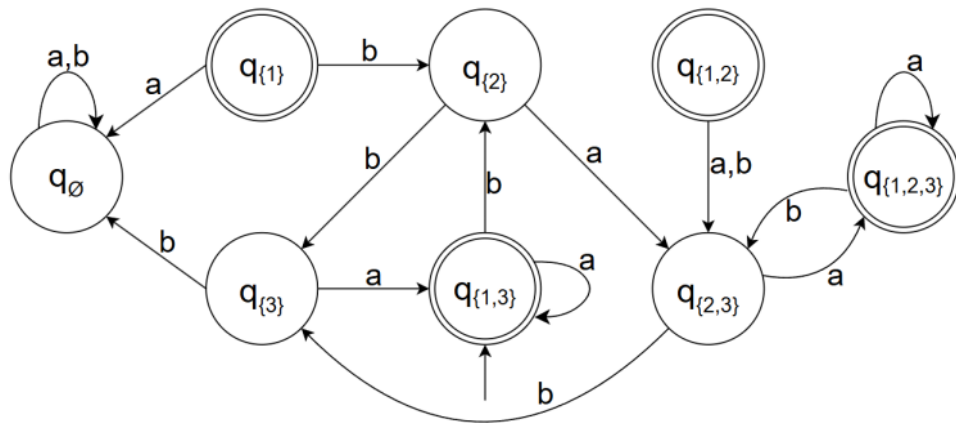
- $Q_D = \mathcal{P}(Q_N)$
- $F_D = \{R \in Q_D : R \cap F_N \neq \emptyset\}$
- $q_0^D = E(q_0^N) = \{q_1, q_{1,2}, q_{1,3}, q_{1,2,3}\}$
- Costruisco tabella dove le colonne sono i simboli "a" e "b" e inserisco come righe gli stati iniziali q_0^D .

Ad ogni insieme di stati, leggendo il simbolo $a \in \Sigma$ eseguo la formula $E(\text{Mossa}(\{\text{stati}\}, a))$

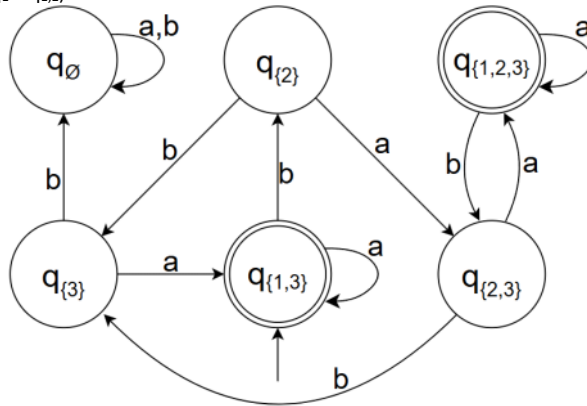
Es da $q_{1,3}$ leggo "a", devo eseguire $E(\delta_N(q_1, a)) \cup E(\delta_N(q_3, a)) = E(\emptyset) \cup E(q_1) = \emptyset \cup \{q_1, q_3\} = q_{1,3}$

f	a	b
q1	$\emptyset$	q2
q1,2	q2,3	q2,3
q1,3	q1,3	q2
q1,2,3	q1,2,3	q2,3
q2	q2,3	q3
q2,3	q1,2,3	q3
q3	q1,3	$\emptyset$

- Ora disegniamo il DFA



- Togliendo gli stati con solo archi uscenti (q_1 e $q_{1,2}$) abbiamo:



(potremmo anche togliere lo stato pozzo $q_{\emptyset}$)

Costruire NFA da Grammatica Acontestuale (Regolare)

Si può trasformare una grammatica in NFA **solo se è Regolare** (produzioni del tipo $A \rightarrow aB$, $A \rightarrow a$, $A \rightarrow \epsilon$) se abbiamo tipo $A \rightarrow aBa$ bisogna fare PDA

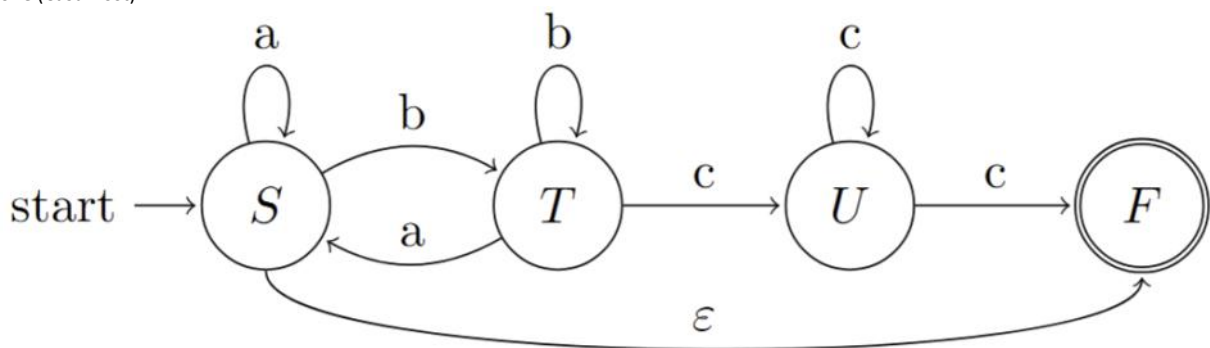
Passaggi pratici:

- Ogni variabile della grammatica diventa uno stato ($S, A, B, \dots$) e aggiungiamo lo stato finale F
- Transizioni:
 - Se $S \rightarrow aB$ allora $\delta(S, a) = B$ (freccia $S \rightarrow aB$)
 - Se $S \rightarrow a$ allora $\delta(S, a) = F$ (freccia $S \rightarrow aF$)
 - Se $S \rightarrow \epsilon$ allora o rendiamo S uno stato finale oppure $\delta(S, \epsilon) = F$ (freccia $S \rightarrow \epsilon F$)

Esempio:

$S \rightarrow aS \mid bT \mid \epsilon$
 $T \rightarrow bT \mid aS \mid cU$
 $U \rightarrow cU \mid c$

Soluzione (CasuFrost):



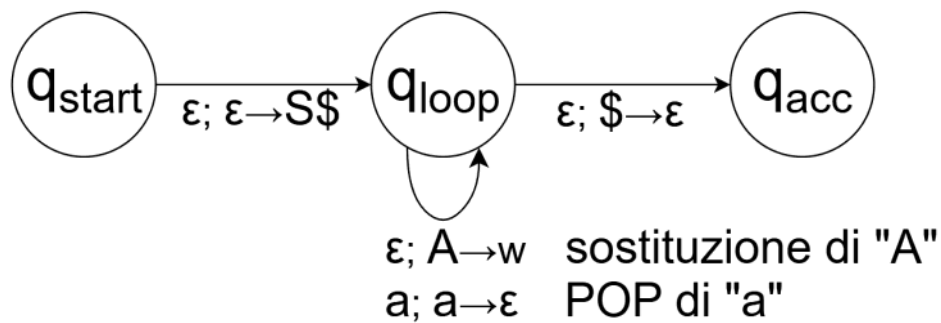
Costruire PDA da Grammatica Acontestuale

Usiamo la struttura a 3 stati.

Ricordiamoci che **non si possono semplificare le transizioni**, quindi se abbiamo una transizione $S \rightarrow xyz$ dobbiamo fare 3 nuovi stati
 Ad esempio la transizione " $\epsilon; \epsilon \rightarrow S\$$ " si deve dividere in due transizioni: prima " $\epsilon; \epsilon \rightarrow \$$ " e poi " $\epsilon; \epsilon \rightarrow S$ ".

$Q = \{q_{\text{start}}, q_{\text{loop}}, q_{\text{acc}}\}$

- Inizializzazione:** faccio il **PUSH** di $S\$$, quindi inserire le transizioni " $\epsilon; \epsilon \rightarrow \$$ " e poi " $\epsilon; \epsilon \rightarrow S$ "
- Dentro q_{loop} :**
 - Per ogni **variabile** $A \rightarrow xYz$ dobbiamo fare un loop dove leggo ϵ , POP di A e PUSH di zYx (in modo inverso)
 Es: $A \rightarrow xYz$ diventa " $\epsilon; A \rightarrow z$ " poi " $\epsilon; \epsilon \rightarrow Y$ " poi " $\epsilon; \epsilon \rightarrow x$ "
 - Per ogni **terminale** $a \in \Sigma$ dobbiamo fare un loop dove leggo a , POP di a e PUSH di ϵ
- Chiusura:** da q_{loop} a q_{acc} leggo ϵ , POP di $\$$ e PUSH di ϵ (se nella pila c'è $\$$ abbiamo finito)



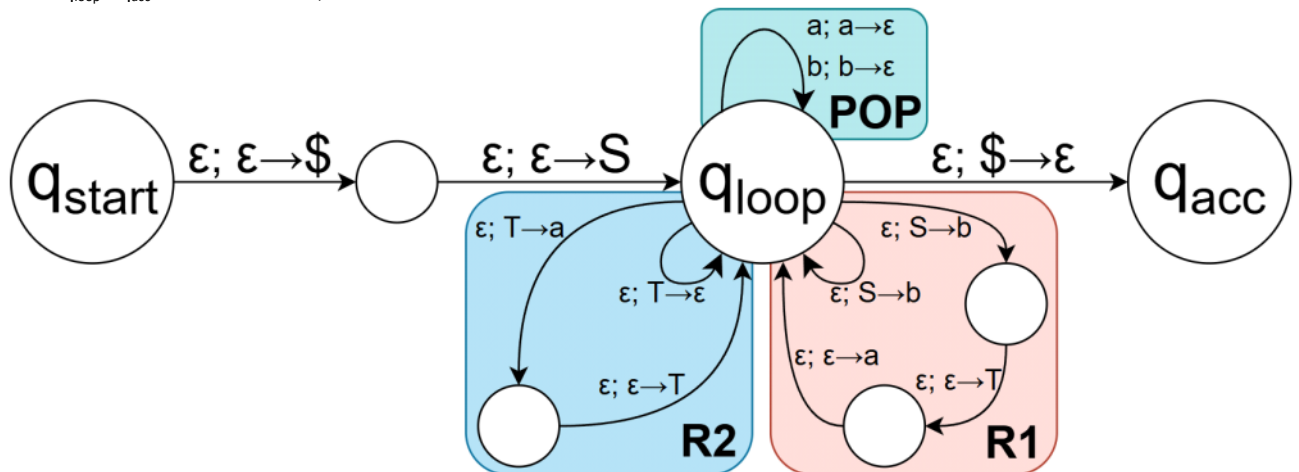
Esempio:

Sia la grammatica:

- **R1:** $S \rightarrow aTb \mid b$
- **R2:** $T \rightarrow Ta \mid \epsilon$

Costruiamo il PDA equivalente:

- Facciamo il PUSH di S
- In q_{loop} aggiungo i loop:
 - Per le variabili:
 - $S \rightarrow aTb$ facciamo il loop " $\epsilon; S \rightarrow b$ " poi " $\epsilon; \epsilon \rightarrow T$ " poi " $\epsilon; \epsilon \rightarrow a$ "
 - $S \rightarrow b$ facciamo il loop " $\epsilon; S \rightarrow b$ "
 - $T \rightarrow Ta$ facciamo il loop " $\epsilon; T \rightarrow a$ " poi " $\epsilon; \epsilon \rightarrow T$ "
 - $T \rightarrow \epsilon$ facciamo il loop " $\epsilon; T \rightarrow \epsilon$ "
 - Per i terminali:
 - Per a facciamo il loop " $a; a \rightarrow \epsilon$ "
 - Per b facciamo il loop " $b; b \rightarrow \epsilon$ "
- Infine da q_{loop} a q_{acc} facciamo il POP di S



Costruire DFA in Espressione Regolare

Prima convertiamo il DFA nel suo GNFA equivalente:

- Aggiungiamo 2 nuovi stati q_{start} (quello di partenza) e q_{acc} (quello di accettazione)
- Collegiamo q_{start} allo stato iniziale del DFA tramite ϵ -archi
- Collegiamo gli stati finali del DFA a q_{acc} tramite ϵ -archi

Procediamo con la **riduzione**, eliminando uno stato alla volta per convertire il GNFA in un'espressione regolare

Finché abbiamo **più di 2 stati** (in mezzo tra q_{start} e q_{acc}) dobbiamo eliminarne uno alla volta:

- Scegliamo uno stato da eliminare q_{rip} (ovviamente diverso da q_{start} e q_{acc}) e quando togliamo q_{rip} e i suoi archi uscenti/entranti dobbiamo ricollegare gli stati rimasti
- Per ogni coppia di stati rimasti (q_i, q_j), se passavano per q_{rip} crea un nuovo arco tra q_i e q_j usando la formula:

$$\delta(q_i, q_j) = (R_1 R_2^* R_3) \cup R_4$$

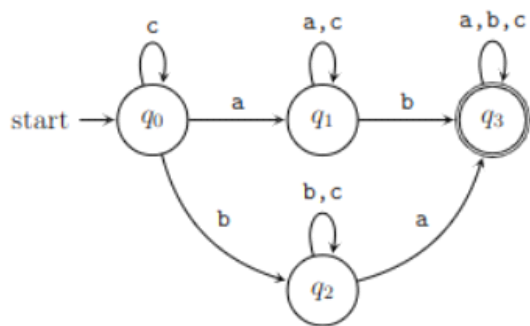
Dove:

- **R1 (ingresso):** espressione dell'arco che va da q_i a q_{rip}
- **R2 (loop):** espressione dell'arco che gira su q_{rip} (se c'è, altrimenti ϵ)
- **R3 (uscita):** espressione dell'arco che va da q_{rip} a q_j
- **R4 (vecchio arco):** espressione dell'arco che c'era già da q_i a q_j

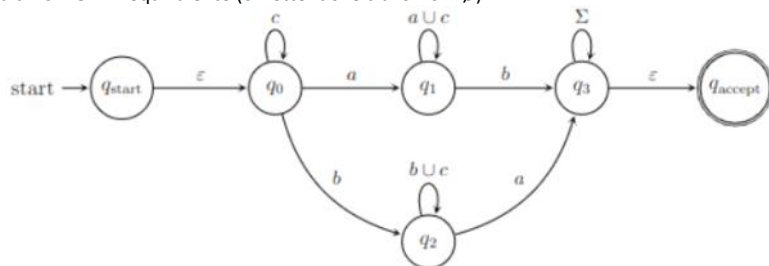
Ripetiamo finché non rimangono i 2 stati q_{start} e q_{acc} con un solo arco che li collega con l'espressione regolare finale

Esempio con immagini di Exys:

DFA per il linguaggio che contiene almeno una "a" e almeno una "b" su alfabeto $\Sigma = \{a, b, c\}$

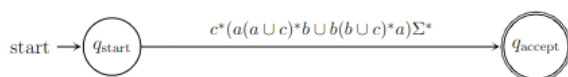
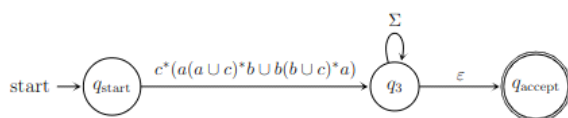
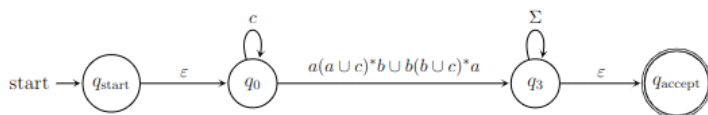
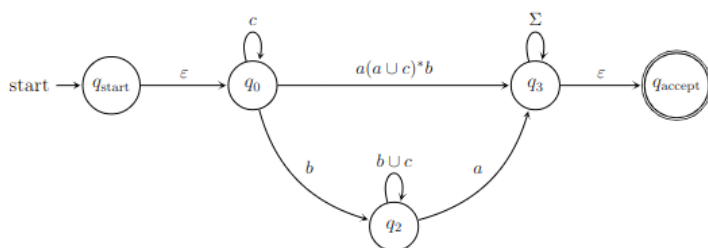


Costruiamo il GNFA equivalente (omettendo le transizioni $\emptyset$)



$(\Sigma = a \cup b \cup c)$

- Procediamo quindi con la riduzione:



- Come anticipato, l'espressione regolare ottenuta corrisponde a:

$$c^*(a(a \cup c)^*b \cup b(b \cup c)^*a)\Sigma^*$$

Linguaggio Turing-Riconoscibile: si ferma solo se accetta ma può **anche non terminare** se la stringa non appartiene al linguaggio
Esiste una TM che **riconosce** le stringhe del linguaggio

- Se $w \in L \Rightarrow$ TM si ferma e **accetta**
- Se $w \notin L \Rightarrow$ TM **può rifiutare o andare in loop**

Linguaggio Turing-Decidibile: ferma sempre e accetta o rifiuta

Esiste una TM che **si ferma sempre**

- Se $w \in L \Rightarrow$ TM si ferma e **accetta**
- Se $w \notin L \Rightarrow$ TM si ferma e **rifiuta**

Ogni linguaggio decidibile è anche riconoscibile ma non viceversa

Mapping Reduction

Dato un linguaggio L_{TM} , dimostriamo che è **indecidibile**

Dimostro esibendo una mapping reduction $ATM \leq_m L_{TM}$

Esibisco una $f: \Sigma^* \rightarrow \Sigma^*$ **calcolabile** t.c. per ogni input

$$\langle M, w \rangle \in A_{TM} \Leftrightarrow f(\langle M, w \rangle) \in L_{TM}$$

Definisco la seguente f :

- Su input $\langle M, w \rangle$
- Definisce un'altra TM M' su input $x \in \Sigma^*$ e simula M su input w :
 - Se M accetta $w \Rightarrow$ **[azione che rende vera la proprietà di L_{TM}]** (es "accetta x " o per $HALT_{TM}$ è "fermati e accetta")
 - Altrimenti, **[azione che rende falsa la proprietà di L_{TM}]** (es "rifiuta x " o per $HALT_{TM}$ è "entra in loop")
- **Output:** $\langle M' \rangle$

Correttezza:

$\Rightarrow$ Sia $\langle M, w \rangle \in A_{TM}$

Siccome $\langle M, w \rangle \in A_{TM}$, M accetta w , quindi M' esegue **[azione vera che soddisfa la proprietà di L_{TM}]**

Ovvero $\langle M' \rangle \in L_{TM}$

$\Leftarrow$ Sia $\langle M, w \rangle \notin A_{TM}$

Siccome $\langle M, w \rangle \notin A_{TM}$, M rifiuta w , quindi M' esegue **[azione falsa che non soddisfa la proprietà di L_{TM}]**

Ovvero $\langle M' \rangle \notin L_{TM}$

Esempio:

Per dimostrare che $HALT_{TM}$ è indecidibile:

- [azione vera] = "si ferma e accetta" (vogliamo che stia in $HALT$)
- [azione falsa] = "entra in loop" (vogliamo che NON stia in $HALT$)

Correttezza:

$\Rightarrow$ Se M accetta w , M' esegue l'azione "si ferma". Quindi $\langle M' \rangle \in HALT_{TM}$

$\Leftarrow$ Se M rifiuta w , M' esegue l'azione "loop". Quindi $\langle M' \rangle \notin HALT_{TM}$

Altro Modo per Dimostrare Linguaggio Indecidibile

Dato un linguaggio L_{TM} , dimostriamo che è **indecidibile**

Supponiamo per assurdo che L_{TM} è **decidibile**, allora esiste una TM R che lo decide

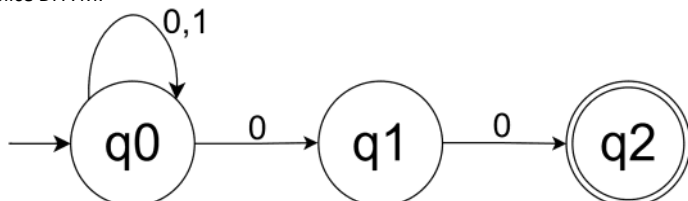
Creiamo la TM S che prende in input $\langle M, w \rangle$:

- Costruiamo poi la TM M' che simula M su input w :
 - Se M accetta $\Rightarrow M'$ fa [azione che rende vera la proprietà di L_{TM}]
 - Altrimenti $\Rightarrow M'$ fa [azione che rende
- S simula R con input M'
 - Se R accetta $\Rightarrow S$

Es in aula Automi

mercoledì 15 ottobre 2025 13:57

Esercizio DFA 1: Progettare automa per
 $L = \{w \in \{0, 1\}^* : w \text{ termina con "00"}\}$
Semplice DFA M:



Correttezza: $w \in L \Leftrightarrow M \text{ accetta } w$ (sempre da dimostrare se l'automa è corretto)

$\Rightarrow$ se $w \in L$, allora M accetta w.

Dimostriamo per induzione:

- Caso base: $w = 00$ e chiaramente M accetta 00
- Passo induttivo: $w'00$ dove $w' \in \{0, 1\}^*$. Anche in questo caso è chiaro che M accetta w

$\Leftarrow$ se M accetta w, allora $w \in L$.

Se M accetta w allora c'è un ramo di computazione che l'ha portato in q_2

Per ispezione, se M accetta w allora $\exists$ ramo di computazione che termina in q_2 .

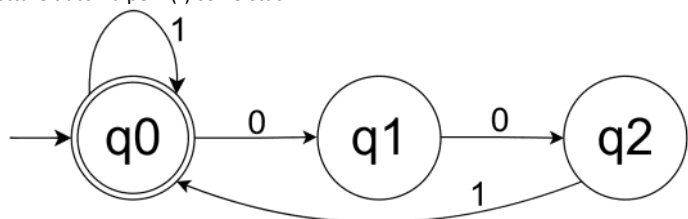
Ovvero w termina con "00" e $w \in L$

Esercizio DFA 2: Consideriamo $r = 1^*(001^+)^*$

Dove 1^+ è come 1^* ma $\geq$ una occorrenza, quindi deve starci almeno una volta ($1^+ = \{1, 11, 111, \dots\}$, $1^* = \{\epsilon, 1, 11, 111, \dots\}$)

Quindi $(001^+)^* = \{\epsilon, 001, 00111, 0010011, 00110010011, \dots\}$

Progettare automa per $L(r)$ con 3 stati



q_0 accetta perché c'è 1^* e $(001^+)^*$ quindi r può essere anche $\{\epsilon, 1, 11, 111, 001, 001001, \dots\}$

non può terminare solo con "00" ma per tornare allo stato di accettazione dobbiamo per forza avere un "1" finale da q_2 a q_0

Se non c'era "1" questo automa non andava bene perché "0011" $\notin L(r)$ dove $r = 1^*(001^+)^*$

(poiché $1^*(001^+)^* = \{\epsilon, 1, 111, 11001, 1001001, 1111001001001, \dots\}$)

Esercizio DFA 3: Data $w = w_1, \dots, w_n$ su Σ sia $w^R = w_n w_{n-1} \dots w_1$.

Sia L regolare, mostrare che $L^R = \{w^R : w \in L\}$ è regolare (L^R = insieme delle stringhe rovesciate)

Idea: scambiare stato iniziale e finale (wlog ce n'è uno solo) e invertire gli archi

Siccome L è regolare, allora significa che esiste un DFA $M = (Q, \Sigma, \delta, q_0, F)$ t.c. $L(M) = L$.

Per poter leggere L^R ci serve un nuovo automa M^R simile a M ma con gli archi invertiti e gli stati iniziali e finali scambiati tra di loro. Siccome M può avere molte transizioni entranti in un suo stato q , invertendole avremmo molte uscite da q , quindi M^R è **una NFA**.

Definiamo $M^R = (Q, \Sigma, \delta^R, F, \{q_0\})$ t.c.:

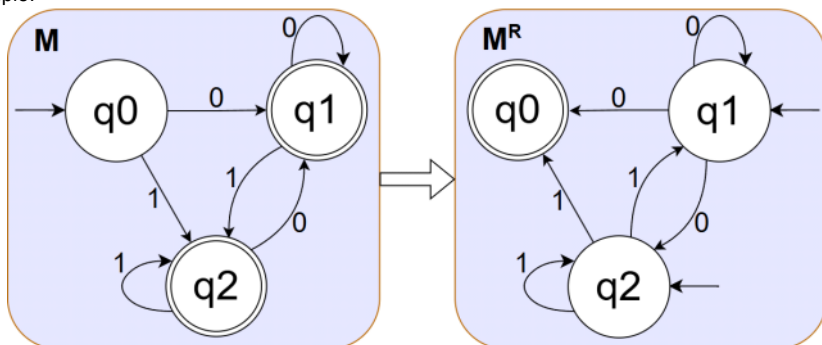
- Q e Σ rimangono uguali tra M e M^R
- La funzione di transizione δ^R è l'inversa di δ :
Se $\delta(q, a) = p$ allora $\delta^R(p, a) = q$
- Gli stati **iniziali** di M^R sono i finali di M (F)
- L'unico stato **finale** di M^R è l'iniziale di M (q_0)

Intuitivamente se M legge $w = w_1, w_2, \dots, w_n$ partendo da q_0 e finendo in uno stato $q_f \in F$,

allora M^R può leggere $w^R = w_n, w_{n-1}, \dots, w_1$, partendo da q_f e terminando in q_0 , poiché le transizioni di M che leggono L sono inverse in M^R .

Quindi siccome M^R è un NFA che accetta L^R , e siccome ogni L riconosciuto da un NFA è regolare, allora L^R è regolare

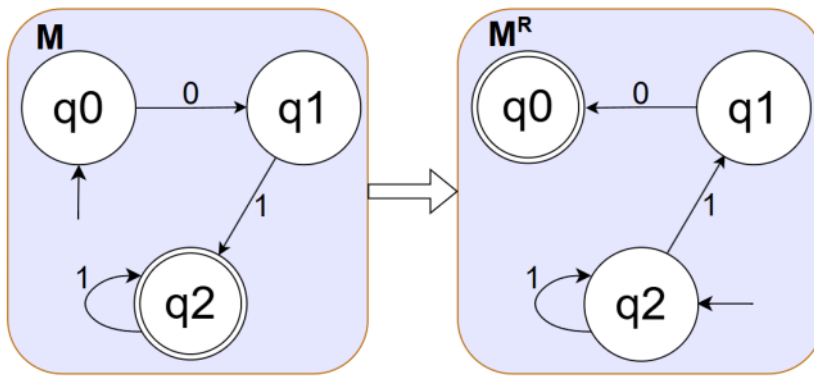
Esempio:



Oppure:

$L(M) = L = \{w \in \{0, 1\}^* : w = 011^*\}$ e $L(M^R) = L^R = \{w^R \in \{0, 1\}^* : w^R = 1^*10\}$, con:

$L = \{01, 0111, 01111, \dots\}$, $L^R = \{10, 1110, 111110, \dots\}$

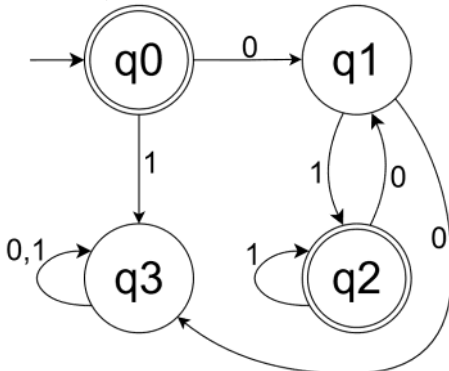


Esercizio DFA 4: Costruire DFA per $L = \{w \in \{0, 1\}^* : w \notin (01^+)^*\}$

$(01^+)^* = \{\epsilon, 01, 0101, 011, 0111, 0111011, \dots\} \Rightarrow$ complemento di $(01^+)^* = \{1, 0, 10, 101, 00, 001, \dots\}$

Uso la chiusura per il complemento:

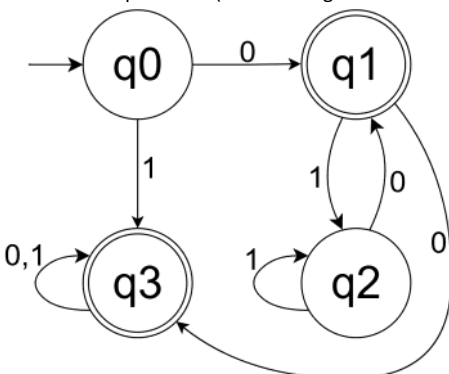
Prima faccio DFA per $(01^+)^*$:



Aggiungiamo il nodo pozzo q_3 perché ci serve per il complemento. Qualsiasi cosa che abbia due "0" non è accettabile (se togliamo q_3 abbiamo che il complemento continua a rifiutare la stringa che comincia con "1")

Importante: per applicare la chiusura è importante che il DFA sia definito su tutti gli stati (una transizione per ogni carattere in Σ)

Ora facciamo il complemento (invertendo gli stati accettabili e non):



Questo nuovo DFA accetterà solo stringhe che sono diverse da $(01^+)^*$

Esercizio Pumping Lemma 1:

$L = \{ww^R : w \in \{0, 1\}^*\}$ non è regolare

$ww^R = \{00, 11, 0110, 101101, \dots\}$

Uso il pumping lemma. Suppongo che L sia regolare, allora $\exists p$ t.c. valgono le condizioni del lemma.

Considero $w = 0^p 110^p \in L$ quindi $|w| \geq p$ ($w' = 0^p 1$ e $w^R = 10^p$)

Prendo una scomposizione arbitraria $w = xy^iz$ t.c. $|y| > 0$ e $|xy| \leq p$

Allora y è fatta di soli 0 (perché $|xy| \leq p$ quindi $xy \in 0^p$)

$w = 0^k 0^l 0^m 110^p$ con

$|xy| = k+l+m = p$; $x = 0^k$ e $y = 0^l$ con $k \geq 0$ e $l > 0$

Per $i = 2$, xy^2z abbiamo:

$0^k 0^{2l} 0^m 110^p$ con

$|xy^2z| = k+2l+m = k+l+m+l = p+l > p$ (perché $l > 0$)

Quindi non è più palindromo perché abbiamo più "0" a sinistra che a destra

Esercizio Pumping Lemma 2:

$L = \{w = 1^n : n \geq 0\}$ non è regolare

$1^n = \{1^0 = \epsilon, 1^1 = 1, 1^2 = 1111, 1^3 = 11111111, \dots\}$ (il numero di "1" deve essere al quadrato)

Suppongo sia regolare e scelgo $w = 1^{p^2} \in L$

Considero una scomposizione generica $w = xy^iz$ t.c. $|y| > 0$ e $|xy| \leq p$

In particolare:

$w = 1^k 1^l 1^{p^2-k-l}$ dove $x = 1^k$, $y = 1^l$, $z = 1^{p^2-k-l}$ con $k \geq 0$, $l > 0$

$|xy| = k+l \leq p$

Per $i = 2$, xy^2z abbiamo:

$w = 1^k 1^{2l} 1^{p^2-k-l}$ (devo dimostrare che la sua lunghezza non è al quadrato)

$$|xy^2z| = k + 2l + p^2 - k - l = p^2 + l > p^2 \text{ (perché } l > 0)$$

Pero incrementando p di 1 abbiamo:

$$(p+1)^2 = p^2 + 2p + 1$$

$$\geq p^2 + 2(k+l) + 1 = p^2 + 2k + 2l + 1 \text{ (perché } k+l \leq p)$$

$$> p^2 + l$$

Quindi:

$$|xy^2z| = p^2 + l < (p+1)^2$$

$$|xy^2z| > p^2$$

Quindi $p^2 < |xy^2z| = p^2 + l < (p+1)^2$ è un numero **compreso tra due quadrati consecutivi**, quindi non può essere un quadrato perfetto, quindi non sta nel linguaggio

Esercizio: Progettare CFG per i linguaggi

- $L_1 = \{w \in \{0, 1\}^* : w \text{ contiene almeno 3 "1"}\}$

Risposta **sbagliata**: Creare 4 regole così:

- $S \rightarrow 1A$
- $A \rightarrow 1B$
- $B \rightarrow 1C$
- $C \rightarrow \epsilon$

Questo genera solo stringhe "111", invece sono richieste tutte le stringhe che hanno "1" almeno 3 volte in qualsiasi posizione

Risposta **corretta**: S impone che ci siano almeno 3 "1" nella stringa e T genera qualsiasi sequenza di "0" e "1"

- $S \rightarrow T1T1T1T$
- $T \rightarrow 0T \mid 1T \mid \epsilon$

Es: $S \rightarrow T1T1T1T \rightarrow 0T1T1T1T \rightarrow 0T1T1T1T \rightarrow 00T1T1T1T \rightarrow 001T1T1T \rightarrow 001T111T \rightarrow 0010T111T \rightarrow 0010T1110T \rightarrow 0010T1110 \rightarrow 00101110$

- $L_2 = \{w \in \{0, 1\}^* : w = w^R, |w| \text{ è pari}\}$ (palindromi di lunghezza pari)

- $S \rightarrow 0S0 \mid 1S1 \mid \epsilon$

Essendo un palindromo possiamo avere sia una stringa $S = \epsilon$ di lunghezza 0 (pari) oppure ad ogni transizione "0S0" o "1S1" aggiungiamo due simboli identici specchiati rispetto a S, quindi la lunghezza resta pari e creiamo una stringa "prima metà + inversa della prima metà" che è un palindromo

- $L_3 = \{a^i b^j c^k : i + j = k \text{ e } i, j, k \geq 0\}$

- $S \rightarrow X$
- $X \rightarrow aXc \mid Y$
- $Y \rightarrow bYc \mid \epsilon$

Poiché i, j, k possono anche essere 0 potremmo anche avere $a^5 b^0 c^5$ ($i+j=k \Rightarrow 5+0=5$) perciò per ogni "a" e "b" univoca viene aggiunta una "c" corrispondente.

Per mantenere l'ordine "prima a", poi "b", poi "c" la regola "aXc" assicura che vengono generate le coppie "a" e "c" con X nel mezzo e quando passiamo a Y la regola "bYc" assicura che venga quella variabile tutte le coppie "b" e "c" che saranno a metà tra le coppie "a" e "c" generate in precedenza

Es: $S \rightarrow X \rightarrow aXc \rightarrow aaXcc \rightarrow aaaXccc \rightarrow aaaYccc \rightarrow aaabYcccc \rightarrow aaabbYccccc \rightarrow aaabbccccc$ (quindi $|a| = 3, |b| = 2, |c| = 5; i+j=k \Rightarrow 3+2=5$)

Esami

giovedì 8 gennaio 2026 12:33

Esami e soluzioni degli esami sono su SapienzaStudentNetwork (<https://github.com/sapienzastudentsnetwork/automi-calcolabilita-complessita>)

Teoremi esami passati

giovedì 8 gennaio 2026 19:02

2023

9 gennaio

- * Automi: Forma normale di Chomsky
- * Calcolabilità: Dimostrare che TM M dove $L(M)$ = insieme vuoto è indecidibile
- * Complessità: Definire PSPACE e NPSPACE e dimostrare $PSPACE = NPSPACE$

6 febbraio

- * Automi: Definire PDA e dimostrare che linguaggio è acontestuale se esiste un PDA che lo riconosce
- * Calcolabilità: Dimostrare che L è decidibile se L è turing-ric e coturing-ric
- * Complessità: Definire P e NP e NP-completezza. Mostrare che se $P = NP$, allora NP coincide con la classe NP-completi

27 marzo

- * Automi: Pumping lemma
- * Calcolabilità: linguaggio è turing-ric se esiste una tm multinastro che lo riconosce
- * Complessità: definire 2-sat e mostrare che appartiene a P

5 giugno

- * Automi: pumping lemma
- * Calcolabilità: esistono linguaggi non decidibili e non turing-ric
- * Complessità: teorema di gerarchia di tempo

6 luglio

- * Automi: dimostrare se linguaggio è regolare esiste una espressione regolare che lo descrive
- * Calcolabilità: dimostrare per ogni TM non-deterministica esiste una deterministica equivalente
- * Complessità: definire coNP e il problema UNSAT. dimostrare che UNSAT è coNP-completo

6 settembre

- * Automi: dimostrare che un linguaggio è acontestuale se esiste una PDA che lo riconosce
- * Calcolabilità: dimostrare che $EQTM = \{ \langle M1, M2 \rangle : L(M1) = L(M2) \}$ è indecidibile
- * Complessità: concetto di riduzione. dimostrare $3COL \leq_{mp} SAT$

2024

10 gennaio

- * Automi: pumping lemma
- * Calcolabilità: concetto di riducibilità tramite funzione
- * Complessità: definire PSPACE e NPSPACE, dimostrare $PSPACE = NPSPACE$

8 febbraio

- * Automi: definire grammatiche acontestuali e dimostrare che ogni grammatica acontestuale ne ammette una equivalente in forma di chomsky
- * Calcolabilità: dimostrare che ATM è indecidibile e che ATM non è turing-ric
- * Complessità: teorema di gerarchia di tempo. usare teorema per mostrare che PSPACE subset di EXPSPACE

8 aprile

- * Automi: dimostrare che linguaggio è regolare se esiste una espressione regolare che lo descrive
- * Calcolabilità: dimostrare che esistono linguaggi non turing-ric.
- * Complessità: definire complessità di spazio. Enunciare teorema di Savitch

6 giugno

- * Automi: pumping lemma
- * Calcolabilità: dimostrare linguaggi non decidibili e non turing-ric
- * Complessità: teorema di gerarchia di tempo

3 luglio

- * Automi: dimostrare se linguaggio è regolare esiste espressione regolare che lo descrive
- * Calcolabilità: dimostrare che per ogni TM non-deterministica esiste una deterministica equivalente
- * Complessità: definire classe coNP e problema UNSAT. Dimostrare che UNSAT è coNP completo

11 settembre

- * Automi: dimostrare linguaggio acontestuale se esiste PDA che lo riconosce
- * Calcolabilità: dimostrare che $EQTM = \{ \langle M1, M2 \rangle : L(M1) = L(M2) \}$ è indecidibile
- * Complessità: concetto di riduzione. dimostrare $3COL \leq_{mp} SAT$

2025

14 gennaio

- * Automi: pumping lemma
- * Calcolabilità: esistono linguaggi non decidibili e non turing-ric
- * Complessità: classi PSPACE e NPSPACE e dimostrare $PSPACE = NPSPACE$

4 febbraio

- * Automi: definire grammatiche acontestuali e dimostrare ogni grammatica acontestuale ne ammette una in forma di Chomsky
- * Calcolabilità: primo teorema di incompletezza di Goedel
- * Complessità: teorema di gerarchia di tempo. usare teorema per mostrare che PSPACE subset di EXPSPACE

4 aprile

- * Automi: dimostra che linguaggio è acontestuale se esiste PDA che lo decide
- * Calcolabilità: secondo teorema di incompletezza di Goedel
- * Complessità: concetti di riducibilità polinomiale NP-completezza e coNP-completezza. Dimostrare che linguaggio L è NP-completo se notL è coNP-completo

10 giugno

- * Automi: linguaggio è regolare se esiste NFA che lo riconosce
- * Calcolabilità: concetto di riducibilità polinomiale mediante funzione. Dimostrare che linguaggio è turing-decidibile se è sia turing-ric e coturing-ric
- * Complessità: classi PSPACE e NPSpace e dimostrare PSPACE = NPSpace

15 luglio

- * Automi: pumping lemma
- * Calcolabilità: linguaggio ATM è indecidibile
- * Complessità: complessità di tempo per TM. Enunciare teorema di gerarchia per la complessità di tempo

9 settembre

- * Automi: linguaggio è acontestuale se esiste PDA che lo riconosce
- * Calcolabilità: dimostrare che EQTM = $\{ \langle M1, M2 \rangle : L(M1) = L(M2) \}$ è indecidibile
- * Complessità: concetto di riduzione. dimostrare 3COL $\leq_p$ SAT

5 novembre

- * Automi: equivalenza tra linguaggi regolari e espressioni regolari
- * Calcolabilità: dimostrare che ATM è indecidibile
- * Complessità: definire complessità di spazio per TM. enunciare teorema di Savitch

2026

12 gennaio

- * Automi: Enunciare e dimostrare il pumping lemma per linguaggi acontestuali. Fornire un esempio di utilizzo
- * Calcolabilità: Dimostrare che esistono linguaggi che non sono Turing-decidibili. Fornire un esempio di linguaggio che non sia Turing-riconoscibile
- * Complessità: Definire le classi di complessità P, NP, EXP, NEXP. Enunciare e dimostrare il teorema di gerarchia di tempo

Esercizi esami passati

giovedì 8 gennaio 2026 19:02

2023

9 gennaio

* Automi: Scrivere espressione regolare per linguaggio su alfabeto $\{a, b, c\}$ contenente almeno una a e una b . Determini un DFA che riconosca il linguaggio

* Calcolabilità: per $k \in \mathbb{N}$, siano $L_1, \dots, L_k$ linguaggi su alfabeto Σ . Si assuma che:

- 1) $L_i \cap L_j = \emptyset$ per ogni $i \neq j$
- 2) $L_1 \cup \dots \cup L_k = \Sigma^*$
- 3) L_i è turing-ric per ogni $i \in [k]$

Dimostrare che $L_1, \dots, L_k$ sono decidibili

* Complessità: Si consideri QP la classe di linguaggi decidibili in tempo polinomiale. Siano L_1 e L_2 due linguaggi, stabilire se l'affermazione è vera/falsa: se $L_1 \leq_m L_2$ e L_2 appartiene a QP, allora L_1 appartiene a QP

6 febbraio

* Automi: Sia $A = (Q, \Sigma, \delta, q_0, \{q_f\})$ una DFA e supponiamo che per ogni $a \in \Sigma$ si abbia $\delta(q_0, a) = \delta(q_f, a)$

- Si mostri che per ogni $w \neq \epsilon$ si ha $\delta^*(q_0, w) = \delta^*(q_f, w)$ dove δ^* è la funz. di transizione estesa
- Si mostri che se x è una stringa non vuota in $L(A)$ allora x^k appartiene a $L(A)$ per ogni $k > 0$

* Calcolabilità: Descrivere una TM che riconosca il linguaggio delle stringhe binarie contenenti lo stesso num. di 0 e 1. Dimostrare correttezza

* Complessità: Determinare se questo problema è in NL: dato grafo non orientato $G = (V, E)$, due vertici $s, t \in V$ e un intero $k \in \mathbb{N}$, stabilire se esiste un cammino da s a t di lunghezza k

27 marzo

* Automi: progettare DFA che riconosca linguaggi su $\Sigma = \{0, 1, \dots, 9\}$ t.c. la somma delle cifre sia multiplo di 3. Provare correttezza

* Calcolabilità: Consideri linguaggio WTM delle stringhe $\langle M, w, a \rangle$ dove M è una TM, w è una stringa dell'alfabeto di input di M e a è un simbolo dell'alfabeto del nastro di M , t.c. M scrive a sul nastro quando è eseguita su input w . Mostrare che WTM è indecidibile

* Complessità: mostrare che il problema 1/2-CLIQUE è NP-completo: dato grafo G con num. pari di vertici, stabilire se esiste una clique in G di taglia uguale alla metà del num. di vertici nel grafo

5 giugno

* Automi: Considerare espressione regolare $1\Sigma^* = 1(0 \cup 1)^*$. Convertire l'espressione in una grammatica acontestuale. Dimostrare correttezza

* Calcolabilità: uno stato di una TM è detto inutile se la TM non entra mai in quello stato per ogni input. Considera il problema di determinare se una TM ha uno stato inutile. Dimostrare che il linguaggio associato al problema non è decidibile

* Complessità: una TM fortemente non-deterministica ha 3 possibili stati finali: accept, reject e notsure. Tale TM decide un linguaggio L come segue: se $x \in L$ tutti i rami di computazione con input x portano ad accept o notsure, con almeno un ramo termina in accept. Se $x \notin L$, tutti i rami con input x portano a reject o notsure con almeno un ramo termina in reject. Dimostrare che L è decidibile in tempo polinomiale da una TM fortemente non-deterministica se e solo se $L \in \text{NP} \cap \text{coNP}$

6 luglio

* Automi: dimostrare che il linguaggio $\{1^i \# 1^j \# 1^{i+j} : i, j \geq 1\}$ su alfabeto $\{\#, 1\}$ non è regolare

* Calcolabilità: Sia DECIDABLETM il linguaggio dove tutte le stringhe $\langle M \rangle$ t.c. M è una TM e $L(M)$ è decidibile. Mostrare che DECIDABLETM è indecidibile

* Complessità: Dimostrare che la classe NL è chiusa rispetto alle operazioni di unione, intersezione e star di Kleene

6 settembre

* Automi: Sia L_k il linguaggio di tutte le stringhe su $\{0, 1, \dots, 9\}$ la somma delle cui cifre è multiplo di k per $k \in \mathbb{N}$. Descrivere DFA che riconosca L_k per ogni $k \in \mathbb{N}$.

Disegnare diagramma di transizione degli stati del DFA per il caso $k = 4$

* Calcolabilità: Sia HALTS_ON_ALLTM il linguaggio di tutte le stringhe $\langle M \rangle$ t.c. M è una TM che termina sempre per ogni scelta dell'input. Mostrare che HALTS_ON_ALLTM è indecidibile

* Complessità: Dimostrare che $\text{NTIME}(n^k) \subseteq \text{PSPACE}$ per ogni $k \in \mathbb{N}$. Questo implica $\text{NP} \subseteq \text{PSPACE}$? motivare risposta

2024

10 gennaio

* Automi: Sia $L = \{11, 110\}^*$. Costruire NFA con 4 stati che riconosce L . Convertire l'NFA in DFA equivalente

* Calcolabilità: considerare linguaggio REJTM = $\{\langle M, w \rangle : M \text{ è una TM che rifiuta } w\}$ mostrare che REJTM è turing-ric ma non turing-decidibile

* Complessità: considerare $3\text{COL} = \{G : G \text{ è grafo 3-colorabile}\}$ e $4\text{COL} = \{G : G \text{ è grafo 4-colorabile}\}$. Mostrare che $3\text{COL} \leq_m 4\text{COL}$

8 febbraio

* Automi: Dimostrare che $L = \{w : w \text{ termina con 0 o ha lunghezza pari}\}$ è regolare

* Calcolabilità: consideri una definizione alternativa di TM in cui il nastro è infinito in entrambe le direzioni. Mostrare che tale definizione è equivalente a quella data (cioè il nastro infinito in una sola direzione)

* Complessità: considerare $4\text{COL} = \{G : G \text{ è grafo 4-colorabile}\}$. Mostrare che $4\text{COL} \leq_m \text{SAT}$

8 aprile

* Automi: usare pumping lemma per mostrare che $\{c^4 a^n b^m : n \geq m\}$ su $\Sigma = \{a, b, c\}$ non è regolare

* Calcolabilità: sia $B = \{\langle M \rangle : M \text{ è una TM e } L(M) = \{01\}^*\}$ Mostrare che $\text{ATM} \leq_m B$. cosa si conclude sulla decidibilità di B ?

* Complessità: Dimostrare che se $P = \text{NP}$, allora $\text{EXP} = \text{NEXP}$

6 giugno

* Automi: Considerare la grammatica G :

$S \rightarrow WbT$
 $T \rightarrow aWbT \mid bVaT \mid \epsilon$
 $W \rightarrow aWbW \mid \epsilon$
 $V \rightarrow bVaV \mid \epsilon$

mostrare che $aabbbbaab$ appartiene a $L(G)$. Descrivere PDA equivalente a G

* Calcolabilità: Sia NONREGULARTM il linguaggio con $\langle M \rangle$ che costituiscono codifiche valide di TM t.c. $L(M)$ non è regolare. Mostrare che $\text{ATM} \leq_m$

NONREGULARTM

* Complessità: Un cammino in un grafo non orientato è detto semplice se non contiene vertici ripetuti. Sia $\text{LONGEST-PATH} = \{ \langle G, s, t, k \rangle : G = (V, E) \text{ grafo non orientato, } s, t \in V, \text{ esiste un cammino semplice } s \rightarrow t \text{ di lunghezza } \geq k \}$ mostrare che $\text{LONGEST-PATH} \in \text{NP}$

3 luglio

* Automi: Mostrare che la seguente grammatica è ambigua:

$S \rightarrow T b T$

$T \rightarrow a T b T \mid b T a T \mid \epsilon$

Per dimostrare che è ambigua bisogna dimostrare che esistono due derivazioni sinistre per la stessa parola

* Calcolabilità: Sia $\text{ALLNFA} = \{ \langle N \rangle : N \text{ è una NFA e } L(N) = \Sigma^* \}$ mostrare che ALLNFA è decidibile

* Complessità: Sia L un linguaggio PSPACE-completo. Mostrare che se $L \in \text{NP}$, allora $\text{NP} = \text{PSPACE}$

11 settembre

* Automi: Sia C un linguaggio context-free e R un linguaggio regolare. Mostrare che il linguaggio $C \cap R$ è context-free. Usare questo risultato per mostrare che il linguaggio L delle stringhe $w \in \{a, b, c\}^*$ t.c. w contiene lo stesso num. di a , b e c non è context-free

* Calcolabilità: considera linguaggio L delle stringhe $u \in \{a, b\}^*$ t.c. u ha un num. di a che è almeno il doppio del num. di b . Descrivere TM che decide L in tempo $O(n^2)$. Descrivere TM multi-nastro che decide L in tempo $O(n)$

* Complessità: supporre esistenza di una funzione iniettiva f che mappa interi a n -bit in interi a n -bit, t.c.:

1) $f(x)$ è calcolabile in tempo polinomiale

2) $f^{-1}(x)$ non è calcolabile in tempo polinomiale.

Mostrare che il linguaggio delle stringhe $x, y \in \{0, 1\}^n$ t.c. $f^{-1}(x) < y$ è contenuto in $(\text{NP} \cap \text{coNP}) \setminus \text{NP}$

2025

14 gennaio

* Automi: Progettare NFA che riconosca la seguente grammatica

$S \rightarrow aS \mid bT \mid \epsilon$

$T \rightarrow bT \mid aS \mid cU$

$U \rightarrow cU \mid c$

* Calcolabilità: Considerare linguaggio $\text{DECIDERTM} = \{ \langle M \rangle : M \text{ è una TM decisore} \}$. Dimostrare che $\text{ATM} \leq_m \text{DECIDERTM}$. Cosa si può affermare della decidibilità di DECIDERTM ?

* Complessità: Dimostrare che se $\text{DTIME}(n^2) = \text{NTIME}(n^2)$, allora $\text{NTIME}(n^5) = \text{TIME}(n^5)$

4 febbraio

* Automi: Si consideri la seguente grammatica G

$S \rightarrow aB \mid bA \mid \epsilon$

$A \rightarrow aS \mid bAA$

$B \rightarrow aBB \mid bS$

Si fornisca derivazione sinistra di $bbaaab$. Si determini PDA che riconosca il linguaggio generato da G

* Calcolabilità: Considerare linguaggio $\text{REJECTTM} = \{ \langle M, w \rangle : M \in \text{TM}, M(w) = \text{rej} \}$. Dimostrare che REJECTTM è turing-ric. Dimostrare poi che lo stesso linguaggio è anche non turing-decidibile usando la diagonalizzazione

* Complessità: Sia A il linguaggio formato da tutte le stringhe $w \in \{a, b\}^*$ t.c. il num. di a contenute in w è il doppio del num. di b . Si descriva TM a singolo nastro che decide A in tempo $O(n^2)$. Si descriva TM multi-nastro che decida A in tempo $O(n)$

4 aprile

* Automi: Definire una grammatica acontestuale equivalente a $L = \{ 0^n 1^m 2^p 3^q : n, m, p, q \geq 0, n > m, p < q \}$

* Calcolabilità: Consideri il problema di determinare se una TM M tenta mai di spostarsi a SX dalla prima posizione del suo nastro mentre è eseguita su input w . Questo corrisponde al linguaggio BUMPS_OFF_LEFTTM delle stringhe $\langle M, w \rangle$ dove M è una TM, w è una stringa dell'input di M e M tenta di spostarsi a SX dalla prima posizione del nastro mentre è eseguita su w . Dimostrare che BUMPS_OFF_LEFTTM è indecidibile

* Complessità: Definiamo il linguaggio $\text{FACTOR} = \{ \langle m, k \rangle : m \text{ ha un fattore primo } p \text{ t.c. } p \leq k \}$, dove m e k sono scritti in binario. Dimostrare che se FACTOR è NP-completo allora $\text{NP} = \text{coNP}$. Nella risoluzione è possibile assumere l'esistenza di un algoritmo di tempo polinomiale per testare se un numero p (scritto in binario) è un primo.

10 giugno

* Automi: Classificare i seguenti linguaggio come 1) regolari, oppure 2) acontestuali ma non regolari oppure 3) non acontestuali. Giustificare le risposte

a) $\{ w\#u : w, u \in \{0, 1\}^* \text{ e } |w| = |u| \}$

b) $\{ w \in \{0, 1\}^* : w \text{ contiene un num. dispari di "0" e un singolo "1"} \}$

c) $\{ w \in \{0, 1, 2\}^* : w \text{ contiene più simboli "0" che simboli "1" e più simboli "0" che simboli "2"} \}$

* Calcolabilità: Si consideri il linguaggio $\text{ALLNFA} = \{ \langle N \rangle : N \in \text{NFA e } L(N) = \Sigma^* \}$. Dimostrare che ALLNFA è turing-decidibile

* Complessità: Considerare i linguaggi $L_1, L_2 \in \text{NP}$ e $S_1, S_2 \in \text{coNP}$. Assumendo che $\text{NP} \neq \text{coNP}$, rispondere alle seguenti domande fornendo una dimostrazione o controesempio:

a) Il linguaggio $L_1 \cup L_2$ è necessariamente in NP? il linguaggio $L_1 \cup S_1$ è necessariamente in coNP?

b) Il linguaggio $L_1 \cap L_2$ è necessariamente in NP? il linguaggio $\text{not} L_1 \cap S_2$ è necessariamente in coNP?

15 luglio

* Automi: considerare la grammatica G :

$S \rightarrow TaT$

$T \rightarrow aTb \mid bTa \mid TT \mid \epsilon$

Fornire PDA M t.c. $L(M) = L(G)$. Mostrare i passi della computazione di M su input $ababbaa$, specificando il contenuto del nastro, stati e contenuto della pila ad ogni passo di computazione

* Calcolabilità: Considerare il problema di determinare se una TM M modifica il contenuto del nastro in posizione i , quando esegue input w . Questo corrisponde al linguaggio $\langle M, w, i \rangle$ dove M è una TM, w è una stringa dell'alfabeto di input di M , i è un intero positivo e M modifica il valore del simbolo in pos i sul nastro quando è eseguita su input w . Dimostrare che questo linguaggio è indecidibile

* Complessità: Fornire due definizioni della classe di complessità NL: tramite non-determinismo e tramite certificati. Dimostrare che le due definizioni sono equivalenti

9 settembre

* Automi: Stabilire se i seguenti linguaggi sono regolari:

- a) $\{a^n b^m c^n d^m : n, m \geq 0\}$
- b) $\{a^n c^m b^n : n > m \geq 0\}$
- c) $\{w \in \{a, b, c\}^* : \#a(w) = \#b(w) \text{ e } \#a(w) > \#c(w)\}$, dove $\#x(w)$ indica il num. di occorrenze del simbolo $x \in \{a, b, c\}$ nella stringa w
- d) $\{w\#t\#w^R : w, t \in \{a, b\}^* \text{ e } |w| = |t|\}$, dove w^R rappresenta la stringa w rovesciata

- * Calcolabilità: Sia $\text{ACCEPTS}_{\epsilon}\text{TM}$ il linguaggio delle stringhe $\langle M \rangle$ dove M è una TM che accetta la stringa vuota ϵ . Mostrare che $\text{ACCEPTS}_{\epsilon}\text{TM}$ è indecidibile
- * Complessità: Dimostrare che $\text{NTIME}(n^k) \subseteq \text{PSPACE}$ per ogni $k \in \mathbb{N}$. Stabilire se questo fatto implica o meno che $\text{NP} \subseteq \text{PSPACE}$

5 novembre

- * Automi: Sia $A = \{a^n b^n : n \geq 0\}$ e sia $B = \text{not}A$. Definire grammatica acontestuale per B usando la proprietà di chiusura dei linguaggi acontestuali rispetto all'unione

- * Calcolabilità: Sia INFINITETM il linguaggio di stringhe della forma $\langle M \rangle$ dove M è una TM e $L(M)$ contiene infinite stringhe. Mostrare che INFINITETM è indecidibile
- * Complessità: Mostrare che se f, g e t sono funzioni tempo costruttibili e $\text{DTIME}(f(n)) = \text{DTIME}(g(n))$ allora $\text{DTIME}(f(t(n))) = \text{DTIME}(g(t(n)))$. Usare questa proprietà per dimostrare che $\text{DTIME}(n) \neq \text{DTIME}(n \log(n))$

2026

12 gennaio

- * Automi: Si progetti un DFA con un singolo stato finale che riconosca il linguaggio corrispondente alla seguente grammatica:

$S \rightarrow 0S \mid 1X \mid \epsilon$
 $X \rightarrow 1X \mid 2X \mid 3Y$
 $Y \rightarrow 3Y \mid 0S \mid 2S$

- * Calcolabilità: Si consideri il linguaggio $\text{REGULARTM} = \{\langle M \rangle : L(M) \text{ è regolare}\}$. Dimostrare che $\text{ETM} \leq \text{M REGULARTM}$, dove $\text{ETM} = \{\langle M \rangle : L(M) = \emptyset\}$. Cosa si può affermare della decidibilità di REGULARTM ?
- * Complessità: Dimostrare che se $\text{SPACE}(n \log(n)) = \text{NSPACE}(n \log(n))$, allora $\text{SPACE}(n \log^2(n)) = \text{NSPACE}(n \log^2(n))$